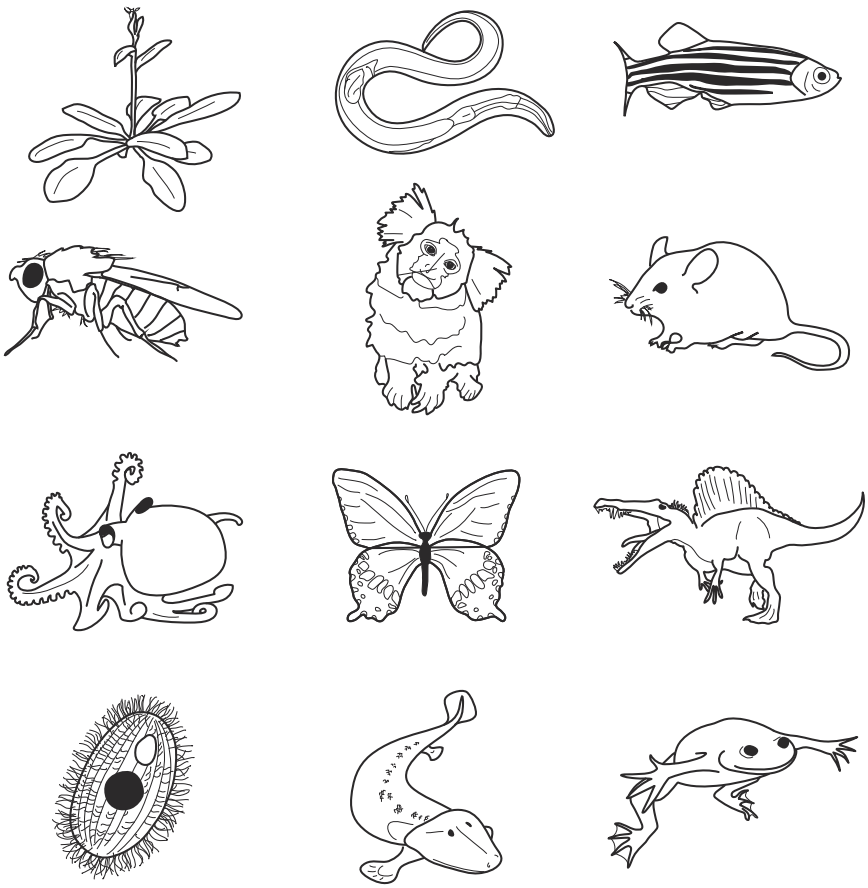
THE UNIVERSITY OF CHICAGO BIOLOGICAL SCIENCES

# BSD qBio$^{11}$ Boot Camp



September 10–17, 2025

Group logos by Professor Stephanie Palmer

# BSD qBio[11]

## General Schedule

### Wednesday, September 10

| | |
|---|---|
| **12:30-1:30** | (Optional) Office hours for help with software installations |
| **1:30-3:30** | BSD-OGPA orientation for new students |
| **3:30-4:00** | Check-in with TAs in BSLC lobby |
| **4:00-4:20** | Welcome to qBio - Novembre / Liu / Myers |
| **4:20-5:00** | Team-building activities |
| **5:00-6:00** | Dinner and group compacts |

### Thursday, September 11

| | |
|---|---|
| **8:30-10:00** | Basic/Advanced comp. I |
| **10:00-10:30** | Coffee break |
| **10:30-12:00** | Basic/Advanced comp. I |
| **12:00-1:30** | Lunch |
| **1:30-2:00** | Welcome to UChicago Biosciences - Dean Kovar |
| **2:00-2:30** | Coffee break |
| **2:30-4:00** | Professional development with TAs |

### Friday, September 12

| | |
|---|---|
| **8:30-10:00** | Basic/Advanced comp. II |
| **10:00-10:30** | Coffee break |
| **10:30-12:00** | Basic/Advanced comp. II |
| **12:00-12:30** | Break + Lunch pick-up |
| **12:30-1:30** | Lunch + Graduate Student Research Talks |
| **2:00-4:00** | Activity - wait for an announcement about this |

### Monday, September 15

| | |
|---|---|
| **8:30-10:00** | Data visualization - Carbonetto (*C.jacchus D.rerio P.polytes T.thermophila*)<br>Defensive programming - Novembre (*A.thaliana C.elegans T.roseae X.laevis*)<br>Statistics for a data rich world - Liu (*D.melanogaster M.musculus O.bimaculoides S.aegyptiacus*) |
| **10:00-10:30** | Coffee break |
| **10:30-12:00** | Data visualization - Carbonetto (*C.jacchus D.rerio P.polytes T.thermophila*)<br>Defensive programming - Novembre (*A.thaliana C.elegans T.roseae X.laevis*)<br>Statistics for a data rich world - Liu (*D.melanogaster M.musculus O.bimaculoides S.aegyptiacus*) |
| **12:00-1:30** | Lunch |
| **1:30-3:00** | Data visualization - Carbonetto (*A.thaliana M.musculus O.bimaculoides X.laevis*)<br>Defensive programming - Novembre (*D.melanogaster D.rerio S.aegyptiacus T.thermophila*)<br>Statistics for a data rich world - Liu (*C.elegans C.jacchus P.polytes T.roseae*) |
| **3:00-3:30** | Coffee break |
| **3:30-5:00** | Data visualization - Carbonetto (*A.thaliana M.musculus O.bimaculoides X.laevis*)<br>Defensive programming - Novembre (*D.melanogaster D.rerio S.aegyptiacus T.thermophila*)<br>Statistics for a data rich world - Liu (*C.elegans C.jacchus P.polytes T.roseae*) |

### Tuesday, September 16

| | |
|---|---|
| **8:30-10:00** | Data visualization - Carbonetto (*C.elegans D.melanogaster S.aegyptiacus T.roseae*) |
| | Defensive programming - Novembre (*C.jacchus M.musculus O.bimaculoides P.polytes*) |
| | Statistics for a data rich world - Liu (*A.thaliana D.rerio T.thermophila X.laevis*) |
| **10:00-10:30** | Coffee break |
| **10:30-12:00** | Data visualization - Carbonetto (*C.elegans D.melanogaster S.aegyptiacus T.roseae*) |
| | Defensive programming - Novembre (*C.jacchus M.musculus O.bimaculoides P.polytes*) |
| | Statistics for a data rich world - Liu (*A.thaliana D.rerio T.thermophila X.laevis*) |
| **12:00-1:30** | Lunch |
| **1:30-3:00** | Workshop - Dev. bio. / shape analysis - Mitchell (*A.thaliana C.elegans D.rerio X.laevis*) |
| | Workshop - Motion tracking / image analysis - Nirody (*C.jacchus O.bimaculoides P.polytes T.thermophila*) |
| | Workshop - Population genetics - Berg (*D.melanogaster M.musculus S.aegyptiacus T.roseae*) |
| **3:00-3:30** | Coffee break |
| **3:30-5:00** | Workshop - Dev. bio. / shape analysis - Mitchell (*A.thaliana C.elegans D.rerio X.laevis*) |
| | Workshop - Motion tracking / image analysis - Nirody (*C.jacchus O.bimaculoides P.polytes T.thermophila*) |
| | Workshop - Population genetics - Berg (*D.melanogaster M.musculus S.aegyptiacus T.roseae*) |

## Wednesday, September 17

| | |
|---|---|
| **8:30-10:00** | Workshop - Dev. bio. / shape analysis - Mitchell (*C.jacchus D.melanogaster T.roseae T.thermophila*) |
| | Workshop - Motion tracking / image analysis - Nirody (*D.rerio M.musculus S.aegyptiacus X.laevis*) |
| | Workshop - Population genetics - Berg (*A.thaliana C.elegans O.bimaculoides P.polytes*) |
| **10:00-10:30** | Coffee break |
| **10:30-12:00** | Workshop - Dev. bio. / shape analysis - Mitchell (*C.jacchus D.melanogaster T.roseae T.thermophila*) |
| | Workshop - Motion tracking / image analysis - Nirody (*D.rerio M.musculus S.aegyptiacus X.laevis*) |
| | Workshop - Population genetics - Berg (*A.thaliana C.elegans O.bimaculoides P.polytes*) |
| **12:00-1:30** | Lunch |
| **1:30-3:00** | Workshop - Dev. bio. / shape analysis - Mitchell (*M.musculus O.bimaculoides P.polytes S.aegyptiacus*) |
| | Workshop - Motion tracking / image analysis - Nirody (*A.thaliana C.elegans D.melanogaster T.roseae*) |
| | Workshop - Population genetics - Berg (*C.jacchus D.rerio T.thermophila X.laevis*) |
| **3:00-3:30** | Coffee break |
| **3:30-5:00** | Workshop - Dev. bio. / shape analysis - Mitchell (*M.musculus O.bimaculoides P.polytes S.aegyptiacus*) |
| | Workshop - Motion tracking / image analysis - Nirody (*A.thaliana C.elegans D.melanogaster T.roseae*) |
| | Workshop - Population genetics - Berg (*C.jacchus D.rerio T.thermophila X.laevis*) |
| **5:10-5:30** | Wrap-up |
| **5:30-7:00** | Reception with Dean's Council Students |

# Materials

**Opening Tutorials:**

Emphasize basic and advanced computing concepts in R

1. Basic computing
2. Advanced computing

**Tutorials:**

Emphasize general challenges in biological data science

3. Data visualization
4. Defensive programming
5. Statistics for a data-rich world

**Workshops:**

Expand upon and apply techniques and ideas from the tutorials in the context of specific disciplinary challenge areas

6. Developmental biology workshop
7. Population genetics workshop
8. Movement analysis workshop

# Base R
Cheat Sheet

## Getting Help

### Accessing the help files

**?mean**
Get help of a particular function.
**help.search('weighted mean')**
Search the help files for a word or phrase.
**help(package = 'dplyr')**
Find help for a package.

### More about an object

**str(iris)**
Get a summary of an object's structure.
**class(iris)**
Find the class an object belongs to.

## Using Packages

**install.packages('dplyr')**
Download and install a package from CRAN.

**library(dplyr)**
Load the package into the session, making all its functions available to use.

**dplyr::select**
Use a particular function from a package.

**data(iris)**
Load a built-in dataset into the environment.

## Working Directory

**getwd()**
Find the current working directory (where inputs are found and outputs are sent).

**setwd('C://file/path')**
Change the current working directory.

**Use projects in RStudio to set the working directory to the folder you are working in.**

## Vectors

### Creating Vectors

| | | |
|---|---|---|
| c(2, 4, 6) | 2 4 6 | Join elements into a vector |
| 2:6 | 2 3 4 5 6 | An integer sequence |
| seq(2, 3, by=0.5) | 2.0 2.5 3.0 | A complex sequence |
| rep(1:2, times=3) | 1 2 1 2 1 2 | Repeat a vector |
| rep(1:2, each=3) | 1 1 1 2 2 2 | Repeat elements of a vector |

### Vector Functions

**sort(x)**
Return x sorted.
**rev(x)**
Return x reversed.
**table(x)**
See counts of values.
**unique(x)**
See unique values.

### Selecting Vector Elements

#### By Position

| | |
|---|---|
| x[4] | The fourth element. |
| x[-4] | All but the fourth. |
| x[2:4] | Elements two to four. |
| x[-(2:4)] | All elements except two to four. |
| x[c(1, 5)] | Elements one and five. |

#### By Value

| | |
|---|---|
| x[x == 10] | Elements which are equal to 10. |
| x[x < 0] | All elements less than zero. |
| x[x %in% c(1, 2, 5)] | Elements in the set 1, 2, 5. |

#### Named Vectors

| | |
|---|---|
| x['apple'] | Element with name 'apple'. |

## Programming

### For Loop

```
for (variable in sequence){
   Do something
}
```

Example

```
for (i in 1:4){
   j <- i + 10
   print(j)
}
```

### While Loop

```
while (condition){
   Do something
}
```

Example

```
while (i < 5){
   print(i)
   i <- i + 1
}
```

### If Statements

```
if (condition){
   Do something
} else {
   Do something different
}
```

Example

```
if (i > 3){
   print('Yes')
} else {
   print('No')
}
```

### Functions

```
function_name <- function(var){
   Do something
   return(new_variable)
}
```

Example

```
square <- function(x){
   squared <- x*x
   return(squared)
}
```

## Reading and Writing Data

Also see the **readr** package.

| Input | Ouput | Description |
|---|---|---|
| df <- read.table('file.txt') | write.table(df, 'file.txt') | Read and write a delimited text file. |
| df <- read.csv('file.csv') | write.csv(df, 'file.csv') | Read and write a comma separated value file. This is a special case of read.table/ write.table. |
| load('file.RData') | save(df, file = 'file.Rdata') | Read and write an R data file, a file type special for R. |

### Conditions

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| a == b | Are equal | a > b | Greater than | a >= b | Greater than or equal to | is.na(a) | Is missing |
| a != b | Not equal | a < b | Less than | a <= b | Less than or equal to | is.null(a) | Is null |

## Types

Converting between common data types in R. Can always go from a higher value in the table to a lower value.

| | | |
|---|---|---|
| `as.logical` | `TRUE, FALSE, TRUE` | Boolean values (TRUE or FALSE). |
| `as.numeric` | `1, 0, 1` | Integers or floating point numbers. |
| `as.character` | `'1', '0', '1'` | Character strings. Generally preferred to factors. |
| `as.factor` | `'1', '0', '1', levels: '1', '0'` | Character strings with preset levels. Needed for some statistical models. |

## Maths Functions

| | | | |
|---|---|---|---|
| `log(x)` | Natural log. | `sum(x)` | Sum. |
| `exp(x)` | Exponential. | `mean(x)` | Mean. |
| `max(x)` | Largest element. | `median(x)` | Median. |
| `min(x)` | Smallest element. | `quantile(x)` | Percentage quantiles. |
| `round(x, n)` | Round to n decimal places. | `rank(x)` | Rank of elements. |
| `signif(x, n)` | Round to n significant figures. | `var(x)` | The variance. |
| `cor(x, y)` | Correlation. | `sd(x)` | The standard deviation. |

## Variable Assignment

```
> a <- 'apple'
> a
[1] 'apple'
```

## The Environment

| | |
|---|---|
| `ls()` | List all variables in the environment. |
| `rm(x)` | Remove x from the environment. |
| `rm(list = ls())` | Remove all variables from the environment. |

**You can use the environment panel in RStudio to browse variables in your environment.**

## Matrices

`m <- matrix(x, nrow = 3, ncol = 3)`
Create a matrix from x.

`m[2, ]` - Select a row

`m[ , 1]` - Select a column

`m[2, 3]` - Select an element

`t(m)`
Transpose

`m %*% n`
Matrix Multiplication

`solve(m, n)`
Find x in: m * x = n

## Lists

`l <- list(x = 1:5, y = c('a', 'b'))`
A list is a collection of elements which can be of different types.

| `l[[2]]` | `l[1]` | `l$x` | `l['y']` |
|---|---|---|---|
| Second element of l. | New list with only the first element. | Element named x. | New list with only element named y. |

Also see the **dplyr** package.

## Data Frames

`df <- data.frame(x = 1:3, y = c('a', 'b', 'c'))`
A special case of a list where all elements are the same length.

| x | y |
|---|---|
| 1 | a |
| 2 | b |
| 3 | c |

**List subsetting**

`df$x`

`df[[2]]`

*Understanding a data frame*

| | |
|---|---|
| `View(df)` | See the full data frame. |
| `head(df)` | See the first 6 rows. |

**Matrix subsetting**

`df[ , 2]`

`df[2, ]`

`df[2, 2]`

`nrow(df)`
Number of rows.

`ncol(df)`
Number of columns.

`dim(df)`
Number of columns and rows.

**`cbind`** - Bind columns.

**`rbind`** - Bind rows.

## Strings

Also see the **stringr** package.

| | |
|---|---|
| `paste(x, y, sep = ' ')` | Join multiple vectors together. |
| `paste(x, collapse = ' ')` | Join elements of a vector together. |
| `grep(pattern, x)` | Find regular expression matches in x. |
| `gsub(pattern, replace, x)` | Replace matches in x with a string. |
| `toupper(x)` | Convert to uppercase. |
| `tolower(x)` | Convert to lowercase. |
| `nchar(x)` | Number of characters in a string. |

## Factors

`factor(x)`
Turn a vector into a factor. Can set the levels of the factor and the order.

`cut(x, breaks = 4)`
Turn a numeric vector into a factor by 'cutting' into sections.

## Statistics

`lm(y ~ x, data=df)`
Linear model.

`glm(y ~ x, data=df)`
Generalised linear model.

`summary`
Get more detailed information out a model.

`t.test(x, y)`
Perform a t-test for difference between means.

`pairwise.t.test`
Perform a t-test for paired data.

`prop.test`
Test for a difference between proportions.

`aov`
Analysis of variance.

## Distributions

| | Random Variates | Density Function | Cumulative Distribution | Quantile |
|---|---|---|---|---|
| Normal | `rnorm` | `dnorm` | `pnorm` | `qnorm` |
| Poisson | `rpois` | `dpois` | `ppois` | `qpois` |
| Binomial | `rbinom` | `dbinom` | `pbinom` | `qbinom` |
| Uniform | `runif` | `dunif` | `punif` | `qunif` |

## Plotting

Also see the **ggplot2** package.

`plot(x)`
Values of x in order.

`plot(x, y)`
Values of x against y.

`hist(x)`
Histogram of x.

## Dates

See the **lubridate** package.

# Basic Computing at qBio11*

**Peter Carbonetto** *University of Chicago*

The aim of this tutorial is to introduce R and to use R to analyze a biological data set. We will analyze the data interactively while also developing Good Practices for data analysis, including writing an *R script*. This workshop is intended for biologists with little to no background in programming. **This tutorial is also a Google doc.**

## How this tutorial is organized

Basic Computing is split into 3 parts:

1. Understanding R, the real fundamentals
2. Understanding R, working with more complex data structures
3. An exploratory data analysis

## What is this document, and how should I use it?

This is an R Markdown document and a Google doc (bit.ly/3UlIl7s) containing text and R code. You can run the code by copying & pasting it into your favorite R IDE ("integrated development environment"). Here we will use the RStudio IDE. (Popular online IDEs include Posit Cloud and Google Colab.)

In RStudio, we will mainly use two "panes": the "Source pane" and the "Console pane".

For some practice, try running these two lines of code:

```
x <- rnorm(200)
hist(x,n = 32)
```

You may notice that your histogram is not the same as mine. *Why is that?*

*Make this document your own* by making a copy of it in your Google Drive, then add your own notes and code. And of course use the workbook for your notes. I've also given you permission to add comments to this Google doc.

## Part 1: Understanding R, the real fundamentals

Now I want us to revisit R from the ground up. Some of you may be already familiar with R or perhaps other programming languages. Even if you are already familiar with R, sometimes it is

*This document is included as part of the Basic Computing—Introduction to R tutorial packet for the BSD qBio Bootcamp, University of Chicago, 2025. **Current version**: August 11, 2025; **Corresponding author**: pcarbo@uchicago.edu. Thanks to Stefano Allesina, John Novembre, Stephanie Palmer and Matthew Stephens for their support and guidance.

helpful to revisit and strengthen your understanding of the programming language. Later on, we will analyze a real data set and apply our R skills to that data set. But for now I want to focus on building skills and getting comfortable with the R environment.

### R is like math

There is a close relationship between math and programming languages: many or most expressions in R look very much like mathematical expressions. Let's take this expression as an example:

```
sqrt((x1 - x2)^2 + (y1 - y2)^2)
```

*What does this expression do?* Break down the mathematical operations step-by-step.

But, when you try to evaluate this expression, you will get an error that "x is not found".

Let's define x and y and try to evaluate that expression again:

```
x1 <- 1
y1 <- 1
x2 <- 2
y2 <- 2
sqrt((x1 - x2)^2 + (y1 - y2)^2)
```

Let's just use our intuition here to write with another expression in R, the distance between 2-d points x and y in "taxicab geometry":

```
abs(x1 - x2) + abs(y1 - y2)
```

### R's single most powerful feature: making your own functions is easy

*Functions in R are everywhere.* We have used at least two already, sqrt and abs. (Side note: almost every function in R is documented. If you want to know what a function does and how to use it, simply type `help(name_of_function)`, e.g., `help(sqrt)`.) But you aren't limited by R's existing functions: R is easily extensible by *creating your own functions.* For example, it is tedious to write the distance between two 2-d points every time you want to do this. One way to make this calculation more efficient is to create a new function that performs this calculation inside it and then returns the result:

```
euclidean_dist <- function (x1, y1, x2, y2) {
  d <- sqrt((x1 - x2)^2 + (y1 - y2)^2)
  return(d)
}
```

We have created a new object in our environment *that is a function* (yes, functions in R are objects, a fact that is also very powerful, but something we will not explore immediately):

```
ls()
euclidean_dist
```

Now that we have created this function, we can use it on its own, or by combining it with other mathematical expressions:

```
euclidean_dist(1,1,2,2)
euclidean_dist(-1,-1,1,1)
euclidean_dist(6*7,4,3/4,exp(1))
sin(euclidean_dist(1,1,2,2))
```

You will create functions in the other tutorials and workshops.

*Exercise:* Create a function called "taxicab_dist" to compute the the taxicab distance between two points in 2 dimensions, and then perform some calculations with it.

*Discuss:* Why was it useful or why could it be useful to create a function to perform your calculations?

*Followup discussion:* Custom functions are one of the most underused and underappreciated features of R. Why do you think this feature is so underused?

### Two evaluation modes in R

There are two ways that R evaluates expressions: *calculator mode* and *environment building mode.*

We have actually seen examples of both. Let's revisit the lines of code we have evaluated to understand the differences between the two modes.

### Data vectors

Perhaps without knowing it, we have already been working with data vectors: for example, the object x1 is a vector of length 1 (sometimes called a "scalar").

Almost everything in R works on *vectors* of any length. So once you get comfortable making calculations on single numbers ("scalars"), it is often very natural to apply your same calculations to vectors. (In fact, over time, you will find it so natural that you may take it for granted!) In order to work with data vectors, it is helpful to know a few useful functions for *creating* vectors.

The most flexible function is the "combine" function, or "c" for short:

```
x <- c(0, 7, 1e-6, -50, 2^5, 1/3, 2*pi)
```

Another useful function is "rep", which can be used to quickly create very large data vectors:

```
x <- rep(1, 100)
```

And "seq", short for "sequence":

```
x <- seq(1,100)
x <- seq(1,100,5)
```

These functions can (of course!) be used in combination in many different ways, e.g.,

```
x <- rep(seq(1,3),10)
x <- c(seq(1,10),-1,rep(0,4))
```

Another interesting function for creating data vectors creates random data vectors:

```
x <- sample(10)
x <- sample(1000,10)
x <- sample(seq(0,100,2),10)
```

Use the square brackets to access an individual element of a data vector:

```
x[1]
x[2]
```

And "length" to get the number of elements in the vector:

```
length(x)
```

There is also a special vector in R, the "null vector", which is a vector of length zero:

```
x <- NULL
length(x)
```

Surprisingly, the "null vector" can be useful! (You will see an example of this later.)

R operates on data vectors in two different ways: (i) by applying the calculation separately to each element of the vector, or (ii) by aggregating across the elements. Here are some examples of the first way:

```
x <- seq(1,10)
x / 10
sqrt(x)
abs(x)
x + 1
y <- sample(10)
x + y
abs(x - y)
```

Here are some examples of the second way:

```
sum(x)
mean(x)
max(x)
min(x)
which.max(x)
which.min(x)
```

These last two examples are rather curious... *what is the value returned from the last two examples?*

And here are some examples that combine the two ways:

```
mean(x - y)
max(abs(x - y))
```

This is R's **invisible magic**. *Why do I call it R's "invisible magic"?*

### In-class activity: random walks

Through an in-class activity, we will explore these distance calculations further, and in the process learn about **loops**, **if-then-else**, and how to simulate data in R and store the simulated data in vectors.

We will work on this activity by writing an **R script**. The first step is to create a new .R file in RStudio and give it a useful name that you will remember (e.g., random_walk.R).

A powerful and ubiquitous idea in quantitative biology is the design of simulations to understand natural phenomena. One of the simplest and oldest simulations is the "random walk", also sometimes called the "drunkard's walk". (Incidentally, I highly recommend the book by Leonard Mlodinow, The Drunkard's Walk: How Randomness Rules Our Lives.) Here we will simulates a 1-d random walk.

Here is a script to get you started:

```
n <- 100
x0 <- 0
x <- x0
locations <- NULL
times <- seq(1,n)
for (i in times) {
  coin <- sample(2,1)
  if (coin == 2) {
    x <- x + 1
  } else {
    x <- x - 1
  }
  locations <- c(locations,x)
}
plot(times,locations,type = "l")
```

*First exercise:* Add some code to your script to determine the time at which the random walk is furthest away from the starting point. (Which distance will you use? Euclidean distance, taxicab distance, . . . ?)

*Second exercise:* Use R's "invisible magic" to simplify the calculation of the largest distance to the starting point, and the time in which it occurred. I recommend creating a new R script for this exercise so that you can compare your previous distance calculation to the new one. (To verify that you are getting the same result, you can add a `set.seed(1)` to your script to make the results "reproducible".)

*Third exercise (time permitting):* Write another script, e.g., random_walk_2d.R, to simulate a 2-d random walk using a 4-sided die. Similar to before, using R's "invisible magic", determine the the largest distance to the starting point, and the time in which it occurred.

## Part 2: Understanding R, working with more complex data structures

In this part we will learn about some more complex data structures and how to work with them through the analysis of a real data set from a paper published in *Genetics.*

### My data analysis

Here is my data analysis. It is an analysis of data from a 2008 Genetics article on the genetics of dog breeds:

```
dogs <- read.csv("dogs.csv",stringsAsFactors = FALSE)
fit <- lm(aod ~ weight,dogs)
print(fit)
```

You will notice that the code is very short! There's nothing wrong with that—successful analyses in R do not need to be long or complicated! Still, we will spend quite some time understanding this code and what it does, and in the process we will learn about R. But before we try to understand what the code is doing, let's start by trying to run the code in RStudio to reproduce the result.

### A mini-course on data frames

A data frame is R's main data structure for *storing tabular data.* The data frame is one of the most important data structures in R, and is important enough that we will spend much of this tutorial seeking to understand how to work with and analyze data in data frames. (Not all data of course is tabular data, but because so many things in R work well with data frames, *it can be helpful to find ways to rework your data so that it fits into a data frame.*)

Before we get to more interesting things, we need to first get comfortable with some basic syntax for data frames.

```
print(dogs)
head(dogs)
tail(dogs)
```

```
summary(dogs)
class(dogs)
nrow(dogs)
ncol(dogs)
names(dogs)
dogs$breed
dogs$aod
dogs$height
dogs$weight
dogs[,"aod"]
dogs[1,]
dogs[,"weight"]
dogs[1,"weight"]
x <- dogs[1,"weight"]
x <- dogs[,"weight"]
x <- dogs[1,]
```

You may have noticed that many of these lines of code *do the exact same thing*. This is a common theme in R (and in programming more generally): *there are often many different ways of accomplishing the same thing, and there is rarely one way that is "best"* (although sometimes R programmers get into passionate debates about this).

A reminder at this point that the advantages of analyzing data in R may be less obvious when working with a small a data set. Later when we work with a large data set the benefits of R will become more clear.

### Checking for mistakes

All researchers, no matter how much experience they have, make mistakes. And most mistakes are stupid mistakes! (I have made my fair share of embarrassing mistakes.) Occasionally, very good researchers make mistakes in their papers. (See also here for a discussion of this mistake years later.) Therefore, *try to catch your mistakes early.*

So how can you find your mistakes? The good news is that R catches many of your mistakes: most methods in R have their own internal error-checking. Discussing your results with your labmates and advisors is also another way to catch mistakes. But ultimately you will need to develop some skills and strategies for performing your own checks.

A relatively simple but nonetheless helpful check is a "sanity check": it doesn't tell you the result is right, but at least it reassures you that the result is not horribly wrong. A good sanity check is simple—one that you can do on "the back of an envelope". Here we will perform a simple sanity check for our analysis of the dogs data (and in the process we will learn more about R).

Here our "sanity check" will be to hand-check that our model, $y = ax + b$, where $x$ = weight and $y$ = aod, is making sensible predictions of $y$ given $x$. This will involve some basic arithmetic so in the process we will learn about how to do arithmetic in R. Also, plots are another powerful way to perform checks, and we will write some simple code to visualize our results.

*Your code for checking your results.*

### In-class activity: Re-run this analysis on a different data set

Let's now try something *a bit audacious.* Let's try to re-run the same analysis as before, but on a different data set from the American Kennel Club. Will this new analysis produce a similar result, or not?

Using the skills you have developed so far—and a bit of creativity—I believe you can adapt the code you ran on the "dogs" CSV file to analyze the AKC data (the file is akc_data.csv). This exercise will involve making some judgments about how to analyze the data and therefore I do not expect everyone to get the same result. It will also involve some initial explorations of the data to understand what this data set contains, and how it differs from the first data set. *Do not be afraid to make mistakes!*

Note that you may encounter a challenge that you have not yet dealt with. When you encounter this challenge, try to figure out what is the issue, and we will discuss as a group how to overcome it.

*Your code for analyzing the AKC data.*

### Tending to your gaRden

*(Almost) every line of code in R acts on your R environment: it takes objects that exist in your environment, then generate new objects or overwrites existing objects.* Therefore, to understand any line of code, you need to understand not only the code itself, but also what is the state of your environment (your "garden") the moment before you run your code; that is, the objects that are in your environment and what they represent. However, this is easier said than done when your environment (garden) is messy (not tended to). *Therefore an important skill as a coder is to tend to your gaRden.*

*Your code for tending to your gaRden.*

*What are all these objects? Could we have named some of these objects better to remind us what they are?*

*S. cerevisiae*

### Programming challenge: facts about dog breeds

The data frame is R's way of storing tabular data. It is one of the most important and more powerful data structures in R. In this Programming Challenge, we will take some time to understand data frames and how to work with them. Although our focus here is on data frames, many of the ideas you will pick up in this Programming Challenge are quite general.

Let's start this part of the tutorial with a clean environment. Then go ahead and import the dogs data set and print out the first few rows of the table:

```
rm(list = ls())
dogs <- read.csv("dogs.csv",stringsAsFactors = FALSE)
head(dogs)
```

The data frame is a data structure for storing tabular data. The data are actually stored in a very specific way: the data frame is a set of columns, and each column is a vector of the same length. Let's run some code to convince ourselves of this fact.

First, for convenience, make a copy of the first column, and call it "x":

```
x <- dogs$breed
```

**Important note:** This makes a copy of the original data, so if you were to modify or delete x, this leaves "dogs" unchanged.

This is a "character" data type. It is R's way of storing text data:

```
class(x)
length(x)
x
```

Note we could have also copied the first column this way:

```
x <- dogs[,"breed"]
```

And here's another way!

```
x <- dogs[,1]
```

*Which way do you prefer?*

By storing tabular data in this way, we can *divide and conquer:* since each column is also an object in its own right, if the data are too complicated to understand all at once, we can make a copy of

the columns we want to look at more closely, and run code on the copy. *This is a useful strategy for dealing with complex data sets.*

To drive home this idea of a data frame as a collection of vectors, the way to create a data frame is in fact to join together a bunch of vectors of the same length. For example:

```
mydogs <- data.frame(
  breed = dogs$breed,
  lbs = dogs$weight,
  years = dogs$aod)
head(mydogs)
```

Each data structure in R has its own features and its own techniques for working with them. In time, you will learn to work with other types of data structures: some are used widely (e.g., an "lm" object), and some are very specialized (e.g., a GRanges object).

Now that we have some basic understanding of what is a data frame, let's jump into the Programming Challenge: the goal is to write code to answer some basic questions about dog breeds from the data. The challenges will get progressively more difficult, and will build on each other, so try not to rush through them. Sometimes the code will be given to you, other times you will be given hints for writing the code to answer the questions. Now, this data set is small enough that you can answer many of these questions by eye, but please don't do that. (That being said, you are welcome to look at the data to verify your answers.)

Before diving deeply into the Programming Challenge questions, first discuss a collaboration strategy with your teammates. How will you work on the problems together? How will you share and discuss solutions? (Maybe do this in a shared Google doc?) How will you make sure that everyone is included in the problem solving? How will you address conflicts, e.g., when your team comes up with more than one solution?

*Warmup: Smallest and largest dog breeds*

This should find the average height (in inches) of the largest dog breed:

```
x <- dogs$height
max(x)
```

Now write code to find the (average) height of the smallest dog breed:

*Add your code here.*

This didn't tell us which breeds were the smallest and largest. For example, to find the largest breed, we can do this:

```
y <- dogs$breed
i <- which.max(x)
y[i]
```

The output of which.max() was stored in object "i". How kind of object is "i"?

What is the smallest breed?

*Add your code here.*

*What objects did you create to answer this question?*

*Another warmup: inspecting data about specific dog breeds*

Suppose you wanted to look more closely at the height, weight and other statistics of the Alaskan Malamute, which is stored in the fifth row of the data frame. This is easily done by selecting the fifth row with the square brackets:

```
dogs[5,]
```

You can also select several rows at once, e.g.,

```
dogs[c(43,46),]
```

Practice a few times selecting different combinations of rows. What happens if you select a row number that is larger than the height of the table?

*Facts about dogs' BMI*

The body-mass index (BMI) is a standard quantity—and sometimes misused quantity!—in science and medicine. In R, the BMI is easily calculated:

```
w <- dogs$weight
h <- dogs$height
bmi <- 703*w/h^2
```

Based on this code, what is the mathematical formula for BMI?

Next, write some code to find the largest, smallest, mean and median BMI. What are the dog breeds with the largest and smallest BMI?

*Add your code here.*

If you would like to use the BMI data later on, you can insert these data in the data frame:

```
dogs$bmi <- bmi
```

*(What is the benefit of adding these data to the data frame as opposed to storing the BMI data in a separate object?)*

*The longest-living dog breeds*

In the dogs data frame we encountered two types of data: numeric data and text data. Another type of data that is very important is *logical data.* Although the data frame does not contain logical data, we can easily create logical data using *logical operators.*

You might create logical data in the process of answering questions about the data. For example, suppose you would like to know how many dogs have an expected longevity of 16 years or greater. Here is some code to answer this question:

```
x <- dogs$aod >= 16
summary(x)
```

*What is "x" here?*

```
class(x)
x
```

To get the indices that are "TRUE", use which():

```
i <- which(x)
length(i)
i
dogs[i,]
```

What does "i" contain?

Other logical operators include equals (==), and (&), or (|) and not (!). Running help(Logic) will give you a longer list.

Now write similar code to find the breeds with the following characteristics:

1. Expected age of death (AOD) at least 15 and average weight greater than 20 lbs.
2. AOD greater than 15 and "shortcoat" value of 1. (Later we will learn what the "shortcoat" column represents.)

*Add your code here.*

*A QTL for weight (also, dealing with missing data)*

In the *Genetics* paper, the strongest QTL ("quantitative trait locus") for weight was a QTL on chromosome 7. The "cfa7_46696633bp" column stores the breeds' allele frequencies for these QTL. What happens when you try to calculate the correlation between the allele frequencies and weights?

```
x <- dogs$cfa7_46696633bp
y <- dogs$weight
cor(x,y)
```

In turns out that the allele frequencies were not available for some of the breeds, so a special value "NA" was entered for those breeds. *Why "NA"?*

Use the is.na() function, and other functions you have used before, to find the missing entries, and determine:

1. How many allele frequencies are missing?
2. Which breeds are missing allele frequencies?

*Add your code here.*

The designers of R, appreciating that missing data is widespread in statistics, made sure that missing values were an integral part of the R programming language. Most statistical functions in R can deal with missing data. Read the documentation for "mean" and "cor", then use the guidance provided in the documentation to compute the average allele frequency at the QTL (that is, averaged across all dog breeds), and the correlation between body weight and allele frequency.

*Add your code here.*

*What is a "factor"?*

So far, we have seen three basic data types: character, numeric and logical. There is a fourth important atomic data type in R: *factor.* What is unusual about factors is that—last I checked—

there is no equivalent in other popular programming languages, at least not as a primitive data type. And yet you will find that they are extremely useful.

None of the columns in the dogs data frame are a factor. But, like logical data, *we can create a factor from other data.*

The "shortcoat" column contains numeric data. As it turns out, *it may be more useful to analyze the data as a factor.*

In this part of the Programming Challenge, you will be given the code, and your task will be to run the code and interpret the outputs, with the aim of gaining some intuition for factors, how to use them, and when they may be useful.

Let's first run a few lines of code to inspect the shortcoat data:

```
x <- dogs$shortcoat
class(x)
x
summary(x)
unique(x)
```

Now run the following lines of code to create a factor:

```
x <- factor(x)
class(x)
x
summary(x)
```

Notice that although the data have stayed the same, the two summaries are different. This is because although the data haven't changed, the *data representation* or *encoding* has: *R treats the numeric encoding differently from the factor encoding.*

What does the second summary tell us? Judging by these outputs, what do you think a factor is?

These observations suggest that we can improve the data representation further; the data should be as easy to interpret as possible ("human readable"). Representing the data as zeros and ones may be convenient for the computer, but it is confusing when representing the data as a factor because it "looks" like numeric data, when in fact it is not.

Fortunately, now that the data are stored as a factor, this is easily fixed. To fix this, we modify a property ("attribute") of the object. Since this is the first time we are using an object's attributes, let's first take stock of the object's attributes:

```
attributes(x)
```

What does the "levels" attribute keep track of?

We can modify the levels attribute. For example, this replaces the zeros with "no" and ones with "yes":

```
levels(x) <- c("no","yes")
summary(x)
```

Having made these improvements to the shortcoat data, let's store the improved data in the data frame:

```
dogs$shortcoat <- x
summary(dogs)
```

To illustrate the power of factors, let's see how easily it can be incorporated into a linear regression analysis.

In one of my analyses, I found that dogs with short coats tended not to live quite as long as dogs with longer coats:

```
sx <- dogs$shortcoat
y <- dogs$aod
i <- which(x == "no")
j <- which(x == "yes")
mean(y[i])
mean(y[j])
```

Is this difference significant? We can check this with lm():

```
fit <- lm(aod ~ shortcoat,dogs)
coef(fit)
summary(fit)
```

What does the "shortcoatyes" output from coef(fit) represent? Is the difference significant?

The difference in expected lifespan might be explained better by differences in the body weights between dogs with short and long coats. Is the AOD difference explained by "shortcoat" still significant when weight is included as an explanatory variable for AOD?

```
fit <- lm(aod ~ weight + shortcoat,dogs)
summary(fit)
```

*An index of dog breeds*

There are many situations in biology research in which your data are text data (consider that DNA sequences are a type of text data). The stringi and stringr packages are popular packages for performing more complex analyses of text data. (In computer science, a "string" means a sequence of characters, so one can think of text data as a collection of strings.) In this last question, we will practice some simple manipulation of text data ("strings") to organize the dog breeds by the first letter of the breed name.

The first step is to extract the data we want. This can be done using substr():

```
x <- dogs$breed
d <- substr(x,start = 1,stop = 1)
d
```

Which is the most common first letter for a dog breed? And how many breeds start with this letter? To answer these questions, try creating a factor.

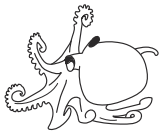
*Add your code here.*

The factor() function automatically determined which letters appeared in the data. But it could be useful to include all the letters in the alphabet, not just the ones that appear in the data. Modify your factor call above to include the unused letters as well. See help(factor) and help(LETTERS) for guidance.

*Add your code here.*

Once you have the new factor using all the letters, write code to determine which letters are not used for the first letter of any dog breed:

*Add your code here.*



*O. bimaculoides*

## Part 3: An exploratory data analysis

Here's the situation. We have generated some data from mouse pancreas samples. We would like to use these data to generate some exciting scientific breakthroughts (obviously!). But before we do that, we need to understand the data. This includes checking our data for errors, and exploring the data. *We will perform all of these steps in R.*

### Setup instructions

1. Create a "project folder" (e.g., pancreas_explorations) where we will store all of the files related to our data analysis, including the data, code and results.
2. In RStudio, change your working directory to the folder you just created. *What is the "working directory" and why do we need to change it?*
3. Create a new R script and give it a memorable name, e.g., pancreas_analysis.R.
4. Add a brief description in comments at the top of the R script, e.g., "This is the code implementing my initial analysis of the pancreas data."
5. Download the data files from Box (bit.ly/4m6pgSS). You should have three files.

### The CSV files

Two of the files you downloaded are CSV files. *What is a CSV file, and what is "CSV" short for? What kind of data can be stored in a CSV file? Why does the CSV file remain a popular way to store data despite the fact that there are more sophisticated alternates (e.g., Excel, HDF5)?*

### The R script

Here we will write code to analyze the data and generate results. The code is as important—or perhaps more important!—than the results themselves.

*Discussion:* Why is the code important? What role(s) does it play in your research?

**Important note:** In this part, we will spend less time understanding the R code. Instead, I want to focus more on the practice the coding and developing the data analysis, which involves a cycle of (i) investigating the data, (ii) leading to a better understanding the data, and then (iii) motivates us to revise our analysis.

### Import the data into R

*Note: put this code in the R script first.*

```
barcodes <- read.csv("pancreas_barcodes.csv")
genes <- read.csv("pancreas_genes.csv")
counts <- readRDS("pancreas_counts.rds")
```

### Internalizing the data and the variables in our computing environment

Here we have three variables so far (soon we will create more). Our immediate goal is to run some code that will help us understand—and *internalize*—these variables.

```
ls()
print(barcodes)
class(barcodes)
```

```
nrow(barcodes)
ncol(barcodes)
head(barcodes)
tail(barcodes)
summary(barcodes)
barcodes$replicate
table(barcodes$replicate)
```

*Now do the same for the "genes" data frame.*

The "counts" variable is a different type of data structure—it is a matrix—so we will want to examine it a little differently.

```
class(counts)
nrow(counts)
ncol(counts)
counts[1:3,1:3]
counts[,1]
counts[1,]
min(counts)
max(counts)
```

Now one thing to note here is that these three variables represent three sets of data, but they are also closely connected.

Indeed, this sort of setup is so common in biology that it has a name and there is software developed specifically to analyze these sorts of data: "annotated data matrix", or "anndata" for short.

The row and column numbers match, but do the actual rows/columns match? First check the genes:

```
head(genes)
counts[1:3,1:6]
```

Anecdotally, it appears so.

The barcodes are harder to check, though:

```
head(barcodes)
counts[1:6,1:3]
```

How can we *automate* this check? This involves slightly more complicated R code:

```
barcodes$barcode == rownames(counts)
all(barcodes$barcode == rownames(counts))
```

Now take some of this code that we have used to explore the data and add it to the script to print out basic stats on the data set (here are provide some sample code, but it doesn't have to be exactly this).

```r
cat("number of cells:\n")
print(nrow(counts))
cat("number of genes:\n")
print(ncol(counts))
cat("number of cells from each replicate:\n")
print(table(barcodes$replicate))
```

### Thinking more about the data and our data exploration aims

What are these data, and what should we do with them? A few thoughts:

1. Each row in the matrix is a gene expression profile of an individual cell in the pancreas at a moment in time.
2. Let's stop and think how incredibly powerful this is: being able to capture the state of a single cell, and gather a massive amount of information of a single cell, was something unimaginable a few years ago. The technology is called "single-cell RNA-sequencing", but really it is not one technology, but rather the accumulation of many scientific and engineering breakthroughs
3. The individual entries of the matrix give the number of "reads" for that gene. It is a measurement of the level of gene transcription in that cell at that point in time.
4. What is the purpose of collecting these data? The exciting possibility is that we can use these data to reconstruct the pancreas from the ground up. This reconstruction should, first of all, confirm our existing understanding of the pancreas. But perhaps it will show us something unexpected and new discoveries—perhaps it will reveal are some types of cells in the pancreas that we did not know existed? Or perhaps some cells that are acting in unexpected ways? How to make sense of these data remains an unresolved question in the scientific community. But we can still try to take some simple steps to extract some understanding from the data, and perhaps even discover something interesting.
5. Single-cell RNA-sequencing produces a huge amount of information. But more is not always better; we need to also be on guard for poor quality, misleading, irrelevant, or "noisy" data.
6. It is much easier to assess the data when have some idea of what to look for. So let's start with some things that we know about the pancreas before we try to discover completely new things. (Also, it is helpful to check that the data confirms our basic expectations about the pancreas—if it doesn't, that might be a cause for concern!) For example, we should find acinar and ductal cells that produce and move the pancreas's digestive enzymes. We should also find different types of islet cells such as $\alpha$ and $\beta$ cells which are the endocrine ("hormone-producing") cells in the pancreas (the $\beta$ cells in particular produce insulin). Do these data contain gene measurements from these sorts of cells?

*Run this code first in the console, then add some of it to the script.*

```r
colnames(counts) <- genes$symbol
counts[,"Krt19"]
x <- counts[,"Krt19"]
hist(x,n = 64)
```

Second attempt:

```
x <- log10(counts[,"Krt19"])
hist(x,n = 64)
```

Third attempt:

```
x <- log10(counts[,"Krt19"] + 1)
hist(x,n = 64)
```

*Why take the log-counts? What are the implications? Why use the base-10 logarithm (as opposed to, say, base-2 or base-e)?*

Let's now search for the insulin-producing $\beta$ cells:

```
x <- log10(counts[,"Ins1"] + 1)
hist(x,n = 64)
```

*Add some of this code to the R script to keep track our progress in exploring these data.*

### Data-driven discovery

So far, we have searched for cells based on prior knowledge of the pancreas. Can we instead make discoveries *using the data alone? How do we do that?*

Typically, we use statistical or machine learning techniques that identify patterns or structure from data. Two very commonly used techniques are PCA (principal component analysis) and clustering. Another more recent technique is a *nonlinear embedding* technique called UMAP. The intuition behind UMAP is that if some cells have similar gene expression profiles, the UMAP will "project" these cells nearby each other on a 2-d map, and so they will appear as a "blob" or "cluster" in 2-d space. But of course biology is complicated... so let's see what happens.

There is also a *computational challenge* in working with these data: it is *a lot* of data, so we will need to use advanced numerical techniques to work with data of this size.

### Reducing the size of the data using PCA

The first "trick" is to use a very simple technique, PCA (equivalently, SVD), to reduce the size of the data so that it is more manageable. But even running PCA is challenging on a data set of this size, so we will use a randomized algorithm implemented in an R package called "rsvd".

```
library(rsvd)
set.seed(1)
shifted_log_counts <- log(counts + 1)
pca <- rpca(shifted_log_counts,k = 30)
```

Out of curiosity, let's plot the top two PCs:

```
x <- pca$x[,1]
y <- pca$x[,2]
plot(x,y,pch = 20)
```

Note that the output is a more complex data structure called a "list":

```
class(pca)
names(pca)
class(pca$x)
nrow(pca$x)
ncol(pca$x)
pca$x[1:4,1:4]
```

### Computing a nonlinear 2-d embedding of the cells

Here we will use the "uwot" R package to run UMAP:

```
library(uwot)
set.seed(1)
res <- umap(pca$x,metric = "cosine",n_neighbors = 30,
            min_dist = 0.3,verbose = TRUE)
```

The "umap" function is an example of a complex function with many input arguments, and learning to use this function well is a "mini-expertise" in itself. Learning to use this function well is obviously beyond the scope of this tutorial. (I'll just note that in R it is easy to access the documentation for this function with the "help" feature.) Although seemingly overwhelming in its complexity, most of the inputs to the umap function have sensible defaults that are expected to work well a lot of the time. But a more in-depth understanding might be needed if this result is important for your work and you are not satisfied with the initial result. (As you can see, in our code we did override some of the defaults.)

The result from umap is a matrix, which we are already a little bit familiar with:

```
class(res)
nrow(res)
ncol(res)
head(res)
```

Let's create a plot similar to the PCA plot above:

```
x <- res[,1]
y <- res[,2]
plot(x,y,pch = 20,xlab = "UMAP 1",ylab = "UMAP 2")
```

The UMAP plot appears to be revealing tons of interesting structure, including small and large "blobs" of cells, some that are quite disconnected from the others.

**Note:** Later we will learn better ways to create plots using the ggplot2 package.

### Save the results

Let's call this version 1.0 of our data analysis.

Your advisor has asked to bring the results of your analysis to your one-on-one with him next week. What should you bring? You have already diligently saved your code. What about the results—what results should you save? Write some code to save the UMAP results:

```
plot(x,y,pch = 20,xlab = "UMAP 1",ylab = "UMAP 2")
dev.copy2pdf(file = "pancreas_umap.pdf",height = 4.5,width = 4)
write.csv(res,"pancreas_umap.csv",quote = FALSE)
```

Your R script might look something like this (this is the minimal version):

```
library(rsvd)
library(uwot)
set.seed(1)
barcodes <- read.csv("pancreas_barcodes.csv")
genes <- read.csv("pancreas_genes.csv")
counts <- readRDS("pancreas_counts.rds")
s <- rowSums(counts)
i <- which(s > 5000)
barcodes <- barcodes[i,]
counts   <- counts[i,]
s <- rowSums(counts)
s <- s/mean(s)
shifted_log_counts <- log(counts/s + 1)
pca <- rpca(shifted_log_counts,k = 30)
res <- umap(pca$x,metric = "cosine",n_neighbors = 30,
            min_dist = 0.3,verbose = TRUE)
x <- res[,1]
y <- res[,2]
plot(x,y,pch = 20,xlab = "UMAP 1",ylab = "UMAP 2")
```

*Exercise:* Test your script by running it in a "clean" environment.

### A wrinkle in the analysis

*Data analyses are always iterative:* an analysis is gradually improved over time with careful review, feedback, literature review, etc. From discussions with your advisor in the one-on-one meeting, you learned that you made a big mistake on a step that we didn't give much thought to (it is the part where we created shifted_log_counts): what matters is not the raw gene expression levels, but instead the *relative levels.* In other words, we need to be comparing *proportions* rather than counts. This reconsideration also brings to light another hidden problem lurking in these data: some of the cells have much less expression the others suggesting some technical issues in getting the data for these cells, and therefore maybe it would be better not to include them in our analysis.

```
s <- rowSums(counts)
hist(log10(s),n = 64)
```

Maybe we will get more trustworthy results if we remove these cells with low total expression.

Fortunately, because we have carefully developed an R script, we do not need to start from scratch—we only need to make a few changes to our script and re-run the script.

```
i <- which(s > 5000)
counts <- counts[i,] # Dangerous!
barcodes <- barcodes[i,]
```

Check the total expression levels after filtering:

```
s <- rowSums(counts)
hist(log10(s),n = 64)
```

Now compute the expression proportions:

```
s <- rowSums(counts)
s <- s/mean(s)
shifted_log_counts <- log(counts/s + 1)
```

With these changes, rerun your script and see how your new plot compares to the old plot.

Your R script should now look something like this after addressing the "wrinkle":

```
library(rsvd)
library(uwot)
set.seed(1)
barcodes <- read.csv("pancreas_barcodes.csv")
genes <- read.csv("pancreas_genes.csv")
counts <- readRDS("pancreas_counts.rds")
s <- rowSums(counts)
i <- which(s > 5000)
counts <- counts[i,]
barcodes <- barcodes[i,]
s <- rowSums(counts)
s <- s/mean(s)
shifted_log_counts <- log(counts/s + 1)
pca <- rpca(shifted_log_counts,k = 30)
res <- umap(pca$x,metric = "cosine",n_neighbors = 30,
            min_dist = 0.3,verbose = TRUE)
x <- res[,1]
y <- res[,2]
plot(x,y,pch = 20,xlab = "UMAP 1",ylab = "UMAP 2")
```

*Exercise:* Test your script by running it in a "clean" environment.

### Recap of Part 3

A partial list of what I hope you will remember from Part 3:

1. Organize your project files in a folder, and develop an R script that records *what you have done.*
2. A data analysis always involves iteration: start with small, achieveable aims, and don't be afraid to make mistakes.
3. While our ultimate goal is to make new discoveries, before we get there we first need to deeply understand the data.
4. Results that are incomplete or contain errors are always more useful than no results.

### Programming challenge questions

These programming challenge questions build on the exploratory analysis of the pancreas data you have performed so far. Using UMAP, you discovered some structure underlying these data, and you improved your analysis by recognizing that the data need to be "normalized" before running the UMAP. The 2-d UMAP embedding clearly shows some clustering of the cells, with some clusters being more distinct than others. Here we will continue on the path of *data-driven discovery* and will try to understand the biological meaning of these clusters.

The first step is to *define* the clusters. Although there are ways to do this more systematically (and perhaps more rigorously), here we will take a simple approach: we will define the clusters visually based on UMAP projection of the cells onto 2-d space. While some clusters are very clearly distinct in this projection, others are not fully distinct, and so you will have to use your judgement to decide which "blob-like" structures are indeed "clusters". (There is no single "correct" answer—I look forward to seeing what decisions you make.)

Some hints for this first step: I suggest creating a new object to keep track of the cluster labels for all the cells. One simple way to visualize the cluster labeling in the UMAP plot is to label the clusters by different colors. Then use the "col" argument in `plot()` to tell R to show the points in different colors. The `axis()` function with `tck = 1` could be helpful for adding a grid to the plot in order to figure out where exactly the clusters are in the UMAP projection. A final hint for this step is that you will probably want to repurpose some of what you did in the "facts about dog breeds" programming challenge (specifically, see the "The longest-living dog breeds" section) to identify the cells belonging to the different clusters. Another function that could be useful here is the "table" function. (Remember to use the R help feature to learn how to use these functions and what the different function arguments are for.)

In the second step, we will try to make sense of these clusters by performing a *differential expression analysis* that compares the expression of each gene *within* the cluster to the expression *outside* the cluster. There are different ways to quantify expression differences, but here we will take a simple approach using the shifted log counts. (Although this approach lacks statistical rigor, the calculations you will perform are actually not far off what is done in the statistical software tools designed to make these calculations rigorous.) The calculation will look something like this:

```
x <- colMeans(shifted_log_counts[cells_inside_cluster,])
y <- colMeans(shifted_log_counts[cells_outside_cluster,])
d <- x - y
```

These three lines of code compute the mean expression level (on the shifted log scale) for cells in the cluster and not in the cluster, and performs these calculations for each column of the shifted log counts matrix, then stores the results of these calculations in two vectors, x and y. (Remember, columns = genes.) Finally, the d is a vector containing the difference in the mean expression levels for each gene. (There is *a lot* of R's "invisible magic" in these lines of code.)

Remember that you will perform a differential expression analysis once per cluster. To help organize these analyses and easily make comparisons across the different clusters, I suggest storing all the results of your differential expression analyses in a single data structure, either a matrix or a data frame, in which the columns correspond to the clusters. Also, I recommend naming both the rows (genes) and the columns (clusters). For example, here is how to create a data frame in which the columns and rows are named:

```
dat <- data.frame(A = 1:3,B = 4:6,C = 7:9)
rownames(dat) <- c("dog","cat","ferret")
```

Once you have a data frame with named rows and columns, the rows and/or columns can be indexed by name, e.g.,

```
dat["cat","A"]
dat["cat",]
dat[,c("B","C")]
```

In the third step, we will make use of the differential expression analysis to interpret the clusters. We might expect that genes that show large increases in expression will be most relevant to understanding the biological role of the cells in the cluster, however this is complicated by the fact that typically there are many genes showing increases in expression, some which are quite relevant, but others less so, or perhaps their relevance is unknown. Therefore, to simplify this task, I suggest focussing on these genes with known relevance to pancreas cell types: *Gcg* ($\alpha$ cells); *Ins1*, *Ins2* ($\beta$ cells); *Sst* ($\delta$ cells); *Ppy* ($\gamma$ cells, also known as "pancreatic polypeptide" cells); *Krt19* (duct cells); *Pecam1* (endothelial cells); *Ccr5* (macrophages); and *Col1a1* (mesenchymal cells). Mice have two insulin genes (*Ins1*, *Ins2*) unlike humans which have only one (*INS*). Note that the mapping between pancreatic cell types and the clusters is not expected to be one-to-one; that is, some clusters may include multiple pancreatic cell types (probably cell types that are more similar to each other), and some pancreatic cell types may be represented by more than one cluster (in which this case the clusters might be capturing some interesting biological variation *within* the cell type, but understanding this within-cell-type variation is not within the scope of this programming challenge).

This is inevitably a simplified exploratory analysis, nonetheless my sincere hope is that Part 3 of Basic Computing has given you some idea of what is involved in making biological discoveries from data.

# Advanced Computing — Data wrangling and plotting*

**Stefano Allesina (original author)** *University of Chicago*
**John Novembre** *University of Chicago*

## Data wrangling

As biologists living in the XXI century, we are often faced with tons of data, possibly replicated over several organisms, treatments, or locations. We would like to streamline and automate our analysis as much as possible, writing scripts that are easy to read, fast to run, and easy to debug. Base `R` can get the job done, but often the code contains complicated operations (think of the cases in which you used `lapply` only because of its speed), and a lot of `$` signs and brackets.

To start, we need to import `tidyverse`:

```
library(tidyverse)
```

`tidyverse` is a fantastic bundle of packages: a collection of `R` packages designed to manipulate large data frames in a simple and straightforward way. These tools are also much faster than the corresponding base `R` commands, and allow you to write compact code by concatenating commands to build "pipelines". Moreover, all of the packages in the bundle share the same philosophy, and are seamlessly integrated. By default, calling `library(tidyverse)` loads the packages `readr`, `tidyr` and `dplyr` (to read, organize and manipulate data), `ggplot2` (data plotting), `stringr` (string manipulation) and a few others; many others ancillary packages that are part of the `tidyverse` can be loaded if needed.

Then, we need a dataset to play with. We take a dataset containing all the papers published by UofC researchers in *Nature* or *Science* between 1999 and July 2019:

```
pubs <- read.csv("../data/UC_Nat_Sci_1999-2019.csv")
```

### A new data type, `tibble`

The data are stored in a `data.frame`:

```
is.data.frame(pubs)
```

`tidyverse` ships with a new data type, called a `tibble`. It also comes with its improved function to read data:

```
pubs <- read_csv("../data/UC_Nat_Sci_1999-2019.csv")
pubs
```

*This document is included as part of the Advanced Computing tutorial packet for the BSD qBio Bootcamp, University of Chicago, 2025. **Current version**: August 11, 2025.

which automatically reads the data as a `tibble`. The nice feature of `tibble` objects is that they will print only what fits on the screen, and also give you useful information on the size of the data, as well as the type of data in each column. Other than that, a `tibble` object behaves very much like a `data.frame`. If you want to transform the `tibble` back into a `data.frame`, use the function `as.data.frame(my_tibble)`; the function `as_tibble(my_data_frame)` transforms a `data.frame` into a tibble.

We can take a look at the data using one of several functions:

- `head(pubs)` shows the first few rows
- `tail(pubs)` shows the last few rows
- `glimpse(pubs)` a summary of the data (similar to `str` in base `R`)
- `View(pubs)` open data in spreadsheet-like window

### Selecting rows and columns

There are many ways to subset the data, either by row (subsetting the *observations*), or by column (subsetting the *variables*). For example, let's select only articles published after 2009:

```
filter(pubs, Year > 2009)
```

You can see that 515 of the 953 documents were published in the last 10 years. We have used the command `filter(tbl, conditions)` to select certain observations. We can combine several conditions, by listing them side by side, possibly using logical operators.

**Exercise:** what does this do?

```
 filter(pubs, Year == 2008, `Source title` == "Nature", `Cited by` > 100)
```

Note that the "back ticks" can be used to type column names that contain spaces and non-standard characters. This is nice, because otherwise the name of the column would need to be altered (as done automatically by `read.csv`, sometimes creating column names that are difficult to interpret or type).

We can also select particular variables using the function `select(tbl, cols to select)`. For example, select only `Authors` and `Title`:

```
select(pubs, Authors, Title)
```

How many years are represented in the data set? We can use the function `distinct(tbl)` to retain only the rows that differ from each other:

```
distinct(select(pubs, Year))
```

Where we first extracted only the column `Year`, and then retained only distinct values.

Other ways to subset observations:

- `sample_n(tbl, howmany, replace = TRUE)` sample `howmany` rows at random with replacement

- `sample_frac(tbl, proportion, replace = FALSE)` sample a certain proportion (e.g. `0.2` for 20%) of rows at random without replacement
- `slice(tbl, 50:100)` extract the rows between `50` and `100`
- `top_n(tbl, 10, Year)` extract the first `10` rows, once ordered by `Year`

More ways to select columns:

- `select(pubs, contains("Cited"))` select all columns containing the word `Cited`
- `select(pubs, -Authors, -Year)` exclude the columns `Authors` and `Year`
- `select(pubs, matches("astring|anotherstring"))` select all columns whose names match a regular expression.

### Creating pipelines using %>%

We've been calling nested functions, such as `distinct(select(pubs, ...))`. If you have to add another layer or two, the code would become unreadable. `dplyr` allows you to "un-nest" these functions and create a "pipeline", in which you concatenate commands separated by the special operator %>%. For example:

```
pubs %>% # take a data table
  select(Year) %>% # select a columns
  distinct() # remove duplicates
```

does exactly the same as the command we've run above, but is much more readable. By concatenating many commands, you can create incredibly complex pipelines while retaining readability.

### Producing summaries

Sometimes we need to calculate statistics on certain columns. For example, calculate the average number of citations. We can do this using `summarise`:

```
pubs %>% summarise(avg = mean(`Cited by`))
```

which returns a `tibble` object with just the average number of citations. You can combine multiple statistics (use `first`, `last`, `min`, `max`, `n` [count the number of rows], `n_distinct` [count the number of distinct rows], `mean`, `median`, `var`, `sd`, etc.):

```
pubs %>% summarise(avg = mean(`Cited by`),
                   sd = sd(`Cited by`),
                   median = median(`Cited by`))
```

### Summaries by group

One of the most useful features of `dplyr` is the ability to produce statistics for the data once subsetted by *groups*. For example, we would like to compute the average number of citations by journal and year:

```
pubs %>%
  group_by(`Source title`, Year) %>%
  summarise(avg = mean(`Cited by`))
```

> **Exercise:** count the number of articles by UofC researchers in *Nature* and *Science* by `Source title` and `Year`.

### Ordering the data

To order the data according to one or more variables, use `arrange()`:

```
pubs %>% select(Title, `Cited by`) %>% arrange(`Cited by`)
pubs %>% select(Title, `Cited by`) %>% arrange(desc(`Cited by`))
```

### Renaming columns

To rename one or more columns, use `rename()`:

```
pubs %>% rename(Cites = `Cited by`)
```

If you want to retain the new name(s), simply overwrite the object:

```
pubs <- pubs %>% rename(Cites = `Cited by`, Journal = `Source title`)
```

### Adding new variables using `mutate`

If you want to add one or more new columns, use the function `mutate`. For example, suppose we want to count the number of authors for each document. Authors are separated by commas (with small errors, but let's disregard that), and therefore a strategy would be to first count the number of commas, and then add 1:

```
pubs <- pubs %>% mutate(Num_authors = str_count(Authors, ",") + 1)
```

use the function `transmute()` to create a new column and drop the original columns. You can also use `mutate` and `transmute` on grouped data.

When writing code, it is good practice to separate the operations by line:

```
# A more complex example: for each paper,
# compute the percentile rank of citations
# compared to other papers of the same year
pubs %>%
  group_by(Year) %>% # group papers according to year
  mutate(pr = percent_rank(Cites)) %>% # compute % rank by Citations
  ungroup() %>% # remove group information
  arrange(Year, desc(pr), Authors) %>% # order by Year then % rank (decreasing)
  head(20) # display first 20 rows
```

in this way, you can easily comment out a part of the pipeline (or add another piece in the middle).

## Data plotting

The most salient feature of scientific graphs should be clarity. Each figure should make crystal-clear a) what is being plotted; b) what are the axes; c) what do colors, shapes, and sizes represent; d) the message the figure wants to convey. Each figure is accompanied by a (sometimes long) caption, where the details can be explained further, but the main message should be clear from glancing at the figure (often, figures are the first thing editors and referees look at).

Many scientific publications contain very poor graphics: labels are missing, scales are unintelligible, there is no explanation of some graphical elements. Moreover, some color graphs are impossible to understand if printed in black and white, or difficult to discern for color-blind people (8% of men, 0.5% of women).

Given the effort that you put in your science, you want to ensure that it is well presented and accessible. The investment to master some plotting software will be rewarded by pleasing graphics that convey a clear message.

In this section, we introduce `ggplot2`, a plotting package for `R` This package was developed by Hadley Wickham who contributed many important packages to `R` (including `dplyr`), and who is the force behing `tidyverse`. Unlike many other plotting systems, `ggplot2` is deeply rooted in a "philosophical" vision. The goal is to conceive a grammar for all graphical representation of data. Leland Wilkinson and collaborators proposed The Grammar of Graphics. It follows the idea of a well-formed sentence that is composed of a subject, a predicate, and an object. The Grammar of Graphics likewise aims at describing a well-formed graph by a grammar that captures a very wide range of statistical and scientific graphics. This might be more clear with an example – Take a simple two-dimensional scatterplot. How can we describe it? We have:

- **Data** The data we want to plot.
- **Mapping** What part of the data is associated with a particular visual feature? For example: Which column is associated with the x-axis? Which with the y-axis? Which column corresponds to the shape or the color of the points? In `ggplot2` lingo, these are called *aesthetic mappings* (`aes`).
- **Geometry** Do we want to draw points? Lines? In `ggplot2` we speak of *geometries* (`geom`).
- **Scale** Do we want the sizes and shapes of the points to scale according to some value? Linearly? Logarithmically? Which palette of colors do we want to use?
- **Coordinate** We need to choose a coordinate system (e.g., Cartesian, polar).
- **Faceting** Do we want to produce different panels, partitioning the data according to one (or more) of the variables?

This basic grammar can be extended by adding statistical transformations of the data (e.g., regression, smoothing), multiple layers, adjustment of position (e.g., stack bars instead of plotting them side-by-side), annotations, and so on.

Exactly like in the grammar of a natural language, we can easily change the meaning of a "sentence" by adding or removing parts. Also, it is very easy to completely change the type of geometry if we are moving from say a histogram to a boxplot or a violin plot, as these types of plots are meant to describe one-dimensional distributions. Similarly, we can go from points to lines, chang-

ing one "word" in our code. Finally, the look and feel of the graphs is controlled by a theming system, separating the content from the presentation.

### Basic `ggplot2`

`ggplot2` ships with a simplified graphing function, called `qplot`. In this introduction we are not going to use it, and we concentrate instead on the function `ggplot`, which gives you complete control over your plotting. First, we need to load the package (note that `ggplot2` is automatically loaded by `tidyverse`). While we are at it, let's also load a package extending its theming system:

```
library(ggplot2)
library(ggthemes)
```

A particularity of `ggplot2` is that it accepts exclusively data organized in tables (a `data.frame` or a `tibble` object). Thus, all of your data needs to be converted into a table format for plotting.

For our first plot, we're going to produce a barplot showing the number of papers in `Science` and `Nature` by UofC researcher for each `Year`. To start:

```
ggplot(data = pubs)
```

As you can see, nothing is drawn: we need to specify what we would like to associate to the *x* axis (i.e., we want to set the *aesthetic mappings*):

```
ggplot(data = pubs) + aes(x = Year)
```

Note that we concatenate pieces of our "sentence" using the + sign! We've got the axes, but still no graph... we need to specify a geometry. Let's use barplot:

```
ggplot(data = pubs) + aes(x = Year) + geom_bar()
```

As you can see, we wrote a well-formed sentence, composed of **data** + **mapping** + **geometry**, and this has produced a well-formed plot. We can add other mappings, for example, showing the journal in which the paper was published:

```
ggplot(data = pubs) + aes(x = Year, fill = Journal) + geom_bar()
```

### Scatterplots

Using `ggplot2`, one can produce very many types of graphs. The package works very well for 2D graphs (or 3D rendered in two dimensions), while it lack capabilities to draw proper 3D graphs, or networks.

The main feature of `ggplot2` is that you can tinker with your graph fairly easily, and with a common grammar. You don't have to settle on a certain presentation of the data until you're ready, and it is very easy to switch from one type of graph to another.

For example, let's plot the number of citations in the `y` axis, the year in the `x` axis. We want a scatterplot, which is produced by the geometry `geom_point`:

```
pl <- ggplot(data = pubs) + # data
  aes(x = Year, y = Cites) + # aesthetic mappings
  geom_point() # geometry

pl # or show(pl)
```

This does not look very good, because some papers have a much larger number of citations than other. We can attempt plotting the `log(Cites + 1)` instead (the `+1` is added because some papers might have 0 citations):

```
pl <- ggplot(data = pubs) + # data
  aes(x = Year, y = log(Cites + 1)) + # aesthetic mappings
  geom_point() # geometry

pl # or show(pl)
```

Much nicer! Now we can add a smoother by typing:

```
pl + geom_smooth() # spline by default
pl + geom_smooth(method = "lm", se = FALSE) # linear model, no standard errors
```

> **Exercise:** repeat the plot of the citations, but showing a different `colour` for each journal; add a smoother for each journal separately. Do papers receive more citations when they're published in *Nature* or *Science*?

### Histograms, density and boxplots

What is the distribution of citations?

```
ggplot(data = pubs) + aes(x = Cites) + geom_histogram()
```

You can see that there are some papers with many more citations than others. Try log-transforming the data:

```
ggplot(data = pubs) + aes(x = log(Cites + 1)) + geom_histogram()
```

Now we observe an histogram much closer to a Normal distribution, meaning that the number of citations is approximately log-normally distributed. You can switch to a density plot quite easily (just change the geometry!):

```
ggplot(data = pubs) + aes(x = log(Cites + 1)) + geom_density()
```

Similarly, we can produce boxplots, for example showing the number of citations for papers in *Nature* and *Science* (log transformed):

```
ggplot(data = pubs) + aes(x = Journal, y = log(Cites + 1)) + geom_boxplot()
```

It is very easy to change geometry, for example switching to a violin plot:

```
ggplot(data = pubs) + aes(x = Journal, y = log(Cites + 1)) + geom_violin()
```

**Exercise:**

- Produce a boxplot showing the number of authors (in log) per year (use `factor(Year)` for the x axis). Is science becoming more collaborative?
- Now produce a scatterplot showing the same trend, and add a smoothing function.

### Scales

We can use scales to determine how the aesthetic mappings are displayed. For example, we could set the $x$ axis to be in logarithmic scale, or we can choose how the colors, shapes and sizes are used. `ggplot2` uses two types of scales: `continuous` scales are used for continuos variables (e.g., real numbers); `discrete` scales for variables that can only take a certain number of values (e.g., treatments, labels, factors, etc.).

For example, let's plot a histogram showing the number of authors per paper:

```
ggplot(pubs, aes(x = Num_authors)) + geom_histogram() # no transformation
ggplot(pubs, aes(x = Num_authors)) + geom_histogram() +
  scale_x_continuous(trans = "log") # natural log
ggplot(pubs, aes(x = Num_authors)) + geom_histogram() +
  scale_x_continuous(trans = "log10") # base 10 log
ggplot(pubs, aes(x = Num_authors)) + geom_histogram() +
  scale_x_continuous(trans = "sqrt", name = "Number of authors")
ggplot(pubs, aes(x = Num_authors)) + geom_histogram() + scale_x_log10() # shorthand
```

We can use different color scales. For example:

```
pl <- ggplot(data = pubs %>% filter(Year %in% c(2000, 2005, 2010, 2015))) +
  aes(x = Num_authors, y = Cites, colour = factor(Year)) +
  geom_point() +
  scale_x_log10() +
  scale_y_log10()
pl + scale_colour_brewer()
pl + scale_colour_brewer(palette = "Spectral")
pl + scale_colour_brewer(palette = "Set1")
pl + scale_colour_brewer("year of publication", palette = "Paired")
```

Or use the number of authors a continuous variable:

```
pl <- ggplot(data = pubs) +
  aes(x = Year, y = log(Cites + 1), colour = log(Num_authors)) +
  geom_point()
pl + scale_colour_gradient()
pl + scale_colour_gradient(low = "red", high = "green")
pl + scale_colour_gradientn(colours = c("blue", "white", "red"))
```

Similarly, you can use scales to modify the display of the shapes of the points (`scale_shape_contintuous`, `scale_shape_discrete`), their size (`scale_size_continuous`, `scale_size_discrete`), etc. To set values manually (useful typically for discrete scales of colors or shapes), use `scale_colour_manual`, `scale_shape_manual` etc.

### Themes

Themes allow you to manipulate the look and feel of a graph with just one command. The package `ggthemes` extends the themes collection of `ggplot2` considerably. For example:

```
library(ggthemes)
pl + theme_bw() # white background
pl + theme_economist() # like in the magazine "The Economist"
pl + theme_wsj() # like "The Wall Street Journal"
```

### Faceting

In many cases, we would like to produce a multi-panel graph, in which each panel shows the data for a certain combination of parameters. In `ggplot` this is called *faceting*: the command `facet_grid` is used when you want to produce a grid of panels, in which all the panels in the same row (column) have axis-ranges in common; `facet_wrap` is used when the different panels do not have axis-ranges in common.

For example:

```
pl <- ggplot(data = pubs %>% filter(Year %in% c(2000, 2005, 2010, 2015))) +
  aes(x = log10(Cites + 1)) +
  geom_histogram()
show(pl)
pl + facet_grid(~Year) # in the same row
pl + facet_grid(Year~.) # col
pl + facet_grid(Journal ~ Year) # two facet variables
pl + facet_wrap(Journal ~ Year, scales = "free") # just wrap around
```

### Setting features

Often, you want to simply set a feature (e.g., the color of the points, or their shape), rather than using it to display information (i.e., mapping some aestethic). In such cases, simply declare the feature outside the `aes`:

```
pl <- ggplot(data = pubs %>% filter(Year %in% c(2000, 2005, 2010, 2015))) +
  aes(x = log10(Num_authors))
pl + geom_histogram()
pl + geom_histogram(colour = "red", fill = "lightblue")
```

### Saving graphs

You can either save graphs as done normally in R:

```
# save to pdf format
pdf("my_output.pdf", width = 6, height = 4)
print(my_plot)
dev.off()
# save to svg format
svg("my_output.svg", width = 6, height = 4)
print(my_plot)
dev.off()
```

or use the function ggsave

```
# save current graph
ggsave("my_output.pdf")
# save a graph stored in ggplot object
ggsave(plot = my_plot, filename = "my_output.svg")
```

### Multiple layers

Finally, you can overlay different data sets, using different geometries. For example, suppose that we have two data sets: one for papers with few authors (say <10) and one for large collaborations:

```
small_collab <- pubs %>% filter(Num_authors < 10)
large_collab <- pubs %>% filter(Num_authors >= 10)
```

We can overlay different geometries for the same data set:

```
ggplot(data = small_collab) +
  aes(x = factor(Num_authors), y = log(Cites + 1)) +
  geom_boxplot(fill = "lightblue") +
  geom_violin(fill = "NA") +
  geom_point(alpha = 0.25) # alpha stands for transparency
```

Or combine different data sets (with the same aes!):

```
ggplot(data = small_collab) +
  aes(x = Year) +
  geom_bar(fill = "red", alpha = 0.5) +
  geom_bar(data = large_collab, fill = "blue", alpha = 0.5)
```

## Tidying up data

The best way to organize data for plotting and computing is the *tidy form*, meaning that a) each variable has its own column, and b) each observation has its own row. When data are not in tidy form, you can use the package `tidyr` to reshape them.

For example, suppose we want to produce a table in which for each journal and year, we report the average number of authors. First, we need to compute the values:

```
avg_authors <- pubs %>%
  group_by(Journal, Year) %>%
  summarise(avg_au = mean(Num_authors))
```

This table is in tidy format (also called "narrow" format); we want to create columns for each journal, and report the average in the corresponding cell. To do so, we "spread" the journals into columns:

```
avg_authors <- avg_authors %>% spread(Journal, avg_au)
```

Note that this is not in tidy form, as two observations are in the same row (also called "messy" or "wide" format). While this is not ideal for computing, it is great for human consumption, as we can easily compare the two numbers in the same row.

If we want to go back to tidy form, we can "gather" the column names, and return to tidy:

```
# gather(where to store col names,
#        where to store values,
#        which columns to gather)
avg_authors %>% gather(Journal, Average_num_authors, 2:3)
# alternatively, if it's cleaner
avg_authors %>% gather(Journal, Average_num_authors, -Year)
```

## Joining tables

If you have multiple data frames or `tibble` objects with shared columns, it is easy to join them (as in a database). To showcase this, we are going to extract papers by very prolific authors. First, we want to compute how many papers are in the data for each "author" (actually, last-name initial combinations, which might represent different authors with common names...). First, we need a data set in which the authors have been separated:

```
by_author <- pubs %>%
  select(Authors, Title) %>%
  separate_rows(sep = ", ", Authors) %>%
  rename(Focal_author = Authors)
```

Now we can count the number of appearances of each name:

```
by_author <- by_author %>%
  group_by(Focal_author) %>%
  mutate(Tot = n())
```

Where we have created a new column (`Tot`) by calling `mutate` on grouped data. Who are the authors most represented in the data?

```
tot_author <- by_author %>%
  select(Focal_author, Tot) %>%
  distinct() %>%
  arrange(desc(Tot))
```

You can see that common Chinese name combinations are in the top few rows (meaning that probably we conflated several authors...). Let's plot an histogram:

```
tot_author %>% ggplot() + aes(x = Tot) + geom_histogram() + scale_y_log10()
```

As you can see, the vast majority of authors appears only once, and very few, appear 15 or more times. We want to extract the papers of the most prolific authors from the data that we have stored in `pubs`. For example, we want to consider authors that are represented 10 or more times in these papers. To do so, we first extract the prolific authors:

```
prolific <- by_author %>% filter(Tot >= 10)
```

and now we can join `pubs` and `prolific`. By calling `inner_join`, only rows that are present in both tables will be retained; because the two tables share a column (`Title`), `dplyr` can proceed automatically:

```
pubs %>% inner_join(prolific)
```

We can use this table to compute the number of citations received by each prolific author:

```
pubs %>% inner_join(prolific) %>% ggplot() +
  aes(x = Focal_author, y = Cites) +
  geom_col() + # similar to bar plot
  theme(axis.text.x = element_text(angle = 90, hjust = 1)) # rotate labels
```

Besides `inner_join(x, y)`, you can use:

- `left_join(x, y)`: return all rows from `x`, and all columns from `x` and `y` (those with no match will show `NA`);
- `right_join(x. y)`: return all rows from `y`, and all columns from `x` and `y`;
- `full_join(x, y)`: return all rows and all columns from both `x` and `y`. Where there are not matching values, returns `NA` for the one missing;
- `anti_join(x, y)`: return all rows from `x` where there are not matching values in `y`.

### Exercise in groups

Chicago's Divvy bike-share system (the light blue bikes you will see around town) collects data on all its rides and shares them publically, after anonymizing the rider info (https://www.divvybikes.com/system-data). The file `data/202207-divvy-tripdata.csv` contains a list of all the Divvy bike rides taken in Chicago in July 2022. Form small groups and work on the following exercises. Hint: use the package `lubridate` to work with days, dates, time:

- **Ride map** write a function that takes as input a calendar date (YYYY-MM-DD), and draws a map of all the starting points of rides. Mark a point for each occurrence using the starting latitude and longitude `start_lat` and `start_lng`. Set the `alpha` to something like 0.1 to show brighter colors in areas with many occurrences. Use color to indicate member type or ride type (Optional: Add a feature that draws a line from the starting location to end location)
- **Daily usage profile for the month**: Make a bar plot of the number of rides per day across the 31 days of the month. Produce a facet graph that stratifies the results by member type (the `member_casual` field). Also use the `fill` to denote the `rideable_type`.
- **Busiest hour and day of the week** on which day of the week do most rides to start? On which hour of the day do most rides start? (Extract day of the week from `started_at` using the `wday` function of `lubridate`) Make a plot of the number of rides per hour of the day, faceted on the day of the week.
- **Ride length classes** add a new column to the dataset specifying whether the ride is considered short (<5 minutes), medium (5-30 minutes), or long (>30 minutes) (Hint: again use the package `lubridate` to work with days, dates, time and extract duration from in the `started_at` and `ended_at` fields)
- **Number of rides of different lengths by day of the week** plot the number of rides against the day of the week, faceting by ride length class.
- **Rides by neighborhood** write a function that takes as input a given day, and produces a histogram of the number of rides per neighborhood on that day (i.e. x-axis is number of rides, y-axis is number of neighborhoods). A table relating `starting_station_id` to neighborhood (where neighborhood is assigned using the Google Maps API) is found in the file `data/202207-divvy-station_id_neighborhoods.csv`. You will need to `join` the tables before plotting. What is the mean number of rides per neighborhood? What is the max number? What are the top 5 "neighborhoods" for Divvy rides.
- **Interactions**: Create a table with the starting station, ending station, and the number of rides between each station. What is a feature of the most frequent trips that you observe? (Advanced: Create a graph showing edges connecting stations where the edge color represents the number of trips taken)
- **Miscellaneous**: if you'd like to play more, the `spDistsN1` function of the sp library returns distances as a function of latitude/longitude pairs.

# Data visualization tutorial: exploring data and telling stories using ggplot2*

**Peter Carbonetto** *University of Chicago*

In this lesson, we will learn how to use ggplot2 to create simple yet effective data visualizations. The ggplot2 package is an incredibly powerful plotting interface that extends the base plotting functions in R. **This tutorial is also a Google doc.**

## Some motivation

A good figure is an important part of an impactful research paper or presentation. A good figure is one that *tells an interesting story.*

Almost inevitably, creating a good figure takes iteration and refinement. You will rarely get it right on the first try.

For these reasons, taking the *programmatic approach* to creating plots is very powerful. It allows you to:

a. Create an endless variety of plots.
b. Reuse code to quickly create and revise plots.
c. Be part of the Frictionless Research Exchange.

In this tutorial we will explore the programmatic approach to plotting using **ggplot2**.

## Setup

*Do I have what I need?* Download the tutorial materials from GitHub, and make sure you know where to find them.

If you have not already done so, install these packages:

```
install.packages("ggplot2")
install.packages("cowplot")
install.packages("ggrepel")
```

If you are running your code in a Jupyter notebook or in Google Colab, I recommend running this line of code so that the outputs look the same as they do in RStudio:

*This document is included as part of the Data Visualization tutorial packet for the BSD qBio Bootcamp, University of Chicago, 2025. **Current version**: August 07, 2025; **Corresponding author**: pcarbo@uchicago.edu. Thanks to Stefano Allesina, John Novembre, Stephanie Palmer and Matthew Stephens for their support and guidance over the years.

```
options(jupyter.rich_display = FALSE)
```

## Do smaller dogs live longer?

The study of dogs is a surprisingly fruitful area of research! In this tutorial, we will make use of some data that was made available by the authors of a 2008 *Genetics* article, *Single-nucleotide-polymorphism-based association mapping of dog stereotypes.* These data are stored in a CSV file.

Our main analysis aim is to investigate the anecdotal claim that smaller breeds (such as Chihuahuas) live longer than larger breeds (such as Saint Bernards).

### Our first ggplot plot

It will take us some time to understand *how* ggplot works, but let's start by quickly creating our first ggplot.

```
library(ggplot2)
dogs <- read.csv("dogs.csv",stringsAsFactors = FALSE)
ggplot(dogs,aes(x = height,y = weight)) +
  geom_point()
```

### A first look at the data

When you load data into R for the first time, it is important to get a basic understanding of the data frame and its contents.

```
# Add your code here.
```

*What different types of data are in this table?*

### The often overlooked scatterplot

In this tutorial, we will learn about ggplot2 through one of the most basic data visualizations: *the scatterplot.*

The scatterplot is easily overlooked because it is so simple. But it can be one of the most effective ways to visualize relationships. And it has many uses.

With embellishments (adding labels, varying color, shape, size, *etc*), scatterplots can produce stunning visualizations.

### Our first ggplot2 (with "ugly" code)

Recall, our objective is to investigate the relationship between size and longevity in dogs. We'll use weight as a proxy for size.

```
# Add your code here.
```

Now for some more elegant code that can accomplish the same thing. (This more elegant code comes with a "for experts only" warning.)

```
# Add your code here.
```

For the moment I want to focus on the less elegant ("uglier") code because it highlights better the key elements of a ggplot2 plot:

1. The first input is the data (stored in a data frame).
2. The second input is an "aesthetic mapping", created using `aes` that defines how columns are mapped to features of the plot (axes, shapes, colors, *etc*).
3. A "geom", short for "geometric object", specifies the type of plot. ggplot2 has an excellent on-line reference at ggplot2.tidyverse.org explaining all the "geoms", from bar charts to contour plots, with code examples for each.
4. ggplot2 outputs a *ggplot object* `p`, which can be drawn to the screen with `print(p)` or just `p` (then hit "enter" or "return").

The distinguishing feature of ggplot2 is that plots are created by *adding layers*. This layering allows for infinite variety of plots to be created. The layering approach means that ggplot2 is easily extendible, and many R packages have been developed to enhance ggplot2. (We will use two of these packages, ggrepel and cowplot.)

### A few improvements

No plot is ever right the first time. *What are ways we can improve our plot?* Add code for your improved plot here:

```
# Add your code here.
```

In ggplot, plots can be improved either by modifying the inputs to the functions you are already using (for example, `geom_point` has many settings that can be fiddled with), or by *adding layers*. The ggplot online reference has many helpful examples that illustrate the variety of ways plots can be improved.

### Save your work

We've made some progress. This is a good point to save our work in an image file that can be shared with others. *What type of image file should we use?*

```
# Add your code here.
```

*D. melanogaster*

### Plot the best-fit line

It has been estimated that an increase of 28 lbs in a dog's body weight corresponds to about a 1-year drop in expected lifespan (with a maximum lifespan of about 13 years). How well does this estimate agree with our data? Let's investigate this question by plotting the line that *best fits* these data (specifically, this is the "least-squares" fit).

```
# Add your code here.
```

Now let's add this "best fit" line to the plot. In case we make a mistake here, let's call our new plot "p2" to avoid writing over our previous plot:

```
# Add your code here.
```

Now let's add another another "abline" layer to compare our estimate against the previous estimate (and name the new plot object "p3"):

```
# Add your code here.
```

Notice how easy it was to add layers to an existing plot!

### Which breeds fit the trend, and which don't?

It would be helpful if we could tell which breeds are being plotted. Adding text labels to a ggplot is also done by adding a layer.

There's one catch here—there simply isn't enough real estate on the plot to accommodate all the breed names. This is a great opportunity to play around with a clever package, **ggrepel**, that adds the labels in a way that makes them more readable, and only adds them to the plot when possible. Let's appreciate how simply this sophisticated plot is created.

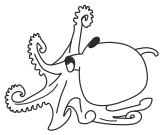
```
# Add your code here.
```

Notice that the labels are automatically redrawn as the plot is resized. Give it a try!

### A QTL for weight

In the *Genetics* paper, the strongest *quantitative trait locus* (QTL) for weight was a QTL on chromosome 7. Using your ggplot coding skills, create a scatterplot to visualize the relationship between weight and the allele frequency at this QTL.

```
# Add your code here.
```



*O. bimaculoides*

## Exploration: a surprising subtlety with color

So far, we have focussed on using the co-ordinate plane to visualize relationships. But other "aesthetics"—color, size, shape, *etc*—can also be used to tell an evocative story. In the last chapter of this tutorial, we will explore the use of color to visualize relationships. In so doing, we will discover a complication.

We learned that the "shortcoat" column had values of 0 or 1 (with a few NAs).

Let's try varying the color of the points using the shortcoat column:

```
p <- ggplot(dogs,aes(x = weight,y = aod,color = shortcoat)) +
  geom_point() +
  theme_cowplot()
print(p)
```

*Are we happy with this plot? How could it be improved?* Write your code for an improved plot here:

```
# Add your code here.
```

*Optional:* Try varying the shape of the points instead of color. If you have have written your ggplot code well, this should only involve a slight change to your code. You can use `scale_shape_manual` to select the shapes. Running `plot(0:23,pch = 0:23)` will give you the full list of shapes to choose from.

```
# Add your code here.
```

*D. rerio*

## More on color

Carefully chosen colors can often be the difference between an effective visualization and an ineffective one. One the few complaints I have with ggplot2 is that the default colors choices are quite bad. (And, perhaps unsurprisingly, these bad defaults frequently appear in published papers!)

What are the ggplot2 defaults? Let's check (here I'm reusing some example code from the ggplot2 reference website that uses the "diamonds" data set):

```
data(diamonds)
n <- nrow(diamonds)
diamonds <- diamonds[sample(n,1000),]
p <- ggplot(diamonds,aes(x = carat,y = price,color = clarity)) +
  geom_point() +
  theme_cowplot()
print(p)
```

If your labmates or co-authors are using these colors, please gently remind them to override the ggplot2 defaults!

Let's now override the ggplot2 default choices using the `scale_color_manual` layer:

```
# Add your code here.
```

The function that controls the color of a discrete variable has an odd name: `scale_color_manual`. This is because, in ggplot2, all methods that control the mapping of variables to colors, shapes, sizes, axes, *etc*, start with `scale_`.

There are several different ways to specify colors. I prefer specifying colors by name; to get the full list of color names, run `colors()`.

In short, when you use ggplot2, you will often need to make adjustments to the colors. One rule-of-thumb in color choice is that "warmer" colors such as red and orange tend to draw the reader's attention. Use this rule-of-thumb to highlight the diamonds with the best (IF) clarity:

```
# Add your code here.
```

*What are other ways you could highlight the diamonds with the best clarity?*

The "clarity" column is an example of an *ordered factor*, and it turns out that ggplot2 treats ordered and unordered factors differently. (Most factors in R are unordered.) What are the default colors for an *unordered* factor?

```
# Add your code here.
```

There are several good resources on use of color in data visualization, and I will mention a couple here: Color Brewer (https://colorbrewer2.org); and a short article, "Color blindness", written by Bang Wong (*Nature Methods*, 2011, doi:10.1038/nmeth.1618). I also recommend the Fundamentals of data visualization book by Claus Wilke.

How would you apply Color Brewer's recommendations to plot ordered or unordered factors? Write some code here to use one of Color Brewer recommended palettes:

```
# Add your code here.
```

Also explore the different options on the Color Brewer website, and see what happens when you restrict to "colorblind safe" palettes.

Obviously, there is so much more to ggplot2. But once you are comfortable with these basic elements, you will find that almost everything else in ggplot2 is a variation of what we covered in this lesson.

*T. thermophila*

## Programming challenge: Joining the Frictionless Research Exchange

I've created a new Data Visualization Programming Challenge this year: it is less structured than in past years, but I hope it will be more fun and stimulate discussion amongst your teammates.

Before the workshop, I've asked each of you to identify and download the *source data* for a figure in a research paper, ideally in an area of research you are interested in, but it doesn't have to be. (It doesn't even have to be a biology paper.) If you are having trouble finding the source data for a published figure, I recommend searching for *eLife* papers (eLife encourages sharing the source data) and Zenodo, where many authors share the data used to generate the results for a paper.

The programming challenge is a team exercise (each team will submit a single set of solutions), but at the outset I want each of you to find a data set. Then you can discuss amongst your teammates what are the most promising or most interesting data sets to focus on for the programming challenge.

There are three parts to the programming challenge.

In the first part, the goal is to import the data into R, and use ggplot2 (as well as potentially ggplot2 extensions such as ggrepel) to create a plot that reproduces as closely as possible the plot in the paper. This may involve some manipulation of the data to get it right.

In the second part, you will modify the figure to make it more accessible to people that are color blind. This could involve using "colorblind safe" colors, or by replacing the color with another visualization element. (If the original figure does not use any color, then this second part can be skipped.)

In the third part, you will visualize the data in a different way with the goal of either improving the visualization, or highlighting a different result from the same source data. The new visualization could involve: (i) using a different type of plot (e.g., bar chart, box plot, violin plot, heatmap); (ii) using different plotting elements (e.g., color, shape, size, facets); and/or (iii) using different parts of the source data from what was used in the original plot. There is a lot of freedom in this third part—the only requirement is that you make use of the source data file that was downloaded.

To submit your solutions, you will submit not only the figures you have created, *but also a script that loads the data and recreates the plots.* (The script should also describe in a comment where the data were downloaded from.) The script should save the plots as *files* (e.g., PDF or PNG files). In so doing, you have participated in the *Frictionless Research Exchange* (FRX) and have engaged in practices that keeps the FRX going. (My hope is that you will continue with these practices in your own research!) Points will be awarded for (i) creating plots that meet these goals, (ii) code that successfully reproduces these plots, and (iii) creativity.

You will need to complete these steps for two different published figures (and accompanying source data). You can decide amongst your teammates how best to proceed, but you may want to split the team into two groups, and each group can focus on one data set. One group can act as "peer reviewers" for the other group; they can comment on and suggest improvements to the plots, and verify that that scripts successfully reproduce the plots.

To illustrate what this could look like, in the GitHub repository I've provided an R script, `marino_fig4.R`, that reanalyzes data from a paper published in *eLife* that studied the relationship between effective population size and genome size. (The GitHub repository also has the source data.)

# Defensive Programming in R*

**Sarah Cobey (adapted by Kyle Hernandez and John Novembre)** *University of Chicago*

**Goal:** Convince new and existing programmers of the importance of defensive programming practices, introduce general programming principles, and provide specific tips for programming and debugging in `R`. **Audience:** Scientific researchers who use `R` and believe the accuracy of their code and/or efficiency of their programming could be improved.

## Installation

For people who have completed the other tutorials, there is nothing new to install. For others starting fresh, install `R` and `RStudio`. To install `R`, follow instructions at `cran.rstudio.com`. Then install `Rstudio` following the instructions at `https://www.rstudio.com/products/rstudio/download/`.

## Motivation

Defensive programming is the practice of anticipating errors in your code and handling them efficiently.

If you're new to programming, defensive programming might seem tedious at first. But if you've been programming long, you've probably experienced firsthand the stress from

- inexplicable, strange behavior by the code
- code that seems to work under some conditions but not others
- incorrect results or bugs that take days or weeks to fix
- a program that seems to produce the correct results but then, months or years later, gives you an answer that you know must be wrong… thereby putting all previous results in doubt
- the nagging feeling that maybe there's still a bug somewhere
- not getting others' code to run or run correctly, even though you're following their instructions

Defensive programming is thus also a set of practices for preserving sanity and conducting research efficiently. It is an art, in that the best methods vary from person to person and from project to project. As you will see, which techniques you use depend on the kind of mistakes you make, who else will use your code, and the project's requirements for accuracy and robustness. But that flexibility does not imply defensive programming is "optional": steady scientific progress depends on it. In general, we need scientific code to be perfectly accurate (or at least have well understood inaccuracies), but compared to other programmers, we are less concerned with security and ensuring that completely naive users can run our programs under diverse circumstances (although standards here are changing).

*This document is included as part of the 'Defensive Programming in R' tutorial packet for the BSD qBio Bootcamp, University of Chicago, 2025. **Current version**: August 11, 2025.

In the first part of this tutorial, we will review key principles of defensive programming for scientific researchers. These principles hold for all languages, not just R. In the second part, we will consider R-specific debugging practices in more depth.

## Part 1: Principles

Part 1 focuses on defense. You saw a few of these principles in Basic Computing, but they are important enough to be repeated here.

1. Before writing code, draft your program as a series of modular functions with high-level documentation, and develop tests for it.
2. Write clearly and cautiously.
3. Develop one function at a time, and test it before writing another.
4. Document often.
5. Refactor often.
6. When you run your code, save each "experiment" in a separate directory with a complete copy of the code repository and all parameters.
7. Don't be *too* defensive.

In part 2, we will focus on what to do when tests (from Principle 3) indicate something is wrong.

*Principle 1. Before writing code, draft your program as a series of modular functions with high-level documentation, and develop tests for it.*

Many of us have had the experience of writing a long paper only to realize, several pages in, that we in fact need to say something slightly different. Restructuring a paper at this stage can be a real chore. Outlining your code as you would outline a paper avoids this problem. In fact, outlining your code can be even more helpful because it helps you think not just generally about how your algorithm will flow and where you want to end up, but also about what building blocks (functions and containers) you'll use to get there. Thinking about the "big picture" design will prevent you from coding yourself into a tight spot—when you realize hundreds of lines in that you should've been tracking past states of a variable, for instance, but your current containers are only storing the current state. Drafting also makes the actual writing much more easy and fun.

For complex programs, your draft may start as a diagram or flowchart showing how major functions and variables relate to one another or brief notes about each of step of the algorithm. These outlines are known as pseudocode, and there are myriad customs for pseudocode and programmers loyal to particular styles. For simple scripts, it is often sufficient to write pseudocode as placeholder comments. For instance, if we are simulating a population in which individuals can be born or die (and nothing else happens), we could write:

```
# initialize population size (N), birth rate (b), death rate (d),
#            total simulation time (t_max), and start time (t=0)
# while N > 0 and t < t_max
# ... generate random numbers to determine whether next event is birth or death
#     and time to event (dt)
# ... update N (increment if birth, decrement if death)
# ... update time t to t+dt
```

Here, `b` and `d` are per capita rates. This is an example of the Gillespie algorithm. It was initially developed as an exact stochastic approach for simulating chemical reaction kinetics, and it is widely used in biology, chemistry, and physics.

Can you see some limitations of the pseudocode so far? First, it lacks obvious modularity, though this is partly due to its vagueness. The first step under the `while` loop could become its own function that is defined separately. Second, it is missing a critical feature, in that it's not obvious what is being output: do we want the population size at the end of the simulation, or the population size at each event? If the latter, we may need to initialize a container, such as a dataframe, in which we store the value of `N` and `t` at every event. After further thought, we might decide such a container would be too big—perhaps we only need to know the value of `N` at 1/1000th the typical rate of births, and so we might introduce an additional loop to store `N` only when the new time `(t+dt)` exceeds the most recent prescribed observation/sampling time. This sampling time would need to be stored in an extra variable, and we could also make the sampling procedure into its own function. And maybe we want a function to plot `N` over time at the end.

The next stage of drafting is to consider how the code might go wrong, and what it would take to convince ourselves that it is accurate. We'll spend more time on this later, but now is the time to think of every possible sanity check for the code. Sometimes this can lead to changes in code design.

> **Exercise**
>
> What sanity checks and tests would you include for the code above?

Some examples:

- Initial values of `b`, `d`, and `t_max` should be non-negative and not change
- The population size `N` should probably start as a positive whole number and never fall below zero
- If the birth and death rates are zero, `N` should not change
- If the birth rate equals the death rate, on average, `N` should not change when it is large
- The ratio of births to deaths should equal the ratio of the birth rate to the death rate, on average
- The population should, on average, increase exponentially at rate `b-d` (or decrease exponentially if `d>b`)

Some of these criteria arise from common sense or assumptions we want to build into the model (for instance, that the birth and death rates aren't negative), and others we know from the mathematics of the system. For instance, the population cannot change if there are no births and deaths, and it must increase on average if the birth rate exceeds the death rate. It helps that the Gillespie algorithm represents kinetics that can be written as simple differential equations. However, because the simulations are stochastic, we need to look at many realizations (randomizations, trajectories) to ensure there aren't consistent biases. Just as with "real" data, we must use statistics to confirm that the distribution of `N` in 10,000 simulations after 100 time units is not significantly different from what we would predict mathematically.

The bottom line is that we have identified some tests for the (1) inputs, (2) intermediate variable states (such as `N`), (3) final outputs to test that the program is running correctly. Note that one of

our tests, the fraction of birth to death events, is not something we were originally tracking, and thus we might decide now to create a separate function and variables to handle these quantities. We will talk in more detail about how to implement these tests in Principle 3. Generally, tests of specific functions are known as **unit tests**, and tests of aggregate behavior (like the trajectories of 10,000 simulations) are known as **system tests**. As a general principle, we need to have both, and we need as much as possible to compare their outputs to analytic expectations. It is also very useful to identify what you want to see right away. For instance, you may want to write a function to plot the population size over time *before* you code anything else because having immediate visual feedback can be extremely helpful (and inspiring!).

> "Let us change our traditional attitude to the construction of programs: Instead of imagining that our main task is to instruct a *computer* what to do, let us concentrate rather on explaining to *human beings* what we want a computer to do." -Donald Knuth

This is also a good time to draft very high-level documentation for your code, for instance in a `readme.MD` file. What are the inputs, what does the code do, and what does it return?

> **Exercise**
>
> Assume there are two discrete populations. Each has nonoverlapping generations and the same generation time. Their per capita birth rates are `b1` and `b2`. Some of the newborns migrate between populations.
>
> - Write pseudocode to calculate the distribution of population frequencies after 100 generations.
> - Is your code optimally modular?
> - What are the inputs and outputs of the program? Of each function?
> - How could you test the inputs, functions, and overall program?
> - Discuss your approach with your neighbor.

*Principle 2. Write clearly and cautiously.*

You're already on your way to writing clearly and cautiously if you outline your program before you start writing it. Here we'll discuss some practices to follow as you write.

A general rule is that it is more important for scientific code to be readable and correct than it is for it to be fast. Do not obsess too much about the efficiency of the code when writing.

> "Premature optimization is the root of all evil." -Donald Knuth

**Develop useful conventions for your variable and function names.** There are many conventions, some followed more religiously than others. It's most important to be consistent within your own code.

- Don't make yourself and others guess at meaning by making names too short. It's generally better to write out `maxSubstitutionRatePerSitePerYear` or `max.sub.rate.per.site.yr` than `maxSR`.
- However, very common variables should have short names.

- When helpful, incorporate identifiers like `df` (data frame), `ctr` (counter), or `idx` (index) into variable names to remind you of their purpose or structure.
- Customarily, variables that start with capital letters indicate global scope in `R`, and function names also start with capital letters. People argue about conventions, and they vary from language to language. Here's Google's style guide.

**Do not use magic numbers.** "Magic numbers" refer to numbers hard-coded in your functions and main program. Any number that could possibly vary should be in a separate section for parameters. The following code is not robust:

```
while ((age >= 5) && (inSchool == TRUE)) {
  yearsInSchool = yearsInSchool + 1
}
```

We may decide the age cutoff of 5 is inappropriate, but even if we never do, being unable to change the cutoff limits our ability to test the code. Better:

```
while ((age >= AgeStartSchool) && (inSchool == TRUE)) {
  yearsInSchool = yearsInSchool + 1
}
```

Most of the time, the only numbers that should be hard-coded are 0, 1, and pi.

**Use labels for column and row names, and load functions by argument.**

Don't force (or trust) yourself to remember that the first column of your data frame contains the time, the second column contains the counts, and so on. When reviewing code later, it's harder to interpret `cellCounts[,1]` than `cellCounts$time`.

In the same vein, if you have a function taking multiple inputs, it is safest to pass them in with named arguments. For instance, the function

```
BirthdayGiftSuggestion <- function(age, budget) {
  # ...
}
```

could be called with

```
BirthdayGiftSuggestion(age = 30, budget = 20)
# or
BirthdayGiftSuggestion(30, 20)
```

but the former is obviously safer.

**Avoid repetitive code.** If you ever find yourself copying and pasting a section of code, perhaps modifying it slightly each time, stop. It's almost certainly worth writing a function instead. The code will be easier to read, and if you find an error, it will be easier to debug.

*Principle 3. Develop one function at a time, and test it before writing another.*

The first part of this principle is easy for scientists to understand. When building code, we want to change one thing at a time. Controlled experiments are a great way to understand what's going on. Thus, we start by writing just a single function. It might not do exactly what we want it to do in the final program (e.g., it might contain mostly placeholders for functions it calls that we haven't written yet), but we want to be intimately familiar with how our code works in every stage of development.

The second part of this principle underscores one of the most important rules in defensive programming: **do not believe anything works until you have tested it thoroughly, and then keep your guard up.** *Expect* your code to contain bugs, and leave yourself time to play with the code (e.g., by trying to "break" it) until you can convince yourself they are gone. This involves an extra layer of defensive programming beyond the straightforward good practices discussed in Principle 2. Testing the code as you build it makes it much faster to find problems.

**Unit tests.** Unit tests are tests on small pieces of code, often functions. An intuitive and informal method of unit testing is to include `print()` statements in your code.

Here's a function to calculate the Simpson Index, a useful diversity index. It gives the probability that two randomly drawn individuals belong to the same species or type:

```
SimpsonIndex <- function(C) {
  print(paste(c("Passed species counts:", C), collapse=" "))
  fractions <- C / sum(C)
  print(paste(c("Fractions:", round(fractions, 3)), collapse=" "))
  S <- sum(fractions ^ 2)
  print(paste("About to return S =",S))
  return(S)
}

# Simulate some data
numSpecies <- 10
maxCount <- 10 ^ 3
fakeCounts <- floor(runif(numSpecies, min = 1, max = maxCount))

# Call function with simulated data
S <- SimpsonIndex(fakeCounts)
```

```
## [1] "Passed species counts: 569 766 849 674 297 489 749 806 433 825"
## [1] "Fractions: 0.088 0.119 0.131 0.104 0.046 0.076 0.116 0.125 0.067 0.128"
## [1] "About to return S = 0.107732503480393"
```

It's very useful to print values to screen when you are writing a function for the first time and testing that one function. When you've drafted your function, I recommend walking through the function with `print()` and comparing the computed values to calculations you perform by hand or some other way. It can also be useful to do this at a very high level (more on that later).

The problem with relying on `print()` is that it rapidly provides too much information for you to process, and hence errors can slip through.

*E. coli*

*Assertions*

A more reliable way to catch errors is to use assertions. As someone from the internet once said: if you catch yourself thinking "this should and will always be true", use assertions to check for specific conditions in that case. Assertions are automated tests embedded in the code. The built-in function for assertions in `R` is `stopifnot()`. It's very simple to use.

Let's remove the `print` statements and add a check to our input data:

```
SimpsonIndex <- function(C) {
  stopifnot(C > 0)
  fractions <- C / sum(C)
  S <- sum(fractions ^ 2)
  return(S)
}
```

If each element of our abundances vector `C` is positive, `stopifnot()` will be `TRUE`, and the program will continue. If any element does not satisfy the criterion, then `FALSE` will be returned, and execution will terminate. Explore for yourself:

```
# Simulate two sets of data
numSpecies <- 10
maxCount <- 10 ^ 3
goodCounts <- floor(runif(numSpecies, min = 1, max = maxCount))
badCounts <- floor(runif(numSpecies, min = -maxCount, max = maxCount))

# Call function with each data set
S <- SimpsonIndex(goodCounts)
S <- SimpsonIndex(badCounts)
```

This gives a very literal error message, which is often enough when we are still developing the code. But what if the error might arise in the future, e.g., with future inputs? We can use the built-in function `stop()` to include a more informative message:

```
SimpsonIndex <- function(C) {
  if(any(C < 0)) stop("Species counts should be positive.")
  fractions <- C / sum(C)
  S <- sum(fractions ^ 2)
  return(S)
}
```

Now try it with `badCounts` again.

What about warnings? For instance, our calculation of the Simpson Index is an approximation: the index formally assumes we draw without replacement, but we have been computing $S = \sum p^2$, where $p$ is the fraction of each species. It should be $S = \sum \frac{n(n-1)}{N(N-1)}$, where $n$ is the abundance of each species $n$ and $N$ the total abundance. This simplification becomes important at small sample sizes. We could add a warning to alert users to this issue:

```
SimpsonIndex <- function(C) {
  if(any(C < 0)) stop("Species counts should be positive.")
  if((mean(C) < 20) || (min(C) < 5)) {
    warning("Small sample size. Result will be biased. Consider corrected index.")
  }
  fractions <- C / sum(C)
  S <- sum(fractions ^ 2)
  return(S)
}

smallCounts <- runif(10)
S <- SimpsonIndex(smallCounts)
```

```
## Warning in SimpsonIndex(smallCounts): Small sample size. Result will be biased.
## Consider corrected index.
```

The main advantage of `warning()` over `print()` is that the message is red and will not be confused with expected results, and warnings can be controlled (see `?warning`).

You could make a case that `warning()` should be `stop()`. In general, with defensive programming, you want to halt execution quickly to identify bugs and to limit misuse of the code.

**Exercise**

- What other input checks would make sense with `SimpsonIndex()`?
- You can see how the code would be more readable and organized if most of that function were dedicated to actually calculating the Simpson Index. Draft a separate function, `CheckInputs()`, and include all tests you think are reasonable.
- When you and a neighbor are done, propose a bad or dubious input for their function and see if it's caught.

There are many packages that produce more useful assertions and error messages than what is built into R. See, e.g., `assertthat` and `testit`.

*Exception handling*

Warnings and errors are considered "exceptions." Sometimes it is useful to have an automated method to handle them. R has two main functions for this: `try()` allows you to continue executing a function after an error, and `tryCatch()` allows you to decide how to handle the exception.

Here's an example:

```
UsefulFunction <- function(x) {
  value <- exp(x)
  otherStuff <- rnorm(1)
  return(list(value, otherStuff))
}

data <- "2"
results <- UsefulFunction(data)
print(results)
```

Now `results` is quite a disappointment: it could've at least returned a random number for you, right? You could instead try

```
UsefulFunction <- function(x){
  value <- NA
  try(value <- exp(x))
  otherStuff <- rnorm(1)
  return(list(value, otherStuff))
}
results <- UsefulFunction(data)
```

```
## Error in exp(x) : non-numeric argument to mathematical function
```

```
print(results)
```

```
## [[1]]
## [1] NA
##
## [[2]]
## [1] -0.9322
```

Even though the function still can't exponentiate a string (`exp("2")` still fails), execution doesn't terminate. If we want to suppress the error message, we can use `try(..., silent=TRUE)`. This obviously carries some risk!

We could make this function even more useful by handling the error responsibly with `tryCatch()`:

```
UsefulFunction <- function(x){
  value <- NA
  tryCatch ({
      message("First attempt at exp()...")
      value <- exp(x)},
    error = function(err){
      message(paste("Darn:", err, " Will convert to numeric."))
      value <<- exp(as.numeric(x))
      }
```

```
    )
  otherStuff <- rnorm(1)
  return(list(value, otherStuff))
}
results <- UsefulFunction(data)
print(results)
```

It is also possible to assign additional blocks for warnings (not just errors). The <<- is a way to assign to the value in the environment one level up (outside the `error=` block).

> **Exercise**
>
> The package ggridges works with package `ggplot2` to show multiple distributions in a superimposed but interpretable way. Let's say we want to run the following code:

```
library(ggplot2)
library(ggridges)
ggplot(diamonds, aes(x = price, y = cut, fill = cut, height = ..density..)) +
  geom_density_ridges(scale = 4, stat = "density") +
  scale_y_discrete(expand = c(0.01, 0)) +
  scale_x_continuous(expand = c(0.01, 0)) +
  scale_fill_brewer(palette = 4) +
  theme_ridges() + theme(legend.position = "none")
```

> You probably don't have `ggridges` installed yet, so you'll get an error. Use `tryCatch()` so that the package is installed if you do not have it and then loaded.

*Test all the scales!*

It's important to consider multiple scales on which to test as you develop. We've focused on unit tests (testing small functions and steps) and testing inputs, but it is easy to have correct subroutines and incorrect results. For instance, we can be excellent at the distinct skills of toasting bread, buttering bread, and eating bread, but we will fail to enjoy buttered toast for breakfast if we don't pay attention to the order.

With scientific programming, it is critical to simplify code to the point where results can be compared to analytic expectations. You saw this in Principle 1. It is important to add functions to check not only inputs and intermediate results but also larger results. For instance, when we set the birth rate equal to the death rate, does the code reliably produce a stable population? We can write functions to test for precisely such requirements. These are **system tests**. When you change something in your code, always rerun your system tests to make sure you've not messed something up. Often it's helpful to save multiple parameter sets or data files precisely for these tests.

It's hard to overstate the importance of taking a step back from the nitty-gritty of programming and asking, Are these results reasonable? Does the output make sense with different sets of extreme values? Schedule time to do this, and update your system tests when necessary. Please don't expect your collaborators to do this work for you (unless you've arranged to trade this kind of help).

*Principle 4. Document often.*

It is helpful to keep a running list of known "issues" with your code, which would include the functions left to implement, the tests left to run, any strange bugs/behavior, and features that might be nice to add later. Sites like GitHub and Bitbucket allow you to associate issues with your repositories and are thus very helpful for collaborative projects, but use whatever works for you. Having a formal to-do list, however, is much safer than sprinkling to-do comments in your code (e.g., `# CHECK THIS!!!`). It's easy to miss comments.

Research code will always need a `readme` describing the software's purpose and implementation. It's easiest to develop it early and update as you go.

In R, the standard for documenting the code is called `roxygen2`. Upon pressing Ctrl+Shift+option+R, RStudio automatically generate a little roxygen2 skeleton for a local piece of function. We can fill in different fields with basic information about our function.

*Principle 5. Refactor often.*

To refactor code is to revise it to make it clearer, safer, or more efficient. Because it involves no changes in the scientific outputs (results) of the program, it might feel pointless, but it's usually not. Refactor when you realize that your variable and function names no longer reflect their true content or purpose (rename things quickly with `Ctrl + Alt + Shift + M`), when certain functions are obsolete or should be split into two, when another data structure would work dramatically better, etc. Any code that you'll be working with for more than a week, or that others might ever use, should probably be refactored a few times. Debugging will be easier, and the code will smell better.

**Important tip, repeated**: Run unit and system tests after refactoring to make sure you haven't messed anything up. This happens more than you might think. You can even automate/enforce tests to run in GitHub!

*Principle 6. When you run your code, save each "experiment" in a separate directory with a complete copy of the code repository and all parameters.*

When you're done developing the code and are using it for research, keep results organized by creating a separate directory for each execution of the code that includes not only the results but also the precise code used to generate the results. This way, you won't accidentally associate one set of parameters with results that were in fact generated by another set of parameters. Here's a sample workflow, assuming your repository is located remotely on GitHub, and you're in a UNIX terminal:

```
$ mkdir 2021-09-13_rho=0.5
$ cd 2021-09-13_rho=0.5
$ git clone git@github.com:MyName/my-repo
```

If we want, we can edit and execute our code from within `R` or `RStudio`, but we can also keep going with the command line. Here we are using a built-in UNIX text editor known as emacs. If you are a glutton for punishment, you could instead use vi(m). (Current Mac OS users will need to use vim or nano unless they install emacs separately.)

```
$ cd my-repo
$ emacs parameters.json // (edit parameters, with rho=0.5)
$ Rscript mycode.R
```

Keeping your experiments separate is going to save your sanity for larger projects when you repeatedly revise your analyses. It also makes isolating bugs easier.

At this point, we can attempt to argue that when different versions or experiments of your code won themselves a separate directory (this will be the case all the time as you constantly modify your analysis throughout your projects), save those different versions in a package-style. Pick the best documented R package you know and go to its source code package, and you will find that it typically contains the followings:

1. A DESCRIPTION and NAMESPACE file, both have equivalent role as the above-mentioned `readme` file.
2. Of course, the source code, including comments and documentation in `roxygen2` style.
3. Manage dependencies, preferably in a "packrat" repo. A packrat project has its own private package library. Any packages you install from inside a packrat project are only available to that project; and packages you install outside of the project are not available to the project. *Note: renv is a stable replacement for packrat.*
4. The documentation, helping users to understand the code and in particular, if the code is to be part of a pipeline, explaining how to interact with the API it exposes. Where the work product is an analysis rather than a bit of code intended to carry out a task, writing a vignettes.

For more details, see this article by Chris.

*Principle 7. Don't be* too *defensive.*

This is not necessary:

```r
myNumbers <- seq(from = 1, to = 500, length.out = 20)
stopifnot(length(myNumbers) == 20)
```

It's fine to check stuff like this when you're getting the hang of a function, but it doesn't need to be in the code. Code with too many defensive checks becomes a pain to read (and thus debug). Try to find a balance between excess caution and naive optimism. Good luck!

## Part 2: Debugging in R

Part 1 introduced principles that should minimize the need for aggressive debugging. You're in fact already debugging if you're regularly using input, unit, and system tests to make sure things are running properly. But what happens when despite your best efforts, you're not getting the right result?

We'll focus on more advanced debugging tools in this part. First, some general guidelines for fixing a bug:

- *Isolate the error and make it reproducible.* Try to strip the error down to its essential parts so you can reliably reproduce the bug. If a function doesn't work, copy the code, and keep removing pieces that are non-essential for reproducing the error. When you post for help on the website stackoverflow, for instance, people will ask for a MRE or MWE—a minimum reproducible (working) example. You need to have not only the pared code but also the inputs (parameter values and the seeds of any random number generators) that cause the problem.
- *Use assertions and debugging tools to hone in on the problem.* We've already seen how to use `stop()` and `stopifnot()` to identify logic errors in unit tests. We'll cover more advanced debugging here.
- *Change/Test one thing at a time.* This is why we develop only one function at a time.

*Note: Learning how to make MREs and how to find the relevant parts of code is especially applicable when finding bugs/issues in other people's code (even if it isn't R). The better your issue description is, the more likely the author's will be willing to help fix it! You can see an example of a bug I found in the Python package `pysam` and the issue I posted on GitHub here.*

### *Tracing calls*

If an error appears, a useful technique is to see the call stack, the sequence of functions immediately preceding the error. We can do this in R using `trackback()` or in RStudio.

The following example comes from a nice tutorial by Hadley Wickham:

```
f <- function(a) g(a)
g <- function(b) h(b)
h <- function(c) i(c)
i <- function(d) "a" + d
f(10)
```

You can see right away that running this code will create an error. Try it anyway in RStudio. Click on the "Show Traceback" to the right of the error message. What you're seeing is the stack, and it's helpfully numbered. At the bottom we have the most recent (proximate) call that produced the error, the preceding call above it, and so on. (If you're working from a separate R file that you sourced for this project, you'll also see the corresponding line numbers next to each item in the stack.) If you're in R, you can run `traceback()` immediately after the error.

Seeing the stack is useful for checking that the correct functions were indeed called, but it can suggest how to trace the error back in a logical sequence. But we can often debug faster with more information from RStudio's debugger.

### *Examining the environment*

Let's pretend we have a group of people who need to be assigned random partners. These partnerships are directed (so person A may consider his partner person B, but B's partner is person F), and we'll allow the possibility of people partnering with themselves. Some code for this is in the file `BugFun.R`. (It's not terribly efficient code, but it is useful for this exercise.)

```
source("BugFun.R")
peopleIDs <- seq(1:10)
pairings <- AssignRandomPartners(peopleIDs)
```

Try running this a few times. We have an inconsistent bug.

We can use breakpoints to quickly examine what's happening at different points in the function. With breakpoints, execution stops on that line, and environmental states can be inspected. In RStudio, you can create breakpoints by clicking to the left of the line number in the `.R` file. A red dot appears. You can then examine the contents of different variables in debug mode. To do this, you have to make sure you have the right setting defined: Debug > On Error > Break in Code.

> **Exercise**
>
> Use breakpoints to identify the error(s) in the AssignRandomPartners() function. Go to `BugFun.R` and attempt to run the last line. Decide on a place to start examining the code. If you have adjusted your settings, the debug mode should start automatically once you define a breakpoint and try to run the code again. (If a message to source the file appears, follow it.) Your console should now have `Browse[1]>` where it previously had only >.
>
> The IDE is now giving you lots of information. The green arrow shows you where you are in the code. The line that is about to be executed is in yellow. Anything you execute in the console shows states from your current environment. Test this for yourself by typing a few variables in the console. You can see these values in the Environment pane in the upper right, and you can also see the stack in the middle right.
>
> If you hit enter at the console, it will advance you to the next step of the code. But it is good to explore the Console buttons (especially 'Next') to work through the code and watch the Environment (data and values) and Traceback as they change. By calling the function several times, you should be able to convince yourself of the cause of the error.
>
> Much more detail about browsing in debugging mode is available here.
>
> When you are done, exit the debug mode by hitting the Stop button in the Console.

In R, you can insert the function `browser()` on some line of the code to enter debugging mode. This is also what you have to use if you want to debug directly in R Markdown.

*Examining the source code*

The most effective way to debug when using functions from R package is to download their source code, and check the code directly to see for yourself if there is anything that goes wrong.

My 2 cents of advice from my self-taught programming experience: don't reinvent the wheel when your primary goal is to do good research, simply borrow the wheel from experts who focus on pure software development. I learned programming in different languages by reading the source code. Reading source code allows me to assimilate others' good style of writing their code, and allows me to build a very clear connection between errors that I encounter with the parts in code responsible for those errors. Throughout the process, I also get a sense of how my system

and a specific programming language works, as much as I could. I am also able to learn the failure modes of the language when I break its certain component, for example, the association between what I have just changed recently, and what the downstream impact of my change in the cascade of my entire code network could be. Eventually I am able to narrow down the chance of producing any error in my code as I improve my programming skill, and even when an error happens, I will have a good hunch about what could go wrong immediately.

## Programming Challenge

Avian influenza cases in humans usually arise from two viral subtypes, H5N1 and H7N9. An interesting observation is that the age distributions for H5N1 and H7N9 cases differ: older people are more likely to get very sick and die from H7N9, and younger people from H5N1. There's no evidence for age-related differences in exposure. A recent paper showed that the risk of severe infection or death with avian influenza from 1997-2015 could be well explained by a simple model that correlated infection risk with the subtype of seasonal (non-avian) influenza a person was first exposed to in childhood. Different subtypes (H1N1, H2N2, and H3N2) have circulated in different years. Perhaps because H3N2 is more closely related to H7N9 than to H5N1, people with primary H3N2 infections seem protected from severe infections with H7N9. The complement is true for people first infected with H1N1 or H2N2 and later exposed to H5N1.

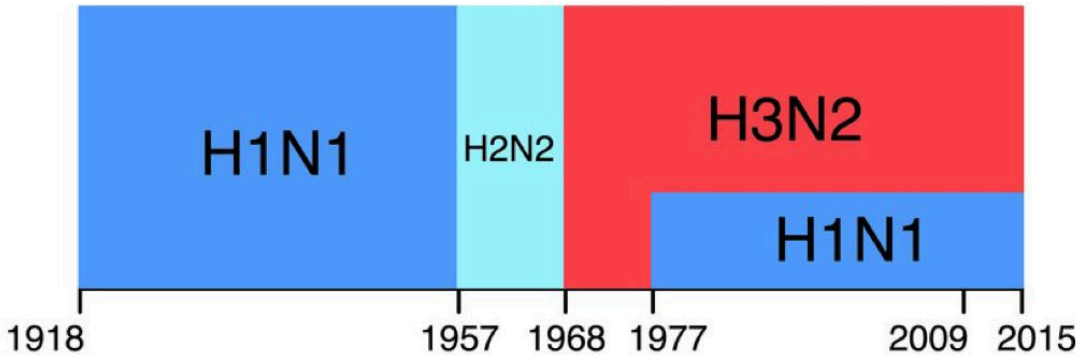



Figure 1: *Endemic influenza subtypes since 1918 (Gostic et al. 2016).*

Of course, we do not know the full infection history of any person who was hospitalized with avian influenza. We only know the person's age and the year of hospitalization or death. To perform their analysis, the authors needed to calculate the probability that each case had a primary infection with each subtype, i.e., the probability that a person born in a given year was first infected with each subtype. Your challenge is to calculate these probabilities.

The authors had to make some assumptions. First, they assumed that the risk of influenza infection is 28% in each year of life. Second, they assumed that the frequency of each circulating subtype could be inferred from the numbers of isolates sampled (primarily in hospitals) each year. These counts are given in `subtype_counts.csv`. [1]

**The challenge: For every year between 1960 and 1996, calculate the probability that a person born in that year had primary influenza infection with H1N1, H2N2, and H3N2. You must program defensively to pull this off.**

[1] The counts are actually given for each influenza season in the U.S., which is slightly different

from a calendar year, but you can ignore this. You'll notice that "1" and "0" are used where we know (or assume) that only one subtype was circulating. The authors made several other assumptions, but this is good enough for now.

# Statistics for a data rich world—some explorations*

**Stefano Allesina (adapted by Lin Chen and Xuanyao Liu)** *University of Chicago*

**Goal:** More and more often we need to analyze large and complex data sets. However, the statistical methods we've been taught in college have evolved in a data-poor world. Modern biology requires new tools, which can cope with the new questions and methods that arise in a data-rich world. Here we are going to discuss problems that often arise in the analysis of large data sets. We're going to review hypothesis testing (and what happens when we have many hypotheses) and discuss model selection. We're going to see the effects of selective reporting and p-hacking, and how they contribute to the *reproducibility crisis* in the sciences. **Audience:** Biologists with some programming background.

## I. Review of hypothesis testing

Statistics is the science of collecting and analyzing data from samples in order to estimate and make inference regarding the the population. A census data is often not feasible, and also not necessary. A sample is a smaller and random subset of the target population and the sample is collected to represent the population. Hypothesis testing is a major component of statistical inference.

The basic idea of hypothesis testing is the following: we have devised an hypothesis on our system, which we call $H_1$ (the "alternative hypothesis"). We have collected our data, and we would like to test whether the data are consistent (or inconsistent) with the so-called "null hypothesis" ($H_0$). The null hypothesis is a contradiction to the alternative hypothesis.

The simplest example is that of a bent coin: my friend Aiden likes to bet on a coin toss with people, and he often chooses head and wins. I suspect that he has a bent coin in favor of head ($H_1$: the coin is bent). We therefore toss the coin several times and check whether the number of heads we observe is consistent with the null hypothesis of a fair coin ($H_0$: the coin is a fair coin).

In `R` we can toss many coins in no time at all. Call `p` the probability of obtaining a head, and initially toss a fair coin (`p = 0.5`) a thousand times:

First, when simulating a data, we always want to set seed to make sure the results are reproducible.

```
set.seed(222)
p <- 0.5 # probability of a head (fair coin)
flips <- 1000 # number of times we flip the coin
data <- sample(c("H", "T"),
                flips, prob = c(p, 1 - p),
                replace = TRUE)
heads <- sum(data == "H")
```

*This document is included as part of the 'Statistics for a data rich world' tutorial packet for the BSD qBio Bootcamp, University of Chicago, 2025. **Current version**: August 11, 2025.

There is also a faster way to flip the coins and count the heads by assuming that the number of heads out of 1000 flips follows a binomial distribution:

```
heads <- rbinom(1, flips, p)
```

If the coin is fair, we expect approximately 500 heads, but of course we might have small variations due to the randomness of the process. We therefore need a way to distinguish between "bad luck" and an incorrect hypoythesis.

What is customarily done is to compute the probability of recovering the observed or a more extreme version of the pattern under the null hypothesis: if the probability is very small, it implies that when the null hypothesis is true, there is a very small probability (i.e., very unlikely) that you can observe things as or more extreme than what you have observed in the current data. We call this probability a *p-value*. A small *p-value* (typically less than 0.05) indicates strong evidence against the null hypothesis, so you reject the null hypothesis and conclude that the data is inconsistent with the null hypothesis. A large *p-value* (> 0.05) only means that when the null hypothesis is true, you are likely to observe what has been observed in the current data but that is not enough to prove the null hypothesis is true, so you fail to reject the null hypothesis and the conclusion is inconclusive. When the null hypothesis is indeed true or when the alternative hypothesis is true but you do not have enough power to reject the null, you may end up with a large p-value.

For example, if the coin is fair, the number of heads should follow the binomial distribution. The probability of observing a larger number of heads than what we've got is therefore

```
one.sided.pvalue <- 1 - pbinom(heads, flips, 0.5)
```

Here and in the following sections, we calculated a one-sided p-value testing $H_0 : p = 0.5$ versus $H_A : p > 0.5$. Note that in many other cases, we recommend a two-sided test, and a two-sided p-value together with a 95% confidence interval can be obtained via:

```
heads <- rbinom(1, flips, p)
pvalue <- binom.test(heads, flips, 0.5, alternative="two.sided")
```

What if we repeat the tossing many, many times?

```
# flip 1000 coins 1000 times, and count the number of heads in each of the 1000 experiments
# produce histogram of number of heads
heads_distribution <- rbinom(1000, flips, p)
hist(heads_distribution, main = "distribution number heads", xlab = "number of heads")
```

You can see that it is very unlikely to get more than 560 (or less than 440) heads when flipping a fair coin 1000 times. Therefore, if we were to observe say 400 heads (or 600), we would tend to believe that the coin is biased (though of course this could have happened by chance if we are repeating the tossing 1 trillion times!).

## Type I and type II errors

When testing an hypothesis, we can make two types of errors:

- **Type I error**: reject $H_0$ when it is in fact true. Also known as false positive.
- **Type II error**: fail to reject $H_0$ when in fact it is not true. Also known as false negative.

We call $\alpha$ the probability of making a type I error (or type I error rate), and $\beta$ as type II error rate. And power is in fact $1 - \beta$. We can calculate the p-value based on the data and compare the p-value with the significance threshold $\alpha$. The p-value quantifies how strongly the data contradicts the null hypothesis, and if $p < \alpha$, we reject the null hypothesis. Type I and Type II error rates are inversely related. In choosing a significance level for a test, you are actually deciding how much you want to risk committing a type I error — rejecting the null hypothesis when it is. The more stringent $\alpha$ is (0.01 versus 0.05) in controlling type I error rate, the less likely you would make a rejection decision and consequently the power will be reduced too.

## The distribution of p-values

Suppose that we are tossing each of several fair coins 1000 times. For each, we compute the corresponding (one-sided) p-value testing the null hypothesis `p = 0.5` against the alternative `p > 0.5`. How are the p-values distributed?

```
ncoins <- 2500
heads <- rbinom(ncoins, flips, p)
pvalues <- 1-pbinom(heads, flips, 0.5)
hist(pvalues, xlab = "p-value", freq = FALSE)
abline(h = 1, col = "red", lty = 2)
```

As you can see, if the data were generated under the null hypothesis, the distribution of the p-values would be approximately uniform between 0 and 1. This means that if we set $\alpha = 0.05$, we would reject the null hypothesis 5% of the time (even though in this case we know the hypothesis is correct!).

What is the distribution of the p-values if we are tossing biased coins? We will find an enrichment in small p-values, with stronger effects for larger biases:

```
p <- 0.52 # the coin is biased
heads <- rbinom(ncoins, flips, p)
pvalues <- 1 - pbinom(heads, flips, 0.5)
hist(pvalues, xlab = "p-value", main = paste0("p =  ", p), freq = FALSE)
abline(h = 1, col = "red", lty = 2)

p <- 0.55 # the coin is biased
heads <- rbinom(ncoins, flips, p)
pvalues <- 1 - pbinom(heads, flips, 0.5)
hist(pvalues, xlab = "p-value", main = paste0("p =  ", p), freq = FALSE)
abline(h = 1, col = "red", lty = 2)
```

*C. elegans*

## II. The challenges with p-values

### Selective reporting

Articles reporting positive results are easier to publish than those containing negative results. Authors might have little incentive to publish negative results, which could go directly into the file-drawer.

This tendency is evidenced in the distribution of p-values in the literature: in many disciplines, one finds a sharp decrease in the number of tests with p-values just below 0.05 (which is customarily–and arbitrarily–chosen as a threshold for "significant results"). For example, we find many a sharp decrease in the number of reported p-values of 0.051 compared to 0.049–while we expect the p-value distribution to decrease smoothly.

Selective reporting leads to irreproducible results: we always have a (small) probability of finding a "positive" result by chance alone. For example, suppose we toss a fair coin many times, until we find a "signficant" result.

On the other hand, more and more journals would require us to report effect size estimates and confidence intervals – "Were this procedure to be repeated on numerous samples, the fraction of calculated confidence intervals (which would differ for each sample) that encompass the true population parameter would tend toward 90%."(source: Cox D.R., Hinkley D.V. (1974) Theoretical Statistics, Chapman & Hall, p49, p209.)

### Problem: p-hacking

The problem is well-described by Simonsohn et al. (J. Experimental Psychology, 2014): "While collecting and analyzing data, researchers have many decisions to make, including whether to collect more data, which method to use, which measure(s) to analyze, which covariates to use, what to do with outliers and missing data, and so on. If these decisions are not made in advance but rather are made as the data are being analyzed, then researchers may make them in ways that self-servingly increase their odds of publishing. Thus, rather than placing entire studies in the file-drawer, researchers may file merely the subsets of analyses that produce nonsignificant results. We refer to such behavior as *p-hacking*." The term p-hacking describes the conscious or subconscious manipulation of data in a way that produces a desired p-value.

The same authors showed that with careful p-hacking, almost anything can become significant (read their hylarious article in Psychological Science, where they show that listening to a song can change the listeners' age!).

### Discussion on p-values

Selective reporting and p-hacking are only two of the problems associated with the widespread use and misuse of p-values. The discussion in the scientific community on this issue is extremely topical. I have collected some of the articles on this problem in the `readings` folder. Importantly, in 2016 the American Statistical Association released a statement on p-values every scientist should read.

### Reproducibility crisis

P-values and hypothesis testing contribute considerably to the so-called *reproducibility crisis* in the sciences. A survey promoted by *Nature* magazine found that "More than 70% of researchers have tried and failed to reproduce another scientist's experiments, and more than half have failed to reproduce their own experiments."

This problem is due to a number of factors, and addressing it will likely be one of the main goals of science in the next decade.

### Exercise: p-hacking

Go to goo.gl/a3UOEF and try your hand at p-hacking, showing that your favorite party is good (bad) for the economy.

 *D. rerio*

## III. Multiple comparisons (also known as multiple testing)

The problem of multiple comparisons arises when we perform multiple statistical tests. Since each test is subject to some small chance of producing a false positive result, when jointly considering many many tests, the chances of producing some false positive findings are much higher.

Suppose we perform our coin tossing exercise, flipping 1000 coins 1000 times each. For each coin, we determine whether our data differs significantly from what expected by contrasting our p-value with a significance level $\alpha = 0.05$.

Even if the coins are all perfectly fair, we would expect to find approximately $0.05 \cdot 1000 = 50$ coins that lead to the rejection of the null hypothesis.

In fact, we can calculate the probability of making at least one type I error (reject the null when in fact it is true). This probability is called the Family-Wise Error Rate (FWER). It can be computed as 1 minus the probability of making no type I error at all. If we set $\alpha = 0.05$, and assume the tests to be independent, the probability of making no errors in $m$ tests is $1 - (1 - 0.05)^m$. Therefore, if we perform 10 tests, we have about 40% probability of making at least a mistake; if we perform 100 tests, the probability grows to more than 99%. If the tests are not independent, we can still say that in general $FWER \leq m\alpha$.

**This means that setting an $\alpha$ per test does not control for FWER**.

Moving from tossing coins to biology, consider the following examples:

- **Gene expression** In a typical RNAseq experiment, we compare the differential expression levels of tens of thousands of genes in the treatment and control tissues.
- **GWAS** In Genome-Wide Association Studies we want to find single-nucleotide polymorphisms (SNPs) associated with a given phenotype. It is common to test millions of SNPs for signficant associations.
- **Identifying binding sites** Identifying candidate binding sites for a transcriptional regulator requires scanning the whole genome, yielding tens of millions of tests.

### Organizing the tests in a table

Suppose that we're testing $m$ hypotheses. Of these, an unknown subset $m_0$ is true, while the remaining $m_1 = m - m_0$ are false. We would like to correctly call the true/false hypotheses (as much as possible). We can summarize the results of our tests in a table, of which the elements are unobservable:

What we would like to know is $m_1 = T + S$ and $m_0 = U + V$. Then $V$ is the number of type I errors (rejected $H$ when in fact it is true), and $T$ is the number of type II errors (failed to reject a false $H$). However, we can only observe $V + S$ (the number of "discoveries"), and $U + T$ (number of "failures").

The type I error rate is $E[V]/m$ (where $E[X]$ stands for expectation). When there are many tests $(m)$ being considered, controlling for type I error rates at 0.05 means that by random chance, one could make $0.05 \times m$ false positive findings even if all $m$ tests are under the null. For example, when testing genetic associations between 10 million genetic variants to the risk of a disease, if still using 0.05 as the significance threshold, there could be 500k significant findings by random chance even if the disease is not heritable and has no genetic association. Apparently, we need to choose a much more strigent significance threshold to account for the number of tests, and there are some other error measures. The Family-wise error rate is defined as $P(V > 0)$. Another quantity of interest is the False Discovery Rate (FDR), measured as the proportion of true discoveries $FDR = E[V/(V + S)]$ when $V + S > 0$. FDR measures the proportion of falsely rejected hypotheses.

Importantly, FWER guards against any single false positive finding among all $m$ tests, and is a more stringent significance criteria than FDR in multiple comparison problems. It also means that when we control for FWER, we're automatically controlling for the FDR, but not vice versa. It should be noted that when a large number of tests is performed, controlling FWER could be quite conservative and may lose power.

### Methods for multiple testing correction: Bonferroni correction

One of the simplest and most widely-used procedures to control for FWER is Bonferroni's correction. This procedure controls for FWER in the case of independent or dependent tests. It is typically quite conservative, especially when the tests are not independent (in practice, it becomes "too conservative" when the number of tests is moderate to high). Fundamentally, for a desired FWER $\alpha$ we choose as a significance treshold of $\alpha/m$ for each single test, where $m$ is the number of tests we're performing. Equivalently, we can "adjust" the p-values as $q_i = \min(m \cdot p_i, 1)$, and call significant the values $q_i < \alpha$. In R it is easy to perform this correction:

```
original_pvals <- c(0.012, 0.06,  0.77, 0.001, 0.32)
adjusted_pvals <- p.adjust(original_pvals, method = "bonferroni")
print(adjusted_pvals)
```

With these adjusted p-values, and an $\alpha = 0.05$, we would still single out as significant the fourth test, but not the first. The strength of Bonferroni is its simplicity, and the fact that we can perform the operation in a single step. Moreover, the order of the tests does not matter.

### Other procedures: the Holm–Bonferroni method (or the Holm method)

There are several refinements of Bonferroni's correction, some of which use the sequence of ordered p-values. For example, the Holm method starts by sorting the p-values in increasing order $p_{(1)} \leq p_{(2)} \leq p_{(3)} \leq \dots p_{(m)}$. The hypothesis $H_{(i)}$ is rejected if $p_{(j)} \leq \alpha/(m-j+1)$ for all $j = 1, \dots, i$. Equivalently, we can adjust the p-values as $q_{(i)} = \min(1, \max((m-i+1)p_{(i)}, q_{(i-1)}))$. In this way, we use the most stringent threshold to determine whether the smallest p-value is significant, the next smallest p-value uses a slightly higher threshold and so on. The Holm method is uniformly more powerful than the Bonferroni correction.

For example, using the same p-values above:

```
original_pvals <- c(0.012, 0.06,  0.77, 0.001, 0.32)
adjusted_pvals <- p.adjust(original_pvals, method = "holm")
print(adjusted_pvals)
```

We see that we would be calling the first test significant, contrary to what obtained with Bonferroni.

The function `p.adjust` offers several choices for p-value correction. Also, the package `multcomp` provides a quite comprehensive set of functions for multiple hypothesis testing.

### An example: testing mixed coins

We're going to test these concepts by tossing repeatedly many coins. In particular, we're going to toss 1000 times 50 biased coins ($p = 0.55$) and 950 fair coins ($p = 0.5$). For each coin, we're going to compute a p-value, and count the number of type I, type II, etc. errors when using unadjusted p-values as well as when correcting using the Bonferroni or Holm procedure.

```
toss_coins <- function(p, flips){
  # toss a coin with probability p of landing on head several times
  # return a data frame with p, number of heads, pval and
  # H0 = TRUE if p = 0.5 and FALSE otherwise
  heads <- rbinom(1, flips, p)
  pvalue <- 1 - pbinom(heads, flips, 0.5)
  if (p == 0.5){
    return(data.frame(p = p, heads = heads, pval = pvalue, H0 = TRUE))
  } else {
    return(data.frame(p = p, heads = heads, pval = pvalue, H0 = FALSE))
  }
}

# To ensure everybody gets the same results, we're setting the seed
set.seed(8)
data <- data.frame()
# the biased coins
for (i in 1:50) data <- rbind(data, toss_coins(0.55, 1000))
# the fair coins
```

```
for (i in 1:950) data <- rbind(data, toss_coins(0.5, 1000))
# here's the data structure
head(data)
```

Now we write a function that adjusts the p-values and builds the table above

```
get_table <- function(data, adjust, alpha = 0.05){
  # produce a table counting U, V, T and S
  # after adjusting p-values for multiple comparisons
  data$pval.adj <- p.adjust(data$pval, method = adjust)
  data$reject <- FALSE
  data$reject[data$pval.adj < alpha] <- TRUE
  return(table(data[,c("reject","H0")]))
}
```

First, let's see what happens if we don't adjust the p-values:

```
no_adjustment <- get_table(data, adjust = "none", 0.05)
print(no_adjustment)
```

We correctly declared 48 of the biased coins "significant", but we also incorrectly called 2 biased coins "not significant" (Type II error). More worringly, we called 45 fair coins biased when they were not (Type I error). To control for the family-wise error rate, we can correct using Bonferroni:

```
bonferroni <- get_table(data, adjust = "bonferroni", 0.05)
print(bonferroni)
```

With this correction, we dramatically reduced the number of type I errors (from 45 to 0), but at the cost of increasing type II errors (from 2 to 40) and losing power. In this way, we would make only 10 discoveries instead of 50.

In this case, Holm's procedure does not help:

```
holm <- get_table(data, adjust = "holm", 0.05)
print(holm)
```

More sophisticated methods, for example the Benjamini-Hochberg (BH) procedure based on controlling false discovery rate ( FDR=E(false discoveries/signifiant tests), where E stands for expectation), can reduce the type II errors and improve power, at the cost of a few and estimable type I errors:

```
BH <- get_table(data, adjust = "BH", 0.05)
print(BH)
```

*C. jacchus*

## FDR and q-values

Inspired by the need for controlling for FDR in genomics, Storey and Tibshirani (PNAS 2003) have proposed the idea of a q-value, measuring the probability that a feature that we deemed significant turns out to be not significant after all.

One uses p-values to control for the false positive rate (# false positive / total test): when determining significant p-values we control for the rate at which null features in the data are called significant. The False Discovery Rate (# false positive / # signficant test), on the other hand, measures the rate at which results that are deemed significant are truly null. While setting $PCER = 0.05$ we are stating that about 5% of the truly null features will be called significant, an $FDR = 0.05$ means that among the features that are called significant, about 5% will turn out to be null.

They proposed a method that uses the ensamble of p-values to determine the approximate (local) FDR. The idea is simple. If you plot your histogram of p-values when you have few true effect, and many nulls, you will see something like:

```
hist(data$pval, breaks = 25)
```

where the right side of the histogram is close to a uniform distribution. We could use the high p-values to find how tall the histogram would be if all effects were null, thereby estimating the proportion of truly null features $\pi_0 = m_0/m$.

Storey has built an `R`-package for this type of analysis:

```
# To install:
#install.packages("devtools")
#library("devtools")
#install_github("jdstorey/qvalue")
library("qvalue")
qobj <- qvalue(p = data$pval)
```

Here's the estimation of the $\pi_0$

```
hist(qobj)
```

which is quite good (in this case we know that $\pi_0 = 0.95$). The small p-values under the dashed line represent our false discoveries. Even better, through randomizations one can associate a q-value to each test, representing the probability of making a mistake when calling a result significant (formally, the q-value is the minimum FDR that can be attained when calling that test significant).

For example:

```
table((qobj$pvalues < 0.05) & (qobj$qvalues < 0.05), data$H0)
```

Note that the estimation of FDR is unstable if the demoniator (# significant test) is expected to be small. Therefore, you may notice that FDR was widely used in detecting differentially expressed genes in diseased versus normal samples where the expected number of non-null tests is large. In contast, in GWAS, researchers use the Bonferroni-adjusted p-value threshold of $5 \times 10^{-8}$ to declare significance.

## IV. Linear regression, logistic regression, and model selection

### Linear regression

In statistics, linear regression is an approach to model the linear relationship between one response variable (or dependent variable) and one or more explanatory variables (or independent variables, or predictor). If there is only one exlanatory variable, it is called simple linear regression. If the linear regression involves more than one explanatory variable, it is a multiple linear regression.

```
# create fake data for a simple linear regression
set.seed(5)
x <- 1:20
y <- 3 + 0.5 * x + rnorm(20)
plot(y ~ x)
```

We can fit a simple linear regression to the data

```
model1 <- lm(y ~ x)
summary(model1)
plot(y~x)
points(model1$fitted.values~x, type = "l", col = "blue")
```

In a data rich world, often, we need to select a model out of a set of reasonable alternatives with different combinations and/or (even nonlinear) patterns of explanatory variables. However, we run the risk of overfitting the data (i.e., fitting the noise as well as the pattern). The best fitted model for the current data from the current sample may not be the best model representing the pattern in the population of interest. Here is a simple example of a overfitted regression:

For the above data, we can also fit a more complex polynomial function of $x$.

```
model2 <- lm(y ~ poly(x, 7))
```

Let's see the residuals etc.

```
summary(model1)
summary(model2)
plot(y~x)
points(model1$fitted.values~x, type = "l", col = "blue")
points(model2$fitted.values~x, type = "l", col = "red")
```

Our second model has a much greater $R^2$, i.e., more variation in the response variable can be explained by the model, but the second model also has many more parameters. The first model is more parsimonious. Which is a model we should choose?

Model selection tries to address this and similar problems. Most model fitting and model selection procedures are based on likelihoods (e.g., Bayesian models, maximum likelihood, minimum description length). The likelihood $L(\theta|D, M)$ is (proportional to) the probability of observing the data $D$ under the model $M$ and parameters $\theta$. Because likelihood can be very small when you have much data, typically one works with log-likelihoods. For example:

```
logLik(model1)
logLik(model2)
```

Typically, more complex models will yield better (less negative) log-likelihoods and a better fit for the current data. However, more parameters would also increase the variation of the model. A complex model may not best represent the population (not to say computational burden). We will need to find a balance between bias and variance. We therefore want to penalize more complex models in some way.

### AIC

One of the simplest methods to select among competing models is the Akaike Information Criterion (AIC). It penalizes models accoding to the **number of parameters**: $AIC = 2p - 2\log L(\theta|D, M)$, where $p$ is the number of parameters. Note that **smaller** values of AIC stand for "better" models. In `R` you can compute AIC using:

```
AIC(model1)
AIC(model2)
```

As you can see, AIC would favor the first (and simpler) model, which is also the model we used to simulate the data.

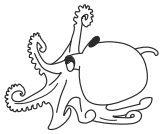
AIC is rooted in information theory and measures (asymptotically) the loss of information when using the model instead of the data. There are several limitations of AIC: a) it only holds asymptotically (i.e., for very large data sets; for smaller data you need to correct it); it penalizes each parameter equally (i.e., parameters that have a large influence on the likelihood have the same weight as parameters that do no influence the likelihood much); it can lead to overfitting, favoring more complex models in simulated data generated by simpler models.

### BIC

In a similar vein, BIC (Bayesian Information Criterion) uses a slightly different penalization: $BIC = \log(n)p - 2\log L(\theta|D, M)$, where $n$ is the number of data points. You may see that for large data, BIC penalizes a complex model more than AIC. Again, smaller values stand for "better" models:

```
BIC(model1)
BIC(model2)
```

Here in this simple example, AIC and BIC agree with each other, and the simpler model is favored.



*O. bimaculoides*

## Logistic regression

Logistic regression is often used to model the nonlinear relationship (modeled by a logistic function) between a binary response variable and a linear combination of explantory variables. When the outcome is binary, for example pass/fail, win/lose, alive/dead, yes/no, one would consider a logistic regression. It can be extended to model several classes of a categorical variable as response.

We will start with an example. Fox et al. (Research Integrity and Peer Review, 2017) analyzed the invitations to review for several scientific journals, and found that "The proportion of invitations that lead to a submitted review has been decreasing steadily over 13 years (2003–2015) for four of the six journals examined, with a cumulative effect that has been quite substantial". Their data is stored in `../data/FoxEtAl.csv`. We're going to build models trying to predict whether a reviewer will agree (or not) to review a manuscript.

```
# read the data
reviews <- read.csv("../data/FoxEtAl.csv", sep = "\t")
# take a peek
head(reviews)
# set NAs to 0
reviews[is.na(reviews)] <- 0
# how big is the data?
dim(reviews)
# that's a lot! Let's take 5000 review invitations for our explorations;
# we will fit the whole data set later
set.seed(101)
small <- reviews[order(runif(nrow(reviews))),][1:5000,]
```

The response variable of interest is a reviewer $i$ agreeing to review a manuscript or not, and is binary; and so we decided to use a logistic regression. Call $\pi_i$ the probability that a reviewer $i$ agree to review a manuscript. We model $logit(\pi_i) = \log(\pi_i/(1-\pi_i))$ as a linear function.

*Constant rate*

As a null model we build a model in which the probability of agreeomg to review does not change in time/for journals:

```
# suppose the rate at which reviewers agree is a constant
mean(small$ReviewerAgreed)
# fit a logistic regression
model_null <- glm(ReviewerAgreed~1, data = small, family = "binomial")
summary(model_null)
# interpretation:
exp(model_null$coefficients[1]) / (1 + exp(model_null$coefficients[1]))
```

*Declining trend*

We now build a model in which the probability to review declines steadily from year to year:

```
# Take 2003 as baseline
model_year <- glm(ReviewerAgreed~I(Year - 2003), data = small, family = "binomial")
#The I() function acts to convert the argument to "as.is"
summary(model_year)
```

*Journal dependence*

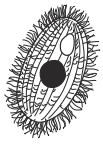
Reviewers might be more likely to agree for more prestigious journals:

```
# Take the first journal as baseline
model_journal <- glm(ReviewerAgreed~Journal, data = small, family = "binomial")
summary(model_journal)
```

*Model journal and year*

Finally, we can build a model combining both features: we fit a parameter for each journal/year combination

```
# Take the first journal as baseline, the colon mark stands for interaction term only.
model_journal_yr <- glm(ReviewerAgreed~Journal:I(Year-2003),
                        data = small, family = "binomial")
#summary(model_journal_yr)
```



*T. thermophila*

### Likelihoods

In R, you can extract the log-likelihood from a model object calling the function `logLik`

```
logLik(model_null)
logLik(model_year)
logLik(model_journal)
logLik(model_journal_yr)
```

Interpretation: because we're dealing with binary data, the likelihood is the probability of correctly predicting the agree/not agree for all the 5000 invitations considered. Therefore, the probability of guessing a (random) invitation correctly under the first model is:

```
exp(as.numeric(logLik(model_null)) / 5000)
```

while the most complex model yields

```
exp(as.numeric(logLik(model_journal_yr))/ 5000)
```

We didn't improve our guessing much by considering many parameters! This could be due to specific data points that are hard to predict, or mean that our explanatory variables are not sufficient to model our response variable.

### AIC

We can also calculate AIC for logistic models. In R you can compute AIC using:

```
AIC(model_null)
AIC(model_year)
AIC(model_journal)
AIC(model_journal_yr)
```

As you can see, the model `model_journal_yr` has the smallest AIC among all and is preferred here.

### BIC

In a similar vein, BIC (Bayesian Information Criterion) uses a slightly different penalization: $BIC = \log(n)p - 2\log L(\theta|D, M)$, where $n$ is the number of data points. Again, smaller values stand for "better" models:

```
BIC(model_null)
BIC(model_year)
BIC(model_journal)
BIC(model_journal_yr)
```

Note that according to BIC, `model_year` is favored. As mentioned before, BIC would penalize a complex model more.

### Cross validation

One very robust method to perform model selection, often used in machine learning, is cross-validation. The idea is simple: split the data in three parts: a small data set for exploring; a large set for fitting; a small set for testing (for example, 5%, 75%, 20%). You can use the first data set to explore freely and get inspired for a good model. The data will be then discarded. You use the largest data set for accurately fitting your model(s). Finally, you validate your model or select over competing models using the last data set.

Because you haven't used the test data for fitting, this should dramatically reduce the risk of overfitting. The downside of this is that we're wasting precious data. There are less expensive methods for cross validation, but if you have much data, or data are cheap, then cross-validation has the virtue of being fairly robust.

Let's try our hand at cross-validation. First, we split the data into three parts:

```
reviews$cv <- sample(1:3, nrow(reviews), prob = c(0.05, 0.75, 0.2), replace = TRUE)
dataexplore <- reviews[reviews$cv == 1,]
datafit <- reviews[reviews$cv == 2,]
datatest <- reviews[reviews$cv == 3,]
# We've already done our exploration.
# Let's fit the data using model_journal
# and model_journal_yr, which seem to be the most promising
cv_model1 <- glm(ReviewerAgreed~I(Year-2003), data = datafit, family = "binomial")
cv_model2 <- glm(ReviewerAgreed~Journal:I(Year-2003), data = datafit,
      family = "binomial")
```

Now that we've fitted the models, we can use the function `predict` to find the fitted values for the `testdata`:

```
mymodel <- cv_model1
# compute probabilities
pi <- predict(mymodel, newdata = datatest, type = "resp")
# compute log likelihood
mylogLik <- sum(datatest$ReviewerAgreed * log(pi) +
                  (1 - datatest$ReviewerAgreed) * log(1 - pi))
print(mylogLik)
```

repeat for the other model

```
mymodel <- cv_model2
# compute probabilities
pi <- predict(mymodel, newdata = datatest, type = "resp")
# compute log likelihood
mylogLik <- sum(datatest$ReviewerAgreed * log(pi) +
                  (1 - datatest$ReviewerAgreed) * log(1 - pi))
print(mylogLik)
```

Cross validation supports the choice of the more complex model here.

*S. aegyptiacus*

### Other approaches

Bayesian models are gaining much traction in biology. The advantage of these models is that you can get a posterior distribution for the parameter values, reducing the need for p-values and AIC. The downside is that fitting these models is computationally much more expensive (you have to find a distribution of values instead of a single value).

There are three main ways to perform model selection in Bayesian models:

- **Reversible-jump MCMC** You build a Monte Carlo Markov Chain that is allowed to "jump" between models. You can choose a prior for the probability of being in each of the models; the posterior distribution gives you an estimate of how much the data supports each model. Upside: direct measure. Downside: difficult to implement in practice – you need to avoid being "trapped" in a model.
- **Bayes Factors** Ratio between the probability of two competing models. Can be computed analytically for simple models. Can also be interpreted as the average likelihood when parameters are chosen according to their prior (or posterior) distribution. Upside: straightforward interpretation — it follows from Bayes theorem; Downside: in most cases, one needs to approximate it; can be tricky to compute for complex models.
- **DIC** Similar to AIC and BIC, but using distributions instead of point estimates.

Another alternative paradigm for model selection is Minimum-Description Length. The spirit is that a model is a way to "compress" the data. Then you want to choose the model whose total description length (compressed data + description of the model) is minimized.

### A word of caution

The "best" model you've selected could still be a terrible model (best among bad ones). Out-of-fit prediction (such as in the cross-validation above) can give you a sense of how well you're modeling the data.

When in doubt, remember the (in)famous paper in Nature by Tatem et al. 2004, which used some flavor of model selection to claim that, according to their linear regression, in the 2156 Olympics the fastest woman would run faster than the fastest man. One of the many memorable letters that ensued reads:

> Sir — A. J. Tatem and colleagues calculate that women may out-sprint men by the middle of the twenty-second century (Nature 431,525; 2004). They omit to mention, however, that (according to their analysis) a far more interesting race should occur in about 2636, when times of less than zero seconds will be recorded.
>
> In the intervening 600 years, the authors may wish to address the obvious challenges raised for both time-keeping and the teaching of basic statistics.

Kenneth Rice

Prediction for population outside the current data is prophecy.

## V. Programming Challenge

### P-hacking COVID-19

To show firsthand how p-hacking and overfitting are possible, we want you to show how these practices can lead to completely nonsensical results.

You can download a complete list of data on COVID-19 (coronavirus) by Our World in Data (https://ourworldindata.org/coronavirus). The data is updated daily and contains the lastest publicly available data on COVID-19 by country and by date. The data report the total cases, new cases, total deaths, new deaths, and hospitalization data of 233 countries and regions. Note that you are not expected to analyze the entire data. You may choose one or a few countries, or select one or some dates for analysis or for comparison.

The challenge is to build an analysis pipeline that produces a "significant" p-value for a relationship between COVID-19 cases and another variable, where the relationship is non-sensical, cannot possibly be causal, or could be argued either way. You may even simulate a fake variable as your key variable of interest. Prepare an Rmarkdown document with the results. At the end of the document write a paragraph to explain your "findings". As if you were in a debate team, pick on a subjective conclusion, and "cherry-pick" partial data to support your claim. Provide a non-statistical explanation for your group's fake result, and/or critique your statistcal approach and why your group got an apparently significant p-value.

As an example, below on a particular date (02/26/2020), I found a positive relationship between handwashing facilities and new cases in Asia countries.

Some sample code:

```
library(dplyr)
#read the Dataset sheet into "R". The dataset will be called "data".
data <- read.csv("https://covid.ourworldindata.org/data/owid-covid-data.csv",
     na.strings = "",header=T)


res <- NULL
for (i in 1:length(unique(data$date))){
  data1 <- data[which((data$date==unique(data$date)[i])&(data$continent=="Asia")),]
  data1 <- data1 %>% select("iso_code","date","new_cases", "handwashing_facilities")
  if (sum(rowSums(!is.na(data1[,3:4]))==2)>=10){
  res <- rbind(res, c(unique(data$date)[i],
      cor.test(data1[,3],data1[,4])$estimate,
      cor.test(data1[,3],data1[,4])$p.value))
}}

res[which((as.numeric(res[,2])>0)& (as.numeric(res[,3])<=0.05)),]
```

# Workshops

Developmental biology workshop → Noah Mitchell
Population genetics workshop → Jeremy Berg
Movement analysis workshop → Jasmine Nirody

# Mesh triangulations for biological shape analysis*

**Noah Mitchell** *University of Chicago*

This tutorial introduces some core concepts in scientific programming and shape analysis using Python. We'll explore how to analyze and compare embryonic organ shapes using triangulated meshes. You will learn how to build meshes from scratch, analyze them for correctness, and register two shapes using Iterative Closest Point (ICP) alignment. The notebook is intended for early-stage graduate students with no serious coding experience.

## Part 1: Intro to python and mesh triangulations

### 0. Getting Started

*0a. Pycharm*

To begin working with Python code, we recommend using PyCharm, a free, user-friendly code editor. First, download PyCharm Community Edition and install it on your computer following the instructions here: `https://www.jetbrains.com/help/pycharm/installation-guide.html`. Also download the data folders from Box (`wildtype` and `bynGAL4_UASMyo1C`) and place them in the same place where you have the tutorial code cloned (see Figure 1A). When you open PyCharm for the first time, create a new project: choose the folder where this tutorial's code resides – same place as where the `wildtype` and `bynGAL4_UASMyo1C` data live (see Figure 1B).

Then configure a Python interpreter. Select or create an interpreter that uses Python 3.10 — this version is required for this tutorial. If no interpreter is available, click "Add Interpreter" and choose a Virtualenv, then select Python 3.10 as the base interpreter during setup (you may need to install it, see Figure 1C).

*Default method:* Once your project is open, open `organ_geometry.py`. At the top, some of the modules will be underlined in red since they are not installed. Hover over one and click `Install all missing packages` (see Figure 1D). Then close `organ_geometry.py` and open the Python Console using the top icon in the bottom left (see arrowhead in Figure 2). Now you can copy-paste snippets of code into the Python Console to run them (note the copy icon circled in Figure 2). If you see an error like `ModuleNotFoundError` for any reason while playing with the code, click the red light bulb or go to Settings > Python Interpreter to install the missing package. That's it — you're ready to start coding in Python!

*Conda method for experienced students only:* If you are experienced with python, feel free to use a Conda Environment and create a base interpreter from python 3.10. For Conda users, there is a .yml file included that will load up your virtual environment with what you need with:

```
$ conda env create -f environment.yml
```

*This document is included as part of the Developmental Biology / Shape Analysis workshop packet for the BSD qBio Bootcamp, University of Chicago, 2025. **Current version**: August 11, 2025.

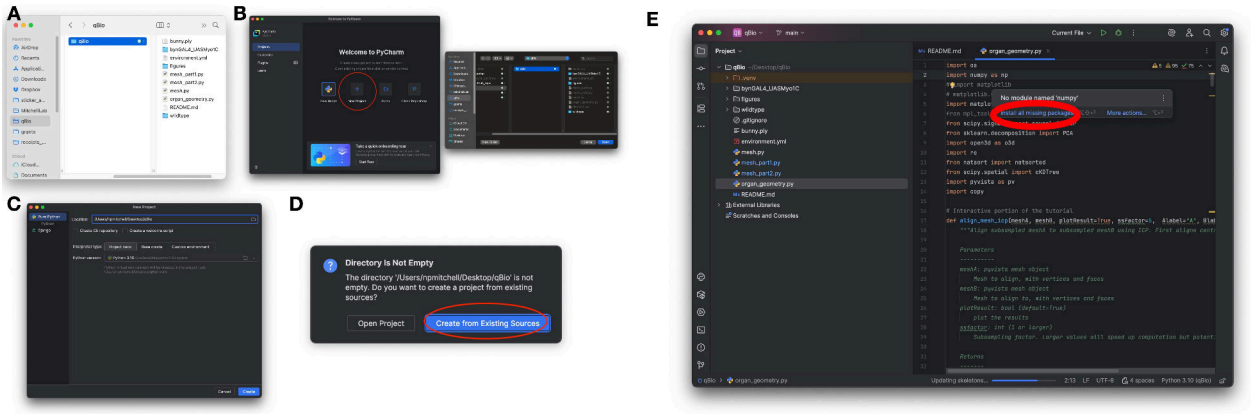



Figure 1: Setting up in PyCharm.

*0a. Python*

Python provides several core data structures that are widely used in both everyday scripting and advanced scientific workflows. Understanding their characteristics will help you choose the right tool for the job and write clearer, more efficient code.

We will work with the following fundamental data structures:

- **Lists**
  Lists are ordered, mutable collections of elements. They can hold elements of different types, and are typically used for sequential data or collections you plan to modify.
  *Example*: `mylist = [3.2, "gut", 5]`
- **Tuples**
  Tuples are like lists, but immutable. This means their contents cannot be changed after creation. Tuples are useful for representing fixed-size records or coordinates.
  *Example*: `coords = (1.0, 2.0, 3.0)`
- **Dictionaries**
  Dictionaries store key-value pairs. They are ideal for associating labels or identifiers with values, like mapping timepoints to filenames or experimental conditions to measurements.
  *Example*: `mydict = {"wt": "20240527", "oe": "20240626"}`
- **Arrays (from NumPy)**
  NumPy arrays are powerful containers for numerical data. Unlike lists, they support efficient mathematical operations and are used heavily in scientific computing, especially for vectors, matrices, and multidimensional datasets.
  *Example*: `arr = np.array([[1, 2], [3, 4]])`

Throughout this module, we'll use these data structures to work with 2D and 3D triangulated meshes, align shapes across space and time, and track shape variations of a model organ.

### 0c. The embryonic fly midgut

During development, tissues integrate mechanical and biochemical signals to drive organ-scale geometric transformations. Left-right symmetry breaking is a fundamental feature of organ mor-

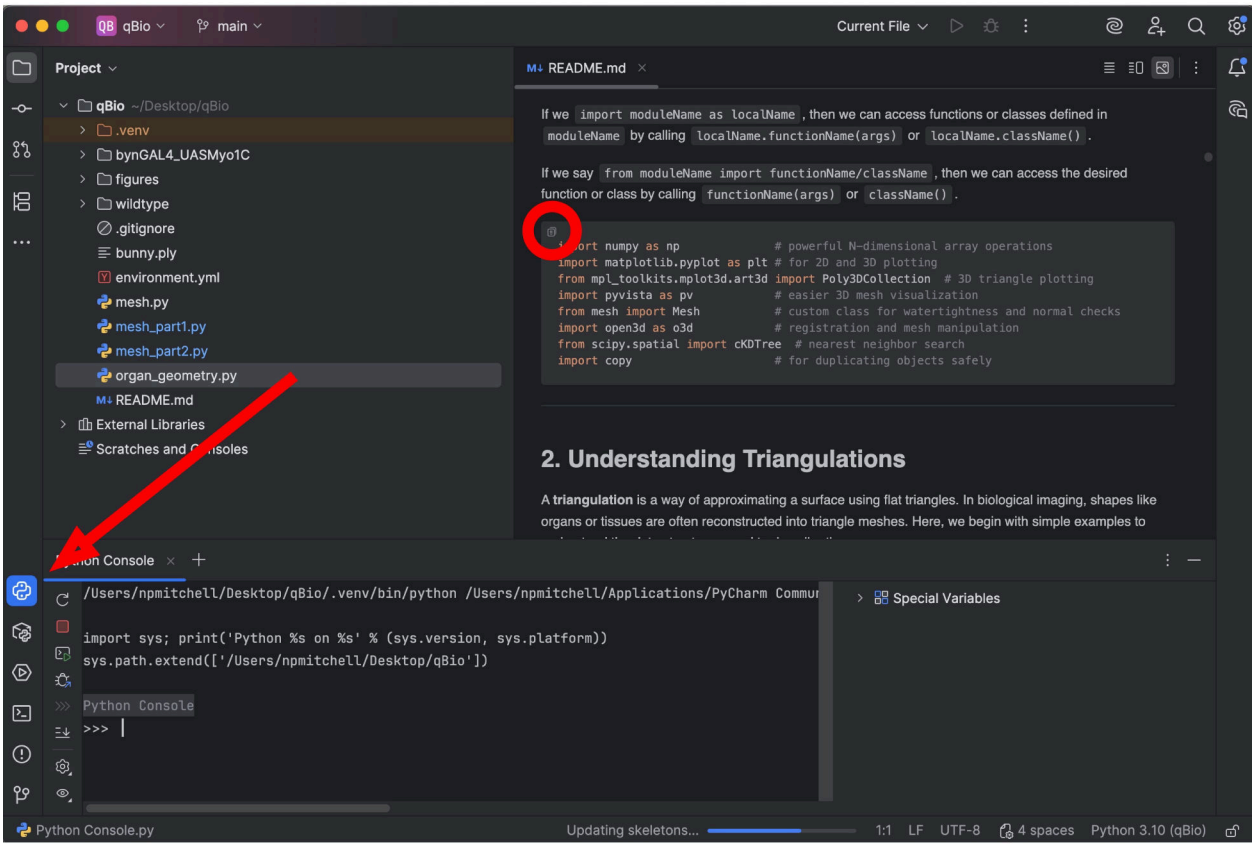



Figure 2: Running this module in PyCharm.

phogenesis that is crucial for organ function. While left-right asymmetric genetic patterning of the early embryonic body plan can trickle down to influence organ scale asymmetries, there is also a role for cell-intrinsic processes: cells themselves break left-right symmetry through cytoskeletal chirality. Remarkably, tuning the concentration of certain molecular motors such as members of the unconventional myosin 1 protein family can invert organ chirality across vertebrates and invertebrates alike, but we currently lack a physical understanding of the tissue-scale forces that drive organ-scale chirality. One direction in the Mitchell Lab aims to uncover the mechanical forces that drive organ asymmetry and relate them to cell-intrinsic activity of Myo1C. Here in this tutorial, we'll look at the chiral shape dynamics of the *Drosophila* midgut as a model system. We will ask: to what extent are the guts of wild-type and embryos with myosin 1C overexpression mirror images of one another?

The data you are working with was taken from live, multiview lightsheet microscopy imaging of the midgut across ~2.5 hrs of development. We used computer vision tools (Mitchell & Cislo, *Nature Methods* 2023) to extract the tissue surface over time. Briefly, we trained a two-stage 3D pixel classifier (iLastik autocontext workflow) to label the pixels of the endoderm and yolk as one probability channel and the muscle layer as another, then identified a 3D segmentation of the gut. The segmentation minimized an energy functional with surface tension, pressure, and attachment terms (see Active Contours Without Edges, see Márquez-Neila et al, *IEEE Trans Pattern Anal Mach Intell* 2014 and Chan & Vese, *IEEE Transactions on Image Processing* 2001). Marching Cubes identified the segmentation's surface as a triangulation, which we then smoothed with a

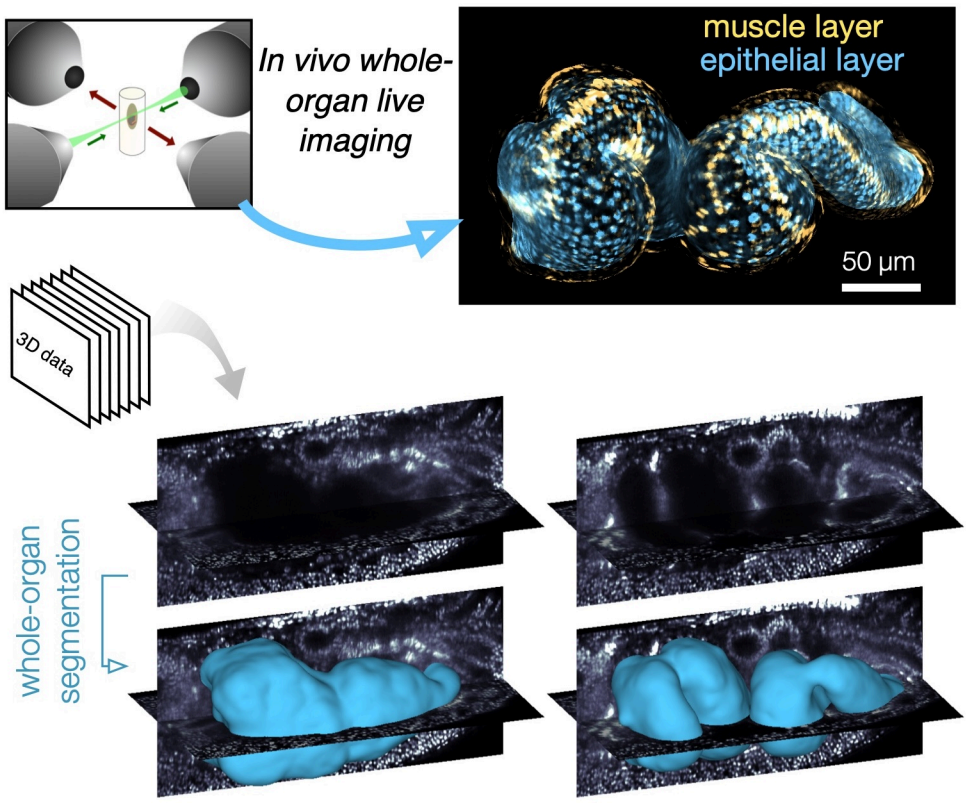



Figure 3: Multiview lightsheet imaging and computer vision tools enable analysis of chiral shape changes in gut morphogenesis.

Poisson disk reconstruction and Laplacian filters. Here you are presented with the resulting mesh triangulations.

### 1. Setup and Imports

Before we begin, we import the necessary Python libraries for 3D geometry processing, visualization, and numerical operations. Each of these libraries is a module. Note that you can write your own modules and import them, as long as they are placed somewhere that python can find them (ex in the current working directory or installed globally). As an example, we import the Mesh class from mesh.py, which is included in this tutorial.

If any of these other statements return an error, simply install the required module into your virtual environment in PyCharm (or on terminal if you have a Conda environment).

If we `import moduleName`, then we can access functions or classes defined in `moduleName` by calling `moduleName.functionName(args)` or `moduleName.className()`.

If we `import moduleName as localName`, then we can access functions or classes defined in `moduleName` by calling `localName.functionName(args)` or `localName.className()`.

If we say `from moduleName import functionName/className`, then we can access the desired function or class by calling `functionName(args)` or `className()`.

```
import numpy as np                # powerful N-dimensional array operations
import matplotlib.pyplot as plt # for 2D and 3D plotting
from mpl_toolkits.mplot3d.art3d import Poly3DCollection  # 3D triangle plotting
import pyvista as pv              # easier 3D mesh visualization
from mesh import Mesh             # custom class for watertightness and normal checks
import open3d as o3d              # registration and mesh manipulation
from scipy.spatial import cKDTree  # nearest neighbor search
import copy                       # for duplicating objects safely
```

## 2. Understanding Triangulations

A **triangulation** is a way of approximating a surface using flat triangles. In biological imaging, shapes like organs or tissues are often reconstructed into triangle meshes. Here, we begin with simple examples to understand the data structures used to describe them.

A mesh consists of:

- a **list of vertices** (points in 2D or 3D space, stored as a NumPy array)
- a **list of faces** (which connect 3 or more vertex indices to form polygons)

### *2a. An example triangulation: the Stanford Bunny*

Let's first learn by example. We read in a PLY file of the Stanford Bunny, provided by the Stanford Computer Graphics Laboratory.

```
m = pv.read('bunny.ply')
print(m)
m.plot(show_edges=True)
```

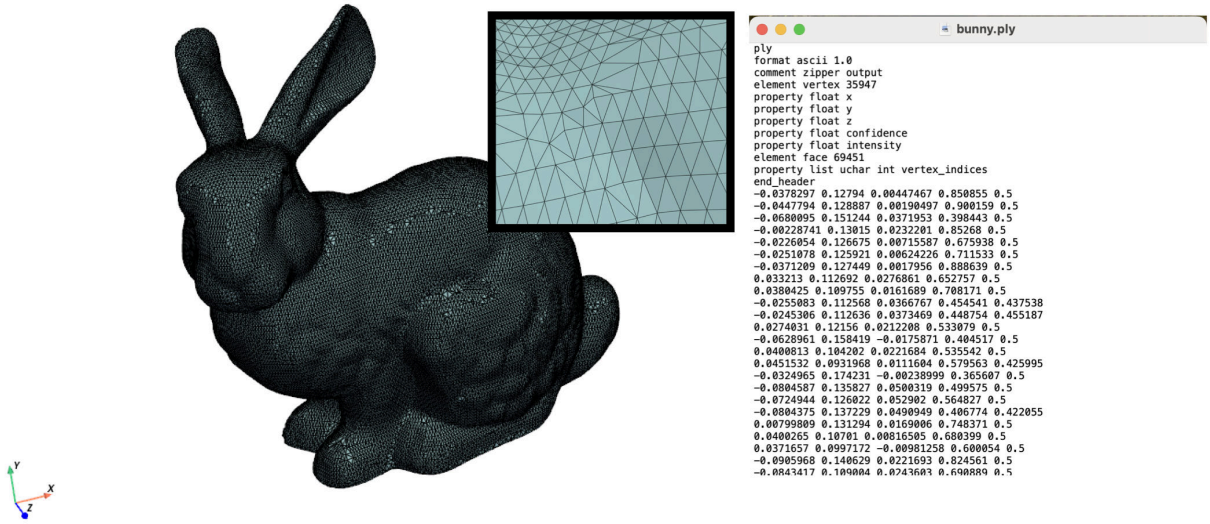



Figure 4: The Stanford bunny as a mesh triangulation, composed of vertices and faces.

How is this defined? It has vertices and faces. A face connects three vertices. Because the faces are triangles, this is a *triangulation*.

Open up `bunny.ply` in a text editor. See how it has vertices and faces defined (Figure 4).

### *2b. A Single Triangle*

This is the simplest mesh, made of 3 points and 1 triangle. Here `x` and `y` are NumPy arrays and `faces` is a Python list of vertex index triplets.

```
x = np.array([0, 1, 0])          # x-coordinates of the vertices
y = np.array([0, 0, 1])          # y-coordinates
faces = [[0, 1, 2]]              # one triangle made from those three points

# Plotting in python: use matplotlib.pyplot
# Let's make lines that connect the vertices.
# Concept check: Why do we have to index into faces?
plt.figure()
plt.plot(x[faces[0]], y[faces[0]], '-')
plt.title('single triangle...almost')
plt.axis('equal')
plt.show()
```

Coding concept check: why is there a missing edge in the plot?

```
# Go all the way around!
face = faces[0]
plt.figure()
plt.plot(np.append(x[face], x[face[0]]), np.append(y[face], y[face[0]]), '-o')
plt.title('A Single Triangle')
plt.axis('equal')
plt.show()
```

Coding concept check: do you understand what appending the 0th element does above?

We can also do this with triplot():

```
plt.figure()
plt.triplot(x, y, faces, color='black')
plt.plot(x, y, 'o', color='blue')  # plot the points as dots
plt.title('A Single Triangle')
plt.axis('equal')
plt.show()
```

### *2c. A Triangulated Square*

We build a square from two triangles.

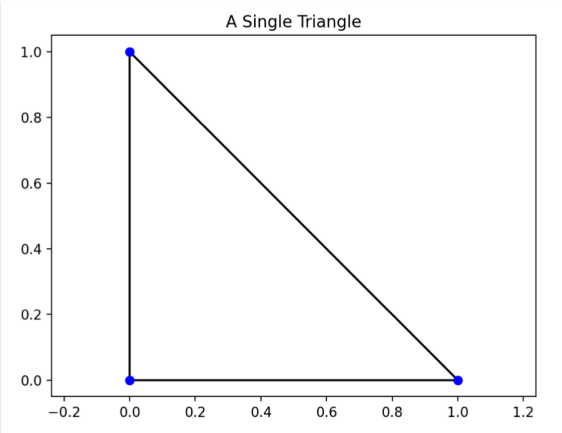



Figure 5: A single triangle plotted using `triplot`.

```
x = np.array([0, 1, 0, 1])       # 4 corner points of a square
y = np.array([0, 0, 1, 1])
faces = [[0, 1, 2], [1, 3, 2]]   # two triangles

plt.figure()
plt.triplot(x, y, faces, color='black')
plt.plot(x, y, 'o', color='blue')
plt.title('2D Triangulation of a Square')
plt.axis('equal')
plt.show()
```

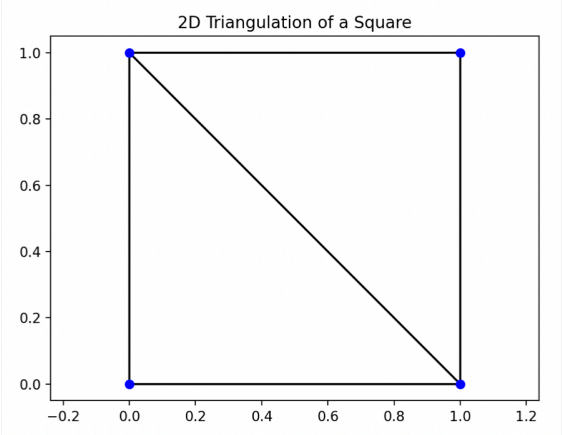



Figure 6: A square composed of two triangles.

*2d. A Triangulated Cube in 3D*

Now we move to 3D, building a cube from 8 corner points and 12 triangles. Can you build this yourself?

```
vertices = np.array([
    [0, 0, 0],  # 0
    [1, 0, 0],  # 1
    [1, 1, 0],  # 2
    [0, 1, 0],  # 3
    [0, 0, 1],  # 4
    [1, 0, 1],  # 5
    [1, 1, 1],  # 6
    [0, 1, 1]   # 7
])

# Define faces (two per face, 12 faces total)
faces = [
    [0, 2, 1], [0, 3, 2],  # bottom face
    [4, 5, 6], [4, 6, 7],  # top face
    [0, 1, 5], [0, 5, 4],  # front face
    [2, 3, 7], [2, 7, 6],  # back face
    [0, 7, 3], [0, 4, 7],  # left face
    ?????? <Fill in here>   # right face
]

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
tri_faces = [[vertices[i] for i in tri] for tri in faces]  # list of 3D triangles
ax.add_collection3d(Poly3DCollection(tri_faces, facecolors='cyan',
                                     edgecolors='black', alpha=0.8))
ax.scatter(vertices[:,0], vertices[:,1], vertices[:,2], color='blue')

# Set labels
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')

# Set equal aspect ratio
ax.set_box_aspect([1, 1, 1])
plt.title('3D Triangulation of a Cube')
plt.show()
```

Try moving it around.

### 3. Mesh Validation

Before doing analysis or alignment, we often must ensure that the mesh is well-formed:

- **Watertightness**: no missing connections or open edges.
- **Consistent Orientation**: all face normals point outward.

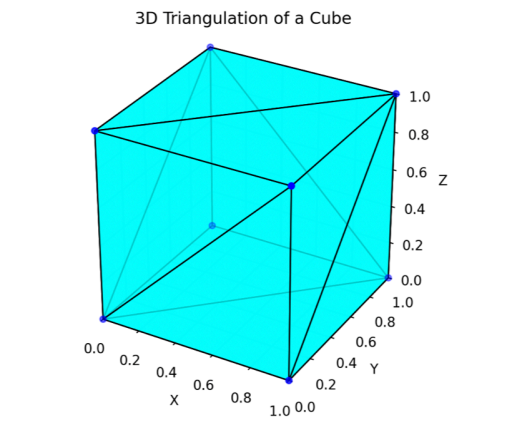



Figure 7: Triangulated cube from 8 vertices and 12 triangles.

*3a. Watertightness Check*

Let's count how many times each edge appears. If watertight, it should appear twice. This demo highlights the use of dictionaries.

```
# Initialize an empty dictionary to count edges
edge_count = {}

# Loop over all faces
for tri in faces:
    # Extract edges from the triangle
    edges = [
        (min(tri[0], tri[1]), max(tri[0], tri[1])),
        (min(tri[1], tri[2]), max(tri[1], tri[2])),
        (min(tri[2], tri[0]), max(tri[2], tri[0]))
    ]

    # Count how many times each edge appears
    for edge in edges:
        if edge in edge_count:
            edge_count[edge] += 1
        else:
            edge_count[edge] = 1

# Collect edges that appear < 2 or > 2 times (open or non-manifold edges)
open_edges = []
for edge in edge_count:
    if edge_count[edge] != 2:
        open_edges.append((edge, edge_count[edge]))

# Report
if len(open_edges) == 0:
```

```
    print("The mesh is closed (watertight).")
else:
    print("The mesh is NOT closed. Problematic edges:")
    for edge, count in open_edges:
        print(f"  Edge {edge} appears {count} times")
```

Concept check: why did we use min() and max() in the definition of edges?

*3b. Face Orientation (somewhat advanced)*

Recall that the order of a triangle matters: the order of vertex indices listed for a triangle determines which way is "in" or "out" in 3D. Here we do a simple check for whether the faces of the cube are pointing the right way. Let's use dot products to check if face normals point away from the mesh center. A dot product measures the amount of alignment between two vectors. If the face's "normal" direction (ie its "outward" direction) is pointing in a similar direction as the vector pointing from the origin to the face, then the dot product will be positive. If the face is oriented the wrong way, then the dot product is negative.

```
# Compute cube center
center = np.mean(vertices, axis=0)

def is_outward_facing(tri):
    v0, v1, v2 = vertices[tri[0]], vertices[tri[1]], vertices[tri[2]]
    # Compute normal vector
    normal = np.cross(v1 - v0, v2 - v0)
    # Vector from the center of the triangle to center of the cube
    tri_center = (v0 + v1 + v2) / 3
    from_center = tri_center - center
    # Dot product tells us if the normal is pointing toward or away from the center
    return np.dot(normal, from_center) > 0  # Should be negative if pointing outward

# Check all faces
inward_facing = []
for i, tri in enumerate(faces):
    if not is_outward_facing(tri):
        inward_facing.append(i)

# Report
if len(inward_facing) == 0:
    print("All faces are consistently outward-facing.")
else:
    print("Found inward-facing faces at indices:")
    print(inward_facing)
```

Concept check: This works for a cube. Would it work for a more complex shape (ex the Stanford bunny)? Provide a simple example that fails, and verify this with code.

### 3c. Advanced checks

Challenge: how does this check work? At the least, make sure you can follow the structure: we use the `Mesh` class imported from mesh.py. The class has a *method* called `is_closed`. A method is like a function that belongs to a class. The class instance `mm` executes the method `is_closed()` when we say `mm.is_closed()`, effectively asking itself, 'Am I closed?'

```
mm = Mesh(vertices, faces) mm.is_closed()

# To fix face orientations using Mesh() class, do the following:
mm.make_normals()
mm.force_z_normal(direction=1)
```

### 4. Loading meshes from microscopy data

We can now load some experimentally-derived 3D triangle meshes — for example, segmentations from a microscopy dataset of the *Drosophila* gut. We've saved the mesh as a `.ply` file. Take a look at the PLY file in a text editor. Note that all the information about the contents are in the header. First the vertices are listed, then the faces. The header declares what information is in the file, how many vertices, how many faces, etc.

Below we use the `pyvista.PolyData` class to provide convenient 3D plotting for meshes. You can use the more standard `matplotlib.pyplot` functions too, but they are going to be laggy for meshes of this size (or if you don't have enough RAM, they will make your python console freeze up).

```
m = pv.read('./wt/20240527/mesh_000000_APDV_um.ply')  # returns a PyVista mesh object
m.plot(show_edges=True)
```

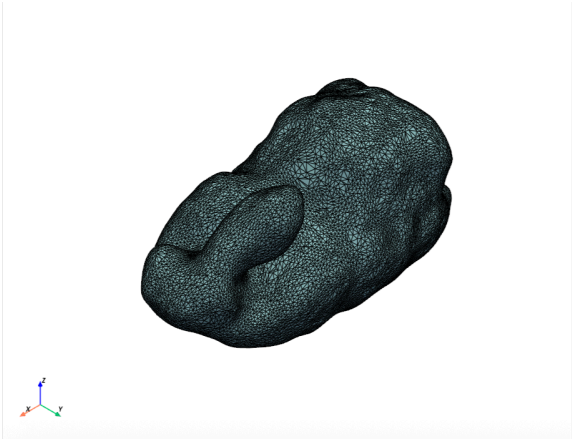


Figure 8: Example mesh from level sets and Poisson disk surface reconstruction.

## 5. Shape Matching Using ICP

The **Iterative Closest Point (ICP)** algorithm finds the rigid transform (rotation + translation) that best aligns two point clouds or meshes.

*5a. Generate and Offset Meshes*

Let's load a couple of meshes – two guts from different embryos at a similar developmental stages. We load these as `TriangleMesh` class instances from Open3D, but don't let that scare you: they are just containers for vertices and faces. We then compute the optimal translation and rotation that best matches one mesh to the other one. I packaged this optimization in a function in `organ_geometry.py`.

```
from organ_geometry import align_mesh_icp, color_mesh_by_distance

ssfactor = 10 # subsampling factor (take 1/N points for the ICP registration).
              # Larger values will run faster but be less precise. This is for
              # speedup

print('Reading in meshes...')
fA = "wildtype/20240527/mesh_000040_APDV_um.ply"
meshA = pv.read(fA)

fB = "wildtype/20240531/mesh_000050_APDV_um.ply"
meshB = pv.read(fB)

# Align one mesh to the other
meshA_t, T = align_mesh_icp(meshA, meshB, ssfactor=ssfactor)
```

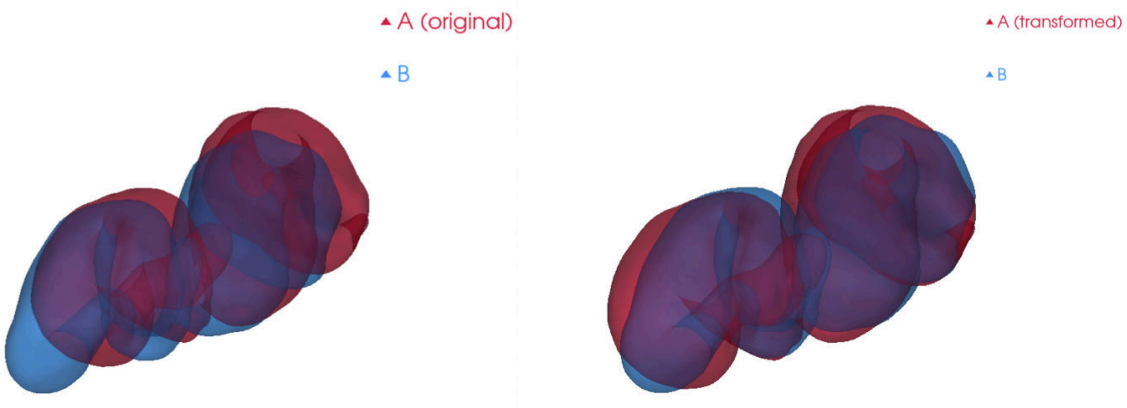



Figure 9: Two different embryos' midguts, captured at a similar stage of development, aligned in space.

Here `T` is a 4×4 NumPy array representing the optimal rigid transform. Advanced concept check: how are data stored inside `T`? Hint: There is a matrix encoding rotations and scaling, but also a translation (dx, dy, dz). Which columns/rows are which?

*5c. Compute and Color Distance*

We measure how close the aligned mesh is to the target and color it by the distance from the nearest matching point.

```
# Color the mesh by distance between the two
color_mesh_by_distance(meshA_t, meshB, transform=None)
```

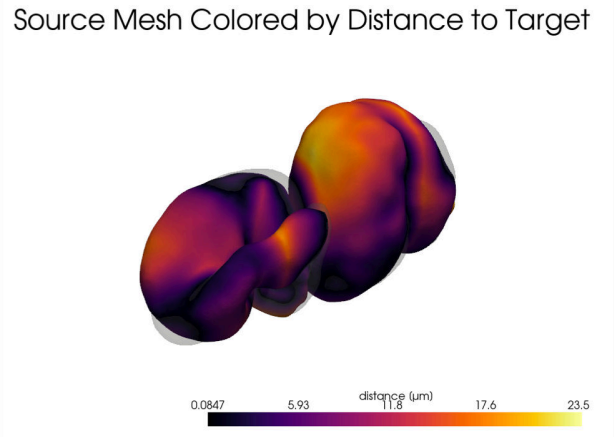



Figure 10: The Euclidean distance between two gut meshes.

Under the hood, in another function I put in `organ_geometry.py`, we computed the distance to the nearest meshB vertex for each vertex in meshA. Then we converted those distances to a color array, a NumPy array with shape `(N, 3)` for RGB values. How does your alignment look?

*5d. Generate and Offset Meshes*

The guts we've been looking at are from a wildtype embryo with three different fluorescent markers (full genotype is *w, HandGFP / w; UAS-mCh:CAAX / +;bynGAL4, klar / H2A-iRFP*. Let's compare to the gut of an embryo that overexpresses Myosin1C (full genotype is *w, HandGFP / w; UAS-Myo1C:RFP / +;bynGAL4, klar / H2A-iRFP*). In this case, the unconventional myosin motor acts in a portion of the digestive tract (the *byn* domain) to invert the chiral shape dynamics.

How similar are these two organ morphologies?

Let's first compare them directly.

```
print('Reading in meshes...')
fA = "wildtype/20240527/mesh_000050_APDV_um.ply"
meshA = pv.read(fA)

fB = "bynGAL4_UASMyo1C/20240528/mesh_000052_APDV_um.ply"
meshB = pv.read(fB)

# Align one mesh to the other
meshA_t, T = align_mesh_icp(meshA, meshB, ssfactor=ssfactor, Alabel="WT",
```

```
                              Blabel='byn>Myo1C')

# Color the mesh by distance between the two
dists, plotter = color_mesh_by_distance(meshA_t, meshB, transform=None)
plotter.show()
```

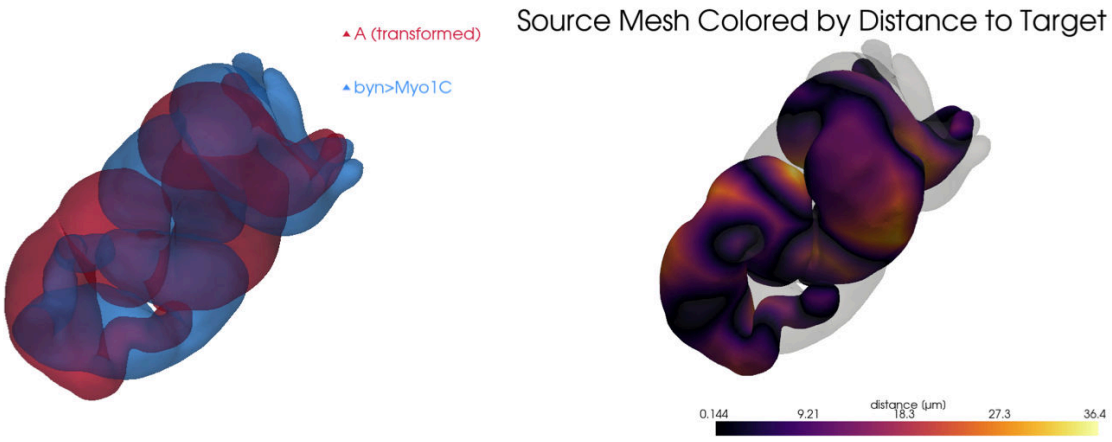



Figure 11: Overexpression of Myo1C dramatically changes the gut shape.

Now let's flip one across the left-right axis and compare them again.

```
# Now flip y -> -y in the Myo1C OE case
meshA = pv.read(fA)
meshB = pv.read(fB)
meshB.points[:, 1] *= -1

# Align one mesh to the other
meshA_t, T = align_mesh_icp(meshA, meshB, ssfactor=ssfactor, Alabel="WT",
                              Blabel='byn>Myo1C, mirrored')

# Color the mesh by distance between the two
dists, plotter = color_mesh_by_distance(meshA_t, meshB, transform=None)
plotter.show()
```

## Part 2: Timepoint Alignment and Spatiotemporal Comparison

This second part builds on our introductory ICP module to analyze time-series data from a developing embryonic gut in WT conditions and with Myo1C overexpression. We align shapes over time, compare shape dynamics between experimental conditions, and visualize mismatch using point cloud alignment. This module assumes that triangulated mesh files (`.ply`) are already segmented from microscopy and stored in directories corresponding to timepoints.

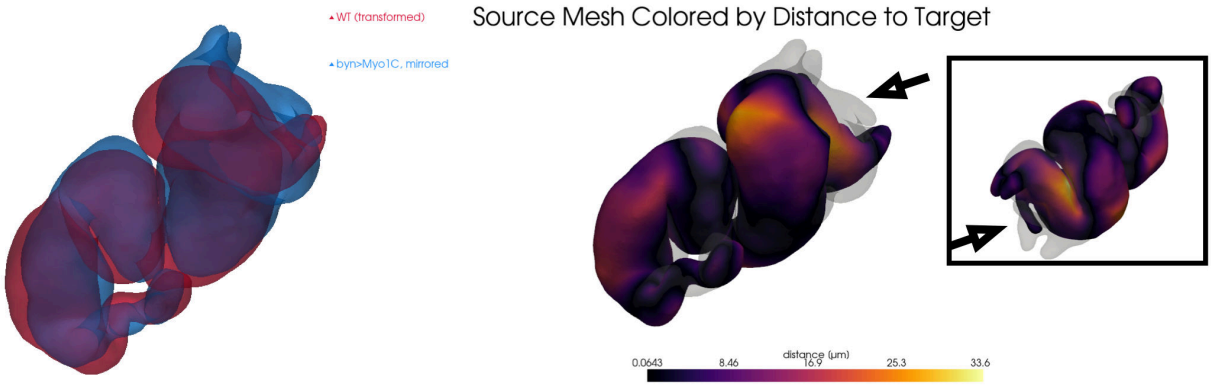



Figure 12: Overexpression of Myo1C nearly mirrors (inverts along the left-right axis) the gut shape. Pronounced residual mismatch is visible in the anterior chamber (arrowhead).

### 6. Setup and Parameters

We begin by importing libraries and setting parameters. These include which experimental conditions we want to compare (`wt`, `oe`, etc.) and how much to subsample meshes for faster computation.

```
from organ_geometry import *
import pandas as pd

# Main parameters
step = 3            # downsampling step in units of timepoints
                    # (higher will run faster and consider fewer timepoints)
ssfactor = 10       # spatial subsampling factor for ICP
                    # (higher is faster but potentially less accurate)
wt_oe = 'wt'        # comparison will initially be between two WT datasets
outdir = 'results' # where to store output on disk
dt = 2              # timestep in minutes between adjacent timepoints
                    # (this should not be changed, taken from experiments)

if not os.path.exists(outdir):
    os.mkdir(outdir)

dirA = "wildtype/20240527/"
dirB = "wildtype/20240531/"
flipy = False
```

### 7. Timepoint Matching with ICP

This section performs pairwise shape comparison using ICP to align all meshes from `dirA` to those in `dirB`. ICP produces a matrix of matching errors, which we smooth and use to infer a time-mapping between the two datasets.

```
# Collect all PLY files and extract their timepoint numbers
# List and sort .ply files
filesA = []
for f in os.listdir(dirA):
    if f.endswith(".ply"):
        filesA.append(f)

filesA = natsorted(filesA)[::step]

# Do the same for dirB
filesB = []
for f in os.listdir(dirB):
    if f.endswith(".ply"):
        filesB.append(f)

filesB = natsorted(filesB)[::step]

# Extract timepoints from filenames
tpsA = np.array([extract_tp(f) for f in filesA])
tpsB = np.array([extract_tp(f) for f in filesB])

# Compute ICP RMSE
icp_raw = build_icp_cost_matrix(dirA, dirB, ssfactor=ssfactor, step=step,
                                flipy=flipy)
icp_smooth = smooth_icp_matrix(icp_raw)
```

Now plot the result.

```
ig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4), sharey=True)
im1 = ax1.imshow(icp_raw, cmap='inferno')
ax1.set_title("Timeline comparison via ICP")
ax1.set_xlabel("Time B")
ax1.set_ylabel("Time A")
fig.colorbar(im1, ax=ax1, label="ICP Error")

im2 = ax2.imshow(icp_smooth, cmap='inferno')
ax2.set_title("Timeline comparison via ICP")
ax2.set_xlabel("Time B")
ax2.set_ylabel("Time A")
fig.colorbar(im2, ax=ax2, label="Smoothed ICP Error")

plt.show()
```

Match timepoints in this matrix by finding row and column minima.

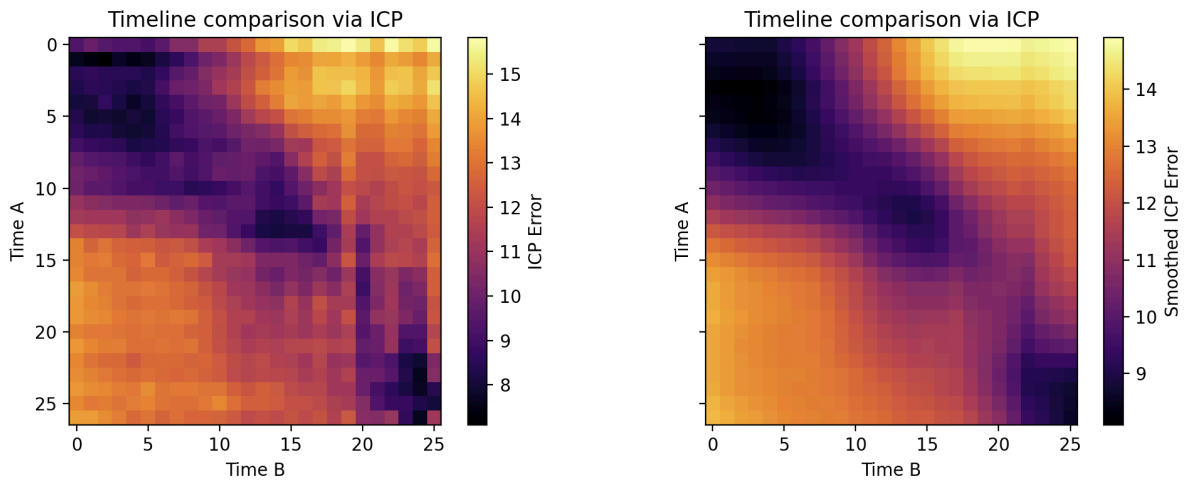



Figure 13: Raw and smoothed root-mean-squared error measurements after ICP alignment for two different WT datasets.

```
# Match timepoints
AtoB, BtoA = match_timepoints(icp_smooth)

# Advanced: using dynamic time warping, we optimize with monotonicity
[path, _, _] = dtw_match(icp_smooth)
path = np.array(path)
path_tps = np.array([tpsA[path[:, 0]], tpsB[path[:, 1]]]).T
```

### 8. Visualizing Timepoint Matching

Plot the smoothed ICP error matrix, with the dynamic time warping path overlayed in red. This helps visualize which timepoints from A match which timepoints from B.

```
plt.figure()
plt.imshow(icp_smooth, cmap='inferno')
plt.plot(path[:,1], path[:, 0], '.-', lw=2, color='tab:purple')
plt.plot(AtoB, np.arange(len(tpsA)), '.-', lw=2, color='tab:blue')
plt.plot(np.arange(len(tpsB)), BtoA, '.-', lw=2, color='tab:orange')
plt.title("Timepoint Matching")
plt.xlabel("Time B")
plt.ylabel("Time A")
plt.colorbar(label="ICP Error")
plt.axis('equal')
plt.tight_layout()
plt.show()
```

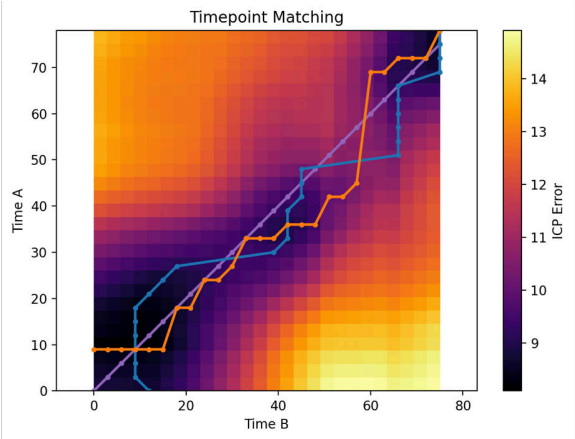



Figure 14: The paths represent AtoB alignment (blue), BtoA alignment (orange), and the result of a simple dynamic time warping implementation (purple).

```
# Plot the result
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4), sharey=True)

# === Subplot 1: A→B ===
ax1.plot(tpsA, icp_raw[np.arange(len(tpsA)), AtoB], 'o--', label='A→B raw',
         color='tab:blue')
ax1.plot(tpsA, icp_smooth[np.arange(len(tpsA)), AtoB], 'o-', label='A→B smoothed',
         color='tab:blue')
# ax1.plot(path_tps[:, 0], icp_smooth[path[:, 0], path[:, 1]], '*-',
#          label='DTW smoothed (A→B)', color='tab:blue')
ax1.set_xlabel("Timepoint A")
ax1.set_ylabel("ICP Error")
ax1.set_title("A→B Matching")
ax1.grid(True)
ax1.legend()

# === Subplot 2: B→A ===
ax2.plot(tpsB, icp_raw[BtoA, np.arange(len(tpsB))], 'o--', label='B→A raw',
         color='tab:orange')
ax2.plot(tpsB, icp_smooth[BtoA, np.arange(len(tpsB))], 'o-', label='B→A smoothed',
         color='tab:orange')
# ax2.plot(path_tps[:, 1], icp_smooth[path[:, 0], path[:, 1]], '*-',
#          label='DTW smoothed (B→A)', color='tab:orange')
ax2.set_xlabel("Timepoint B")
ax2.set_title("B→A Matching")
ax2.grid(True)
ax2.legend()
plt.savefig(os.path.join(outdir,
  f'TPMatching_AtoB_BtoA_{wt_oe}_step{step}_ss{ssfactor}.png'))
plt.show()
```

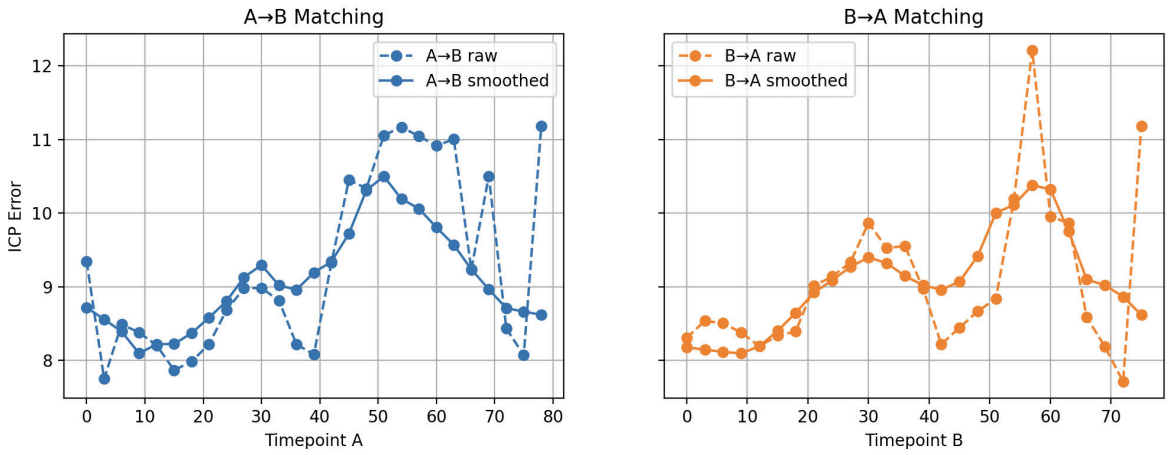



Figure 15: The average root-mean-squared mismatch at each timepoint along the AtoB and BtoA paths show that cross-shape variation rises slightly over time.

### 9. Expected ICP Error from Subsampling

We estimate the smallest error we can expect from subsampling, based on average spacing between points. This gives a baseline to interpret ICP error values.

```
mean_spacing_A = []
mean_spacing_B = []

for fA in filesA:
    # Load and subsample point cloud
    vA = pv.read(os.path.join(dirA, fA)).points[::ssfactor]
    # Compute and store mean spacing
    mean_spacing_A.append(mean_point_spacing(vA))

for fB in filesB:
    # Load and subsample point cloud
    vB = pv.read(os.path.join(dirB, fB)).points[::ssfactor]
    # Compute and store mean spacing
    mean_spacing_B.append(mean_point_spacing(vB))

mean_spacing_A = np.array(mean_spacing_A)
mean_spacing_B = np.array(mean_spacing_B)
expected_rmse_A = mean_spacing_A / np.sqrt(2)
expected_rmse_B = mean_spacing_B / np.sqrt(2)
```

Now let's plot what this looks like.

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4), sharey=True)

# === Subplot 1: A→B ===
```

```
ax1.plot(tpsA, icp_raw[np.arange(len(tpsA)), AtoB], 'o--', label='A→B raw',
         color='tab:blue')
ax1.plot(tpsA, icp_smooth[np.arange(len(tpsA)), AtoB], 'o-', label='A→B smoothed',
         color='tab:blue')
ax1.plot(tpsA, expected_rmse_A, '.--', label='B→A baseline', color='tab:blue')
# ax1.plot(path_tps[:, 0], icp_smooth[path[:, 0], path[:, 1]], '*-',
#          label='DTW smoothed (A→B)', color='tab:blue')
ax1.set_xlabel("Timepoint A")
ax1.set_ylabel("ICP Error")
ax1.set_title("A→B Matching")
ax1.grid(True)
ax1.legend()

# === Subplot 2: B→A ===
ax2.plot(tpsB, icp_raw[BtoA, np.arange(len(tpsB))], 'o--', label='B→A raw',
         color='tab:orange')
ax2.plot(tpsB, icp_smooth[BtoA, np.arange(len(tpsB))], 'o-', label='B→A smoothed',
         color='tab:orange')
ax2.plot(tpsB, expected_rmse_B, '.--', label='A→B baseline', color='tab:orange')
# ax2.plot(path_tps[:, 1], icp_smooth[path[:, 0], path[:, 1]], '*-',
           label='DTW smoothed (B→A)', color='tab:orange')
ax2.set_xlabel("Timepoint B")
ax2.set_title("B→A Matching")
ax2.grid(True)
ax2.legend()
plt.savefig(os.path.join(outdir,
  f'TPMatching_AtoB_BtoA_expectedRMSE_{wt_oe}_step{step}_ss{ssfactor}.png'))
plt.show()
```

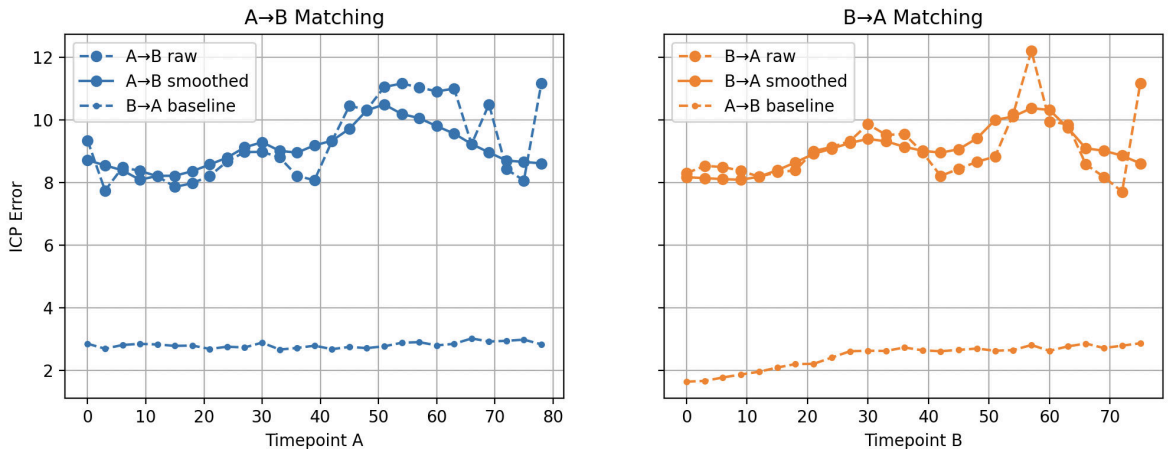



Figure 16: The expected baseline ICP error from finite sampling of the meshes lies far below the measured values.

### 10. Visualize Aligned Meshes

This section loads each matching pair of meshes, re-runs ICP alignment, and overlays them to assess alignment quality. We apply a pre-alignment translation to center the shapes, which improves optimization.

```
# ---------------------------------------
# Batch overlays
# ---------------------------------------
xyzlim = compute_global_bounds(dirA, dirB, filesA, filesB)
batch_icp_overlay(dirA, dirB, filesA, filesB, AtoB,
                  outdir=os.path.join(outdir,
                    f"icp_overlay_{wt_oe}_step{step}_ss{ssfactor}"),
                  ssfactor=ssfactor, xyzlim=xyzlim, flipy=flipy,
                  Alabel=dirA, Blabel=dirB)
```

### 11. Measuring the spatial pattern of misalignment between samples

We loop over all timepoints and visualize distance between each aligned mesh pair. The color intensity shows how well two shapes match locally. The output is saved to the `results` directory.

```
# ---------------------------------------
# Batch color the meshes by mismatch distance
# ---------------------------------------
clims = (0, 20)
batch_color_by_distance(dirA, dirB, filesA, filesB, AtoB,
                        outdir=os.path.join(outdir,
                          f"colored_distance__{wt_oe}_step{step}_ss{ssfactor}"),
                        ssfactor=ssfactor, xyzlim=xyzlim,
                        clim=clims, flipy=flipy)
```

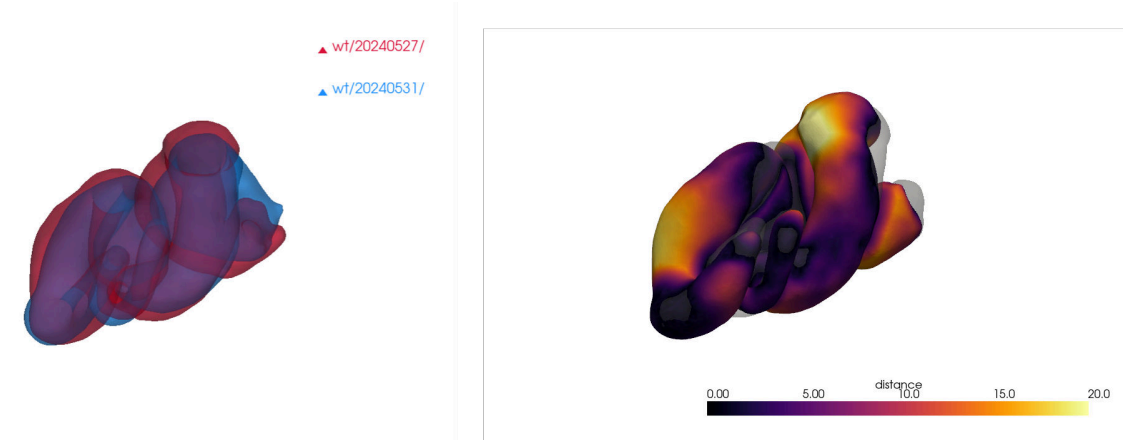



Figure 17: Spatial alignment of two WT midguts at an example timepoint.

### 12. Advanced: PCA-based smoothing as an alternative to dynamic time warping

Dynamic time warping (DTW) gives a monotone, potentially jagged A-to-B mapping (AtoB, BtoA), but sometimes lies "outside" of either set (AtoB or BtoA). Here we instead takes the matched timepoints between two series and smooths their relationship by projecting them into PCA space, smoothing along the orthogonal component, and map back to reconstruct a smoothed trajectory incorporating contributions from both AtoB and BtoA.

```
# ------------------------------------
# Advanced: an alternative to DTW is using both AtoB and
# BtoA to create consensus in PCA1 space.
# ------------------------------------
smoothed_curve = pca_smooth_correspondences(tpsA, tpsB, AtoB, BtoA)
```

We can view the results as usual.

```
plt.figure()
plt.imshow(icp_smooth, cmap='inferno', extent=[tpsB[0], tpsB[-1], tpsA[0], tpsA[-1]],
           origin='lower', aspect='auto')
plt.plot(path[:,1], path[:, 0], '.-', lw=2, color='tab:purple')
plt.plot(tpsB[AtoB], tpsA, '.-', lw=2, color='tab:blue')
plt.plot(tpsB, tpsA[BtoA], '.-', lw=2, color='tab:orange')
plt.plot(smoothed_curve[:, 1], smoothed_curve[:, 0], 'r.-', lw=2)
plt.title("PCA-Smoothed Timepoint Matching")
plt.xlabel("Time B")
plt.ylabel("Time A")
plt.colorbar(label="ICP Error")
plt.axis('equal')
plt.tight_layout()
plt.savefig(os.path.join(outdir,
  f'TPMatching_PCASm_{wt_oe}_step{step}_ss{ssfactor}.png'))
plt.show()
```

### 13. Final Export

We can use a pandas dataFrame for saving the result. This is like a table and we save the data as a csv.

```
# Save the results
icpRawA = icp_raw[np.arange(len(tpsA)), AtoB]
icpSmoothA = icp_smooth[np.arange(len(tpsA)), AtoB]
icpRawB = icp_raw[BtoA, np.arange(len(tpsB))]
icpSmoothB = icp_smooth[BtoA, np.arange(len(tpsB))]
df_icp = pd.DataFrame({
    "tpsA": tpsA,
    "AtoB": AtoB,
```

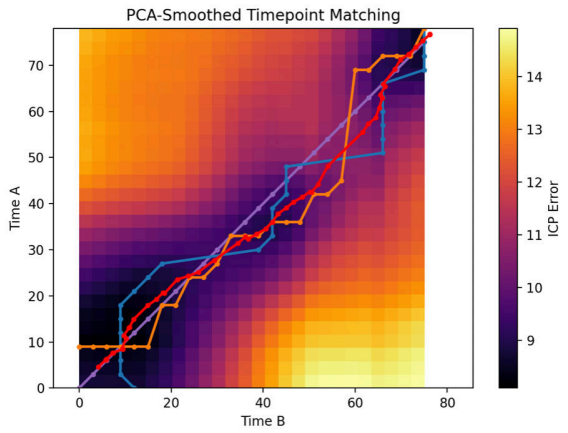



Figure 18: PCA-based smoothing (red curve) offers an alternative for reconciling the AtoB and BtoA timeline matching differences.

```
    "icpRawA": icpRawA,
    "icpSmoothA": icpSmoothA
})

df_icp_B = pd.DataFrame({
    "tpsB": tpsB,
    "BtoA": BtoA,
    "icpRawB": icpRawB,
    "icpSmoothB": icpSmoothB
})

df_icp.to_csv(f"icp_error_AtoB_{wt_oe}_step{step}_ss{ssfactor}.csv", index=False)
df_icp_B.to_csv(f"icp_error_BtoA_{wt_oe}_step{step}_ss{ssfactor}.csv", index=False)
```

You can save numeric results as `.npy` arrays for further analysis. These files store ICP matrices, expected noise floors, and timepoint mappings.

```
np.save(os.path.join(outdir,f"icp_raw_{wt_oe}_step{step}_ss{ssfactor}.npy"),icp_raw)
np.save(os.path.join(outdir,f"icp_smooth_{wt_oe}_step{step}_ss{ssfactor}.npy"),icp_smooth)
np.save(os.path.join(outdir,f"expected_rmse_A_{wt_oe}_step{step}_ss{ssfactor}.npy"),
                    expected_rmse_A)
np.save(os.path.join(outdir,f"expected_rmse_B_{wt_oe}_step{step}_ss{ssfactor}.npy"),
                    expected_rmse_B)
np.save(os.path.join(outdir,f"path_{wt_oe}_step{step}_ss{ssfactor}.npy"),[path, path_tps])
```

### 14. Cross-Condition Comparison

Run the above for different conditions `'wt', 'oe', 'x1', 'x2', 'x3', 'x4'`. We compare how ICP errors change across conditions. This can reveal if shape dynamics differ significantly between WT and perturbed embryos.

Note that whitespace matters in python. Just as delimiters like spaces or punctuation in English can completely change the meaning of a sentence — for example, “Let’s eat, Grandma” vs. “Let’s eat Grandma” — Python relies on whitespace, punctuation, and syntax to interpret code correctly.

```
for i, wt_oe in enumerate(conditions):
    if wt_oe == 'wt':
        # dirA = "HandGFPbynGAL4klar_UASmChCAAXHiFP/20240527/"
        # dirB = "HandGFPbynGAL4klar_UASmChCAAXHiFP/20240531/"
        dirA = "wt/20240527/"
        dirB = "wt/20240531/"
        flipy = False
    elif wt_oe == 'oe':
        dirA = "oe/20240528/"
        dirB = "oe/20240626/"
        flipy = False
    elif wt_oe == 'x1':
        dirA = "wt/20240527/"
        dirB = "oe/20240528/"
        flipy = True
    elif wt_oe == 'x2':
        dirA = "wt/20240527/"
        dirB = "oe/20240626/"
        flipy = True
    elif wt_oe == 'x3':
        dirA = "wt/20240531/"
        dirB = "oe/20240528/"
        flipy = True
    elif wt_oe == 'x4':
        dirA = "wt/20240531/"
        dirB = "oe/20240626/"
        flipy = True
    ...
    # All the batch analysis you ran above here
    ...

# ------------------------------------------------------------
# Compare difference between WT and OE against
# diff within WT and within OE
# ------------------------------------------------------------

conds = ['wt', 'oe', 'x1', 'x2', 'x3', 'x4']
icp_dict = {}

for cond in conds:
    fname = os.path.join(outdir, f"icp_error_AtoB_{cond}_step{step}_ss{ssfactor}.csv")
    df = pd.read_csv(fname)
    icp_dict[cond] = {
        "tps": df["tpsA"].values,
```

```
        "error": df["icpSmoothA"].values
    }

# Align on common timepoints
common_tps = np.intersect1d(np.intersect1d(np.intersect1d(icp_dict['wt']['tps'],
                                                          icp_dict['oe']['tps']),
                                           np.intersect1d(icp_dict['x1']['tps'],
                                                          icp_dict['x2']['tps'])),
                            np.intersect1d(icp_dict['x3']['tps'],
                                           icp_dict['x4']['tps']))
for key in icp_dict:
    mask = np.isin(icp_dict[key]["tps"], common_tps)
    icp_dict[key]["tps"] = icp_dict[key]["tps"][mask]
    icp_dict[key]["error"] = icp_dict[key]["error"][mask]

# Stack WT vs OE error
wt_err = icp_dict['wt']["error"]
oe_err = icp_dict['oe']["error"]
tps = icp_dict['wt']["tps"]

# Mean between-group difference
diff_wtoe = np.abs(wt_err - oe_err)
mean_diff = np.mean(diff_wtoe)

# Stack X1-X4 errors
x_errors = np.stack([icp_dict[f"x{i}"]["error"] for i in range(1, 5)])
x_mean = np.mean(x_errors, axis=0)
x_std = np.std(x_errors, axis=0)

# Plot
plt.figure(figsize=(10, 5))
plt.plot(tps*dt, wt_err, 'b.-', label="WT-WT")
plt.plot(tps*dt, oe_err, 'r.-', label="OE-OE")
plt.plot(tps*dt, x_mean, 'k-', label="WT-to-OE (mean x1-x4)")
plt.fill_between(tps*dt, x_mean - x_std, x_mean + x_std, color='gray', alpha=0.3,
                 label="WT-OE ± std")
plt.xlabel("reference time [min]")
plt.ylabel("ICP RMSE (smoothed)")
plt.title("Within- vs cross-ensemble ICP RMSE")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(outdir, 'cross_ensemble_RMSE.png'))
plt.show()

print(f"Mean WT-OE diff: {mean_diff:.3f}")
print(f"WT internal std: {np.std(wt_err):.3f}")
```

```
print(f"OE internal std: {np.std(oe_err):.3f}")
```

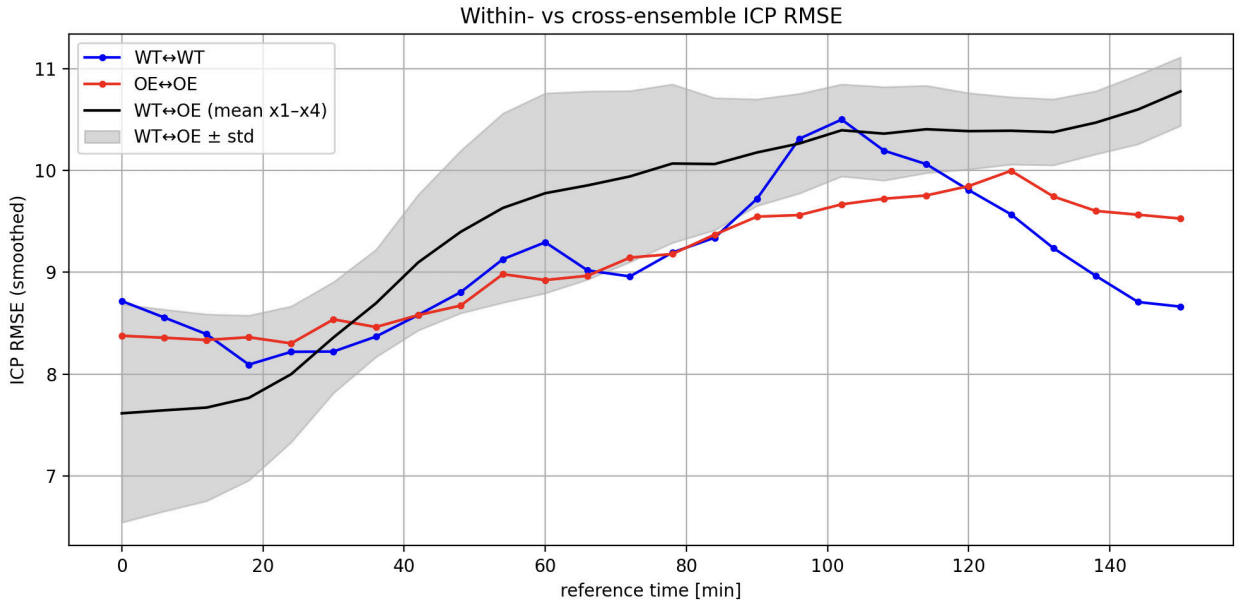



Figure 19: Comparing the distance between mesh pairs within and across groups (with the Myo1C OE case inverted L<->R).

This figure provides a quantitative summary of dynamic shape variability under genetic or experimental perturbation. How do you interpret these results?

### Next Steps

- Try aligning real meshes from your experiments
- Explore ICP variants (e.g., point-to-plane)
- Add mesh preprocessing: smoothing, simplification, hole filling
- Analyze biological trends in alignment error

# Population genetics workshop*

**Jeremy Berg** *University of Chicago*

This exercise is going to expose you to several basic ideas in probability and statistics as well as show you the utility of using R for basic statistical analyses. We'll do so in the context of a basic population genetic analysis.

*The scenario*

As a biologist, you will learn what are the major patterns that are expected when the data you work with is clean. Using that expertise will save you from the mistake of misinterpreting error-prone data. In population genetics, there are a number of patterns that we expect to see immediately in our datasets. In this exercise you will explore one of those major patterns. Rather than give it away — let's begin some analysis and see what we find. In the narrative that follows, we'll refine our thinking as we go.

*Introductory terminology*

- Single-nucleotide polymorphism (SNP): A nucleotide base-pair that is *polymorphic* (i.e. it has multiple types or *alleles* in the population)
- Allele: A particular variant form of DNA (e.g. A particular SNP may have the "A-T" allele in one DNA copy and "C-G" in another; We typically define a reference strand of the DNA to read off of, and then denote the alleles according to the reference strand base - so for example, these might be called simply the "A" and "C" alleles. In many cases we don't care about the precise base, so we might call these simply the $A_1$ and $A_2$ alleles, or the $A$ or $a$ alleles, or the 0 and 1 alleles.)
- Minor allele: The allele that is more rare in a population
- Major allele: The allele that is more common in a population
- Genotype: The set of alleles carried by an individual (E.g. AA, AC, CC; or AA, Aa, aa; or 0, 1, 2)
- Genotyping array: A technology based on hybridization with probes and florensence that allows genotype calls to be made at 100s of thousands of SNPs per individual at an affordable cost.

*The data-set and basic pre-processing*

We will look at Illumina 650Y genotyping array data from the CEPH-Human Genome Diversity Panel. This sample is a global-scale sampling of human diversity with 52 populations in total.

*Adapted from a workshop originally created by John Novembre. This document is included as part of the Population Genetics workshop packet for the BSD qBio Bootcamp, University of Chicago, 2025. **Current version**: August 11, 2025.

The data were first described in Li et al (Science, 2008) and the raw files are available from the following link: http://hagsc.org/hgdp/files.html. These data have been used in numerous subsequent publications (e.g Pickrell et al, Genome Research, 2009) and are an important reference set. A few technical details are that the genotypes were filtered with a GenCall score cutoff of 0.25 (a quality score generated by the basic genotype calling software). Individuals with a genotype call rate <98.5% were removed, with the logic being that if a sample has many missing genotypes it may be due to poor quality of the source DNA, and so none of the genotypes should be trusted. Beyond this, to prepare the data for the workshop, we have filtered down the individuals to a set of 938 unrelated individuals. (For those who are interested, the data are available as plink-formatted files `H938.bed` `H938.fam`, `H938.bim` from this link: http://bit.ly/1aluTln). We have also extracted the basic counts of three possible genotypes.

The files with these genotype frequencies are your starting points.

*D. melanogaster*

*Note about logistics*

We will use some functions from the `dplyr` and `ggplot2` and `reshape2` libraries so first let's load them:

```
library(dplyr)
library(ggplot2)
library(reshape2)
```

If you get an error message saying there is no package titled `dplyr`,`ggplot2`, or `reshape2` you may need to first run `install.packages("dplyr")`, `install.packages("ggplot2")`, or `install.packages("reshape2")` to install the appropriate package.

We will not be outputting files - but you may want to set your working directory to the `sandbox` sub-directory in case you want to output some files.

The `MBL_WorkshopJJB.Rmd` file has the R code that you can run. Code is provided for most steps, but some you will need to devise for yourselves to answer the questions that are part of the workshop narrative.

*Initial view of the data*

Read in the data table:

```
g_raw <- read.table("../data/H938_chr15.geno", header=TRUE)
```

It will be read in as a dataframe in R.

Then use the "head" command to see the beginning of the dataframe:

```
head(g_raw)
```

You should see that there are columns each with distinct names.

```
CHR        SNP A1 A2 nA1A1 nA1A2 nA2A2
```

- CHR: The chromosome number. In this case all SNPs are from chromosome 15.
- SNP: The rsid of a SNP is a unique identifier for a SNP and you can use the rsid to look up information about a SNP using online resource such as dbSNP or SNPedia.
- A1: The minor allele at the SNP.
- A2: The major allele.
- nA1A1 : The number of A1/A1 homozygotes.
- nA1A2 : The number of A1/A2 heterozygotes.
- nA2A2 : The number of A2/A2 homozygotes.

*Calculate the number of observations at each locus*

Next compute the total number of observations by summing each of the three possible genotypes. Here we use the `mutate` function from the `dplyr` library to do the addition and add a new column to the dataframe in one nice step. Note: You could also subset the `nA1A1`, `nA1A2` and `nA2A2` columns and then use the `rowSums` function from the base R library. If you have time, you could try both versions and check that they give the same result.

```
g_raw <- mutate(g_raw, nObs = nA1A1 + nA1A2 + nA2A2)
```

Run `head(g)` and confirm your dataframe g has a new column called `nObs`.

Now use the `summary` function to print a simple summary of the distribution:

```
summary(g_raw$nObs)
```

The `ggplot2` library has the ability to make "quick plots" with the command `qplot`. If we pass it a single column it will make a histogram of the data for that column. Let's try it:

```
qplot(nObs, data = g_raw)
```

Our data are from 938 individuals. When the counts are less than this total, it's because some individuals had array data that was difficult to call a genotype for and so no genotype was reported.

**Question:** Do most of the SNPs have complete data?
**Question:** Why might some SNPs have incomplete data?
**Question:** What proportion of SNPs have a missingness rate greater than 1.5%? Eliminate these SNPs from the dataset.

```
# Compute missingness rate
g_raw <- g_raw %>% mutate(missRate = 1  - (nObs / max(nObs)))

# Find the proportion of SNPs with missingness rate > 1.5%

# Filter based on missingness rate
g <- g_raw %>% filter(missRate < 0.015)
```

*X. laevis*

*Calculating genotype and allele frequencies*

Let's move on to calculating genotype and allele frequencies. For allele $A_1$ we will denote its frequency among all the samples as $p_1$, and likewise for $A_2$ we will use $p_2$.

```
# Compute genotype frequencies
g <- mutate(g, p11 = nA1A1/nObs , p12 = nA1A2/nObs, p22 = nA2A2/nObs )
# Compute allele frequencies from genotype frequencies
g <- mutate(g, p1 = p11 + 0.5*p12, p2 = p22 + 0.5*p12)
```

**Question:** With a partner or group member, discuss whether the equations in the code for $p_1$ and $p_2$ are correct and if so, why?

Run `head(g)` again and confirm that `g` now has the extra columns for the genotype and allele frequencies.

And let's plot the frequency of the major allele (A2) vs the frequency of the minor allele (A1). If we pass the `qplot` command two columns, it will plot them against one another. Let's try it here:

```
qplot(p1, p2, data=g)
```

Notice that $p_2 > p_1$ (be careful to inspect the axes labels here) This makes sense because $A_1$ is supposed to be the minor (less frequent) allele. Note also that there is a linear relationship between $p_2$ and $p_1$

**Question:** What is the equation describing this relationship?

The relationship exists because there are only two alleles - and so their proportions must sum to 1. The linear relationship you found exists because of this constraint. It also provides a nice check on our work (if $p_1$ and $p_2$ didn't sum to 1 it would suggest something is wrong with our code!).

*Plotting genotype on allele frequencies*

Let's look at an initial plot of genotype vs allele frequencies. We could use the base plotting functions, but the following uses the `ggplot2` commands. These are a little trickier, but end up

being very compact (we need fewer lines of code overall to achieve our desired plot). To use `ggplot2` commands effectively our data need to be what statisticians call "tidy" (in this case, that means with one row per point we will plot).

To do this, we first subset the data on the columns we'd like (using the `select` command and listing the set of columns we want), then we pass this (using the %>% operator) to the `melt` command which will reformat the data for us, and output it as `gTidy`:

```
gTidy <- select(g, c(p1,p11,p12,p22)) %>% melt(id='p1',value.name="Genotype.Proportion")
head(gTidy)
ggplot(gTidy) + geom_point(aes(x = p1,
                               y = Genotype.Proportion,
                               color = variable,
                               shape = variable))
```

Now let's look at the graph that we produced. There is some scatter in the relationship between genotype proportion and allele frequency for any given genotype, but at the same time there is a very regular underlying relationship between these variables.

**Question:** What are approximate relationships between $p_{11}$ vs $p_1$, $p_{12}$ vs $p_1$, and $p_{22}$ vs $p_1$? (Hint: These look like parabolas, which suggests are some very simple quadratic functions of $p_1$).

You might start to recognize that these are the classic relationships that are taught in introductory biology courses. If you recall, under assumptions that there is no mutation, no natural selection, infinite population size, no population substructure and no migration, then the genotype frequencies will take on a simple relationship with the allele frequencies. That is: $p_{11} = p_1^2$, $p_{12} = 2p_1(1-p_1)$ and $p_{22} = (1-p_1)^2$. In your basic texts, they typically use $p$ and $q$ for the frequencies of allele 1 and 2, and present these *Hardy-Weinberg proportions* as: $p^2$,$2pq$, and $q^2$.

Another way to think of the Hardy-Weinberg proportions is in the following way. If the state of an allele ($A_1$ vs $A_2$) is *independent* within a genotype, then the probability of a particular genotype state (such as $A_1A_1$) will be determined by taking the product of the alleles within it (so $p_{11} = p_1p_1$ or $p_1^2$).

Let's add to the plot lines that represent Hardy-Weinberg proportions:

```
ggplot(gTidy)+
  geom_point(aes(x=p1,y=Genotype.Proportion,color=variable,shape=variable))+
  stat_function(fun=function(p) p^2, geom="line", colour="red",size=2.5) +
  stat_function(fun=function(p) 2*p*(1-p), geom="line", colour="green",size=2.5) +
  stat_function(fun=function(p) (1-p)^2, geom="line", colour="blue",size=2.5)
```

On average, the data follow the classic theoretical expectations fairly well. It is pretty remarkable that such a simple theory has some bearing on reality!

By eye, we can see that the fit isn't perfect though. There is a systematic deficiency of heterozygotes and excess of homozygotes. Why?

Let's look at this more closely and more formally...

*E. coli*

*Testing Hardy Weinberg*

Pearson's $\chi^2$-test is a basic statistical test that can be used to see if count data $o_i$ conform to a particular expectation. It is based on the $X^2$-test statistic:

$$X^2 = \sum_i \frac{(o_i - e_i)^2}{e_i}$$

which follows a $\chi^2$ distribution under the null hypothesis that the data are generated from a multinomial distribution with the expected counts given by $e_i$.

Here we compute the test statistic and obtain its associated p-value (using the `pchisq` function). We keep in mind that there is 1 degree of freedom (because we have 3 observations per SNP, but then they have to sum to a single total sample size, and we have to use the data once to get the estimated allele frequency, which reduces us down to 1 degree of freedom).

```
g <- mutate(g, X2 = (nA1A1-nObs*p1^2)^2 /(nObs*p1^2) +
             (nA1A2-nObs*2*p1*p2)^2 / (nObs*2*p1*p2) +
             (nA2A2-nObs*p2^2)^2 / (nObs*p2^2))
g <- mutate(g,pval = 1-pchisq(X2,1))
```

*The problem of multiple testing*

Let's look at the p-values for the first SNPs:

```
head(g$pval)
```

How should we interpret these? A p-value gives us the frequency at which the observed departure from expectations (or a more extreme departure) would occur if the null hypothesis that SNPs follows Hardy-Weinberg proportions is true. As an agreed upon standard (of the frequentist paradigm for statistical hypothesis testing), if the observation is relatively rare under the null (e.g. p-value < 5%), we reject the null hypothesis, and we would infer that the given SNP departs from Hardy-Weinberg expectations. This is problematic here though. The problem is that we are testing many, many SNPs (Use `dim(g)` to remind yourself how many rows/SNPs are in the dataset). Even if the null is universally true, we would expect to reject 5% of our SNPs using the standard frequentist paradigm. This is called the multiple testing problem. As an example, if we have 20,000 SNPs and the null hypothesis were true for all of them, on average we would naively reject the null for ~1000 SNPs based on the p-values < 0.05.

We clearly need some methods to deal with the "multiple testing problem". Two frameworks are the Bonferroni approach and false-discovery-rate (FDR) approaches. We will not say more about

these here. Instead, we will do two simple checks to see though if our data are globally consistent with the null.

First, let's see how many tests have p-values less than 0.05. Is it much larger than the number we'd expect on average given the total number of SNPs and a 5% rate of rejection under the null?

```
sum(g$pval < 0.05, na.rm = TRUE)
```

Wow - we see many more. This is our first sign that although by eye these data show qualitative similarities to HW, statistically they are not fitting Hardy-Weinberg well enough.

Let's look at this another way. A classic result from Fisher is that under the null hypothesis the p-values of a well-designed test should be distributed uniformly between 0 and 1. What do we see here?

```
qplot(pval, data = g)
```

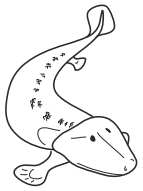
The data show an enrichment for small p-values relative to a uniform distribution. Notice how the whole distribution is shifted towards small values - The data appear to systematically depart from Hardy-Weinberg.

*Plotting expected vs observed heterozygosity*

To understand this more clearly, let's make a quick plot of the expected vs observed heterozygosity (the proportion of heterozygotes):

```
qplot(2*p1*(1-p1), p12, data = g) + geom_abline(intercept = 0,
                                                slope=1,
                                                color="red",
                                                size=1.5)
```

Most of the points fall below the y=x line. That is, we see a systematic deficiency of heterozygotes (and this implies a concordant excess of homozygotes). This general pattern is contributing to the departure from HW seen in the $X^2$ statistics.



*T. roseae*

*Discussion: Population subdivision and departures from Hardy-Weinberg expectations*

We might wonder why the departure from Hardy-Weinberg proportional is directional, in that, on average, we are seeing a deficiency of heterozygotes (and excess of homozygotes). One enlightening way to understand this is by thinking about what Sewall Wright (an eminent former

University of Chicago professor) called "the correlation of uniting gametes". To produce an $A_1A_1$ individual we need an $A_1$-bearing sperm and an $A_1$-bearing egg to unite. If these events were independent of each other, we would expect $A_1A_1$ individuals at the rate predicted by multiplying probabilities, that is, $p_1^2$ (an idea we introduced above). However, what if uniting gametes are positively correlated, in that an $A_1$-bearing sperm is more likely to join with an $A_1$-bearing egg? In this case we will have more $A_1A_1$ individuals than predicted by $p_1^2$, and conversely fewer $A_1A_2$ individuals than predicted by $2p_1p_2$. If our population is structured somehow such that $A_1$ sperm are more likely to meet with $A_1$ eggs, then we will have such a positive correlation of uniting gametes, and the resulting excess of homozygotes and deficiency of heterozygotes.

Given the HGDP data is from 52 sub-populations from around the globe, and alleles have some some probability of clustering within populations, a good working hypothesis for the deficiency of heterozygotes in this dataset is the presence of some population structure.

While statistically significant, the population structure appears to be subtle in absolute terms — based on our plots, we have seen the genotype proportions are not wildly off from Hardy-Weinberg proportions.

**Question:** As an exercise, compute the average deficiency of heterozygotes relative to the expected proportion. This is the average of

$$\frac{2p_1(1-p_1) - p_{12}}{2p_1(1-p_1)}$$

What is this number for this data-set? A common "rule-of-thumb" for this deficiency in a global sample of humans is approximately 10%. Do you find this to be true from the data?

Just ~10% difference between expected and observed seems pretty remarkable given these samples are taken from across the globe. It is a reminder that human populations are not very deeply structured. Most of the alleles in the sample are globally widespread and not sufficiently geographically clustered to generate correlations among the uniting alleles. This is because all humans populations derived from an ancestral population in Africa around 100-150 thousand years ago, which is relatively small amount of time for variation across populations to accumulate.

*Finding specific loci that show large departure from Hardy-Weinberg proportions*

Now, let's ask if we can find any loci that are wild departures from Hardy-Weinberg proportions. These might be loci that have erroneous genotypes, or loci that cluster geographically in dramatic ways (such that they have few heterozygotes relative to expectations).

To find these loci, we'll compute the same relative deficiency you computed above, but let's look at it per SNP. This number is referred to as $F$ by Sewall Wright and has connections directly to correlation coefficients (advanced exercise: Try to work this out!). If we assume there is no inbreeding within populations, this number is an estimator of $F_{ST}$ (a quantity that appears often in population genetics as a measure of the degree of structure between populations).

Let's plot how it's value changes across the chromosome from one end to another:

```
qplot(y = Fstat, data = g, xlab = "SNP Number")
```

There are a few interesting SNPS that show either a very high or low $F$ value.

Now, here's a trick. When a high or low $F$ value is due to genotyping error, it likely only effects

a single SNP. However, when there is some population genetic force acting on a region of the genome, it likely effects multiple SNPs in the region. So let's try to take a local average in a sliding window of SNPs across the genome, computing an average $F$ over every 5 consecutive SNPs (in real data analysis we might use 100 kilobase or 0.1 centiMorgan windows).

The `stats::filter` command below calls the `filter` function from the `stats` library. The code instructs the function to take 5 values centered on a focal SNP, weighting them each by 1/5 and then taking the sum. In this way it produces a local average in a sliding window of 5 SNPs. Let's define the `movingavg` function and then make a plot of its values:

```
movingavg <- function(x, n=5){stats::filter(x, rep(1/n,n), sides = 2)}
avgF <- movingavg(g$Fstat)
qplot(x = seq(1, length(avgF)), y = avgF,  xlab = "SNP number", geom = "line")
```

Wow — there appears to be one large spike where the average $F$ is approximately 60% in the dataset!

Let's extract the SNP id for the largest value, and look at the dataframe:

```
outlier=which (movingavg(g$Fstat) == max(movingavg(g$Fstat),na.rm=TRUE))
outlier=which.max(avgF)
g[outlier,]
```

**Question:** Which SNP is returned? By inserting the rs id into the UCSC genome browser (https://genome.ucsc.edu/), and following the links, find out what gene this SNP resides near. The gene names should start with "SLC.." What gene is it?

**Question:** Carry out a literature search on this gene using the term "positive selection" and see what you find. It's thought the high $F$ value observed here is because natural selection led to a geographic clustering of alleles in this gene region. Discuss with your partners why this might or might not make sense.

*C. elegans*

*Discussion: The outlier region*

The region you've found is one of the most differentiated between human populations that is known. Notice in your literature search, how it is known to affect skin pigmentation and is thought to contribute to differences in skin pigmentation that are seen between human populations. Finding strong population structure for alleles that affect external morphological phenotypes is not uncommon when looking at other chromosomes. Some of the most differentiated genes that exist in humans are those that involve morphological phenotypes - such as skin pigmentation, hair color/thickness, and eye color (the genes OCA2/HERC2, SCL45A2, KITLG, EDAR all come to mind). Many of these are thought to have arisen due to direct or indirect effects of adaptation to local selective pressures (e.g. adaptation to varying levels of UV exposure, local pathogens, local diets, local mating preferences), though in most cases we still do not yet have a fully convincing understanding of their evolutionary histories. Regardless of the reasons, it is notable that

many of the features that humans see externally in each other (i.e. the morphological differences) are controlled by genes that are outliers in the genome. At most variant SNPs, the patterns of variation are much closer to those of a single random mating populations than they are at variant sites like EDAR. Put another way, a genomic perspective shows us many of the differences people see in each other are in a sense, just skin-deep.

*Wrap-up*

Modern population genetics has a lot of additional tools on its workbench, but here using relatively simple and classical ideas combined with genomic-scale data, we have been able to observe and interpret some major features of human genetic diversity. We have also revisited some basic concepts of probability and statistics such as independence vs correlation, the $\chi^2$ test, and the problems of multiple testing. One remarkable thing we saw is that a very simple mathematical model based on assuming independence of alleles and genotypes can predict genotype proportions within ~10% of the true values. This gives us a hint of how simple mathematical models may be useful even in the face of biological complexity. Finally, we have gained more familiarity with R. We didn't discuss how genotyping errors might create Hardy-Weinberg departures, but if we were doing additional analyses, we could use Hardy-Weinberg departures to filter them from our data. It's common practice to do so, but with a Bonferonni correction and using data from within populations to do the filtering.

*Follow-up activities*

In the `data` folder, we are including data files that you can explore to gain more experience. These include global data for other chromosomes (`H938_chr*.geno`) and the same data but limited to different continental samples (`H938_*_chr*.geno`). Here are a few suggested follow-up activities. It may be wise to split the activities across class members and reconvene after carrying them out.

**Follow-up activity**: Look at chromosome 15 from your groups dataset - is the deficiency in heterozygosity as strong as it was on the global scale? Before you begin, what would you expect to see?

**Follow-up activity** Using your dataset, do you find any regions of the genome that are outliers for $F$ on your chromosome? Using genome browsers and/or literature searches, can you find what is the likely locus under selection for that region? Be ready to share what you learned with the class.

*References*

Li, Jun Z, Devin M Absher, Hua Tang, Audrey M Southwick, Amanda M Casto, Sohini Ramachandran, Howard M Cann, et al. 2008. "Worldwide Human Relationships Inferred from Genome-Wide Patterns of Variation." Science 319 (5866): 1100–1104.

Pickrell, Joseph K, Graham Coop, John Novembre, Sridhar Kudaravalli, Jun Z Li, Devin Absher, Balaji S Srinivasan, et al. 2009. "Signals of Recent Positive Selection in a Worldwide Sample of Human Populations." Genome Research 19 (5): 826–37.

# Movement analysis workshop

**Jasmine Nirody and Kiley Kellum** *University of Chicago*

In this workshop, we'll work with movement trajectories in the form of a time course of (x,y) coordinates. We'll provide an overview of some basic image analysis and concepts in biophysics and will get some experience working in ImageJ, R, and Python.

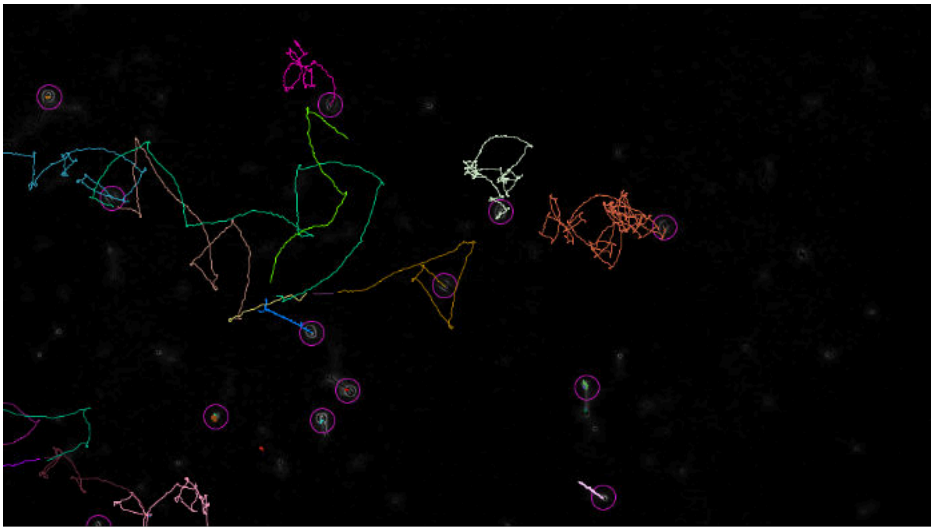
## Introduction

Trajectory data is ubiquitous across biology: living systems are dynamic and we can learn a lot from understaning their movement! If you're working with trajectories, you usually start with a video taken via a microscope or lab camera or drone, depending on your system of interest.

We'll spend a little time before we get to analyzing some trajectory data in R figuring out how we get from a video of cells moving around (in not-always-great contrast images!)...

[If you look carefully at this freeze frame image from the microscope you'll see several flagellated cells scattered around the field of view.]

...to trajectories that provide us with $(x, y)$ positions for our objects of interest over a particular time course.



We'll go through this process together using the TrackMate plugin in ImageJ. You should have already installed FIJI, which includes all the functionality needed for this workshop. If you have any issues, let us know before we start!

### Visualizing and analyzing trajectories in R

Trajectories are simplifications of a real path traveled by an object (animal, cell, molecule). Usually (and also for the purposes of this workshop!) this comes in the form of 2-dimensional spatial coordinates (x,y) with a third temporal dimension (t).

*Loading and plotting trajectory data*

We'll first load an example trajectory, taken from one of the cells in the video we analyzed at the start of this workshop.

[All the data files we'll work with are in the data folder, so we will set our working directory accordingly.]

```
coords <- read.csv("data/species1_track.csv",header=TRUE)
```

Before we do anything else, let's quickly visualize this trajectory for a sanity check and to see what we're working with.

```
plot(coords$POSITION_X,coords$POSITION_Y, type='l', ylab='y', xlab='x')
```

*Basic trajectory features – step lengths, turn angles, speeds*

A trajectory can also be thought of as a series of steps, each comprised of a length ($L_i$), a turning angle ($\Delta_i$), and a time. Between two points $(x_i, y_i)$ and $(x_{i+1}, y_{i+1})$, we can calculate the step length $L_i$ from the distance formula $L_i = \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}$.

***Exercise:*** Using the $(x, y, t)$ data stored in the `coords` variable, calculate out a vector of step lengths for each timestep in the trajectory. Store the outputted list as `step_lengths`. What length should this vector be?

***Exercise:*** Using the $(x, y, t)$ data stored in the `coords` variable, calculate out a vector of step lengths for each timestep in the trajectory. How would you calculate this value from this positional data? Store the outputted vector as `angles`.

We can also do some slightly more complex computations to understand a bit more about the underlying dynamics. First, let's look at how the object's speed varies over the trajectory. We can calculate the 'instantaneous' speed at a time $t_i$ by $speed_i = |\frac{L_i}{(t_i - t_{i-1})}|$. Note: if our trajectory is measured at constant time intervals, then $t_i - t_{i-1}$ is the same for all $i$.

***Exercise:*** Using the $(x, y, t)$ data stored in the `coords` variable, calculate out a list of speeds for each timestep in the trajectory. Store the outputted list as a variable `speeds`.

***Exercise:*** Using the information in `speeds`, calculate out various metrics that might give you some information about the overall trajectory like the mean speed, max speed, standard deviation in speed, etc.

*Working with the `trajr` package*

It's incredibly useful to know how to do some basic analysis like we went through, but one nice thing about languages like R is having a lot of packages that have a lot of built-in functions that can do some of these computations for us! Here we'll work with the `trajr` package so let's load this before we begin.

```
library(trajr)
```

If you get an error message, you might need to install the packages. You can do so using the command `install.packages("trajr")`.

The file `QBio_NirodyWorkshop.Rmd` has R code that you can run, but we'll go through most of the steps here as well.

The `trajr` package represents trajectories as R **objects** in the class `Trajectory`. To create a `Trajectory` object, we use the `TrajFromCoords` function from a set of x-y coordinates and times. For our purposes, we can load these trajectories into R from .csv files, where the first column is the x-coordinates, the second column is the y-coordinates, and the third column is times. [Note that times are an optional input for `trajr` Trajectories.]

We'll use the data we loaded into `coords` and convert that into a `trajr Trajectory`.

```
trj <- TrajFromCoords(coords, spatialUnits='pixels', timeUnits = 'frames')
plot(trj)
```

Using some built-in functions, we can also get out information on the timecourse of the trajectory, including the distribution of turning angles and step lengths like we did earlier

```
trajr_angles <- TrajAngles(trj)
trajr_steplengths <- TrajStepLengths(trj)
```

***Exercise:*** How do these values compare to the ones we calculated by hand earlier? If you find they don't match, what went wrong?

Let's visualize the distribution of step lengths and angles.

***Exercise:*** What might be the best and most informative kind of plot for these data? Why? Use different kinds of plots and see what you get out of them.

Within the `trajr` package, we can also use `TrajVelocity` and `TrajAcceleration` to estimate these metrics at each point along a trajectory. The `TrajDerivatives` function calculates speed and change in speed along a trajectory. [If we find that the trajectory is very noisy, we can smooth it using the `TrajSmooth` function before calculating these derivatives.]

```
# Calculate speed and acceleration
derivs <- TrajDerivatives(trj)
```

Let's compare the speeds calculated by the `TrajDerivatives` function to the speeds we calculated from scratch above. Use the commands `mean(derivs$speed)`, `max(derivs$speed)`, `min(derivs$speed)`, `sd(derivs$speed)` to compare the values to those computed before.

Let's also plot the two speed calculations side by side.

***Question:*** What information might a full timecourse of speeds provide us that the overall descriptive metrics might not?

*Working with and manipulating R objects*

Note that when we loaded our data into `coords` and eventually into a `trajr Trajectory`, we were working with spatial coordinates in pixels and time measured in frames. Before we do any meaningful analysis, it's useful to convert these into more physically meaningful units (especially when comparing different datasets, which may be obtained using different cameras!). The `TrajScale` function allows us to implement these meaningful scaling parameters to transform our trajectory object before analysis. When a trajectory is digitized from a video (like the one in our example!), we can calculate the appropriate spatial scaling factor as $width_m / width_p$, where $width_m$ is the width of an object in your chosen unit of length (e.g., meter, micron) and $width_p$ is the width of the same object in pixels, as measured from the video. Similarly, time in these trajectories can be appropriately scaled by knowing how many frames per second (fps) are in our recording.

Let's re-upload our example trajectory into `TrajFromCoords`.

```
trj <- TrajFromCoords(coords, fps= 50, spatialUnits='pixels', timeUnits = 's')
plot(trj)
```

Now, using the `TrajScale` function, we can scale the spatial units to our chosen ones (here, we are using $\mu$m):

```
trj <- TrajScale(trj, .45 / 1, "um")
plot(trj)
```

There are also many functions included in the `trajr` package that provide us with some relevant information about our trajectory objects, including some of the important scaling parameters we just learned to input.

```
TrajGetFPS(trj)
TrajGetTimeUnits(trj)
TrajGetUnits(trj)
TrajGetNCoords(trj)
```

We can also learn some basic information about our trajectory, including the total duration, total length, and total distance from start to end of the trajectory. [These would be pretty straightforward to compute even without the `trajr` package – try it! We'll come upon some more complex metrics later included within the package later.]

```
TrajDuration(trj)
TrajLength(trj)
TrajDistance(trj)
```

***Question:*** Why would you expect the values outputted from `TrajLength` to be different from those outputted from `TrajDistance`?

***Question:*** Would you expect the sum of all the step lengths to equal the `TrajLength` to be or `TrajDistance` value? Check using the `sum()` function!

*Some more analysis metrics*

The `trajr` package contains several methods to characterize the straightness of trajectories. The simplest one is $D/L$, where $D$ is the distance between the starting and ending points of the trajectory and $L$ is the length of the trajectory [***Note:*** We've talked about this before!] This is calculated by `TrajStraightness`, and the value outputted ranges from 0 to 1, where 1 means the trajectory is a straight line.

```
TrajStraightness(trj)
```

Another metric of straightness can be used by looking at the distribution of turning angles, as we calculated before. As noted in Batschelet (1981), this metric can be calculated by calling `Mod(TrajMeanVectorOfTurningAngles())`, assuming we are working with a Trajectory with constant step length.

***Exercise:*** Compare these metrics and discuss the pros and cons of each for different types of trajectories.

$E^a_{max}$ is a dimensionless estimate of the maximum expected displacement of a trajectory. Larger values of this parameter represent straighter paths (Cheung et al., 2007). $E^b_{max}$ is $E^a_{max}$ multiplied by the mean step length.

***Question:*** What does $E^b_{max}$ tell us?

We use the function `TrajEmax`.

```
TrajEmax(trj)
TrajEmax(trj, eMaxB = TRUE)
```

***Exercise:*** Using the `?TrajEmax` function, think about how `TrajEmax` differentiates between random and directed walks. Why might this be important?

The direction autocorrelation function quantifies regularities within trajectories, for instance wave-like periodicities. From this function, we can get an idea of their wavelength and amplitude. This has, in previous studies been used to detect, for example, the winding movements of trail-following ants (Shamble et al., 2017).

The function `TrajDirectionAutocorrelations` calculates the differences in step angles at all steps separated by $\Delta$, for a range of values of $\Delta$.The position of the first local minimum of this function may be used to characterise the periodicity within a trajectory. This position is calculated by `TrajDAFindFirstMinimum` (or `TrajDAFindFirstMaximum`).

***Question:*** : Some trajectories will not have a first local minimum (or maximum). What does this indicate?

```
corr <- TrajDirectionAutocorrelations(trj)
plot(corr)
```

## Working with multiple trajectories

*Loading multiple trajectories into trajr*

Most studies you'll work with will involve multiple trajectories (per individual, per species, or from multiple species!). Accordingly, the `trajr` package provides some functions to load and work with multiple trajectories. We'll work with some of these now, and then (as time permits) switch to play around with some exploratory analyses with various types of data.

```
tracks <- as.data.frame(rbind(
  c("data/species1_track.csv", "Allomyces macrogynus"),
  c("data/species2_track.csv", "Chytriomyces confervae"),
  c("data/species3_track.csv", "Allomyces_reticulatus"),
  c("data/species4_track.csv", "Rhizoclosmatium globosum"),
  c("data/species5_track.csv", "Synchyrtium microbalum"),
  c("data/species6_track.csv", "Blastocladiella emersonii")
), stringsAsFactors = FALSE)
colnames(tracks) <- c("filename", "species")
```

We can then use the function `TrajsBuild` to load these into `trajr`. `TrajsBuild` assumes that you have a set of trajectory files, each of which may have a scale and a frames-per-second value. The function reads each of the files specified in a list of file names, optionally using a custom function, then passes the result to `TrajFromCoords`. Remember that, with no input given, this function assumes that the first column is x, the second is y, and there is no time column. We can use the `csvStruct` argument identify the x, y and time columns if needed.

```
csvStruct <- list(x = 1, y = 2, time = 3)
trjs <- TrajsBuild(tracks$filename, scale = .45 / 1,
                   spatialUnits = "um", timeUnits = "s",
                   csvStruct = csvStruct)
```

*Getting stats out from multiple trajectories*

The function `TrajsMergeStats` simplifies the construction of a data frame of values, with one row per trajectory. To use it, you need a list of trajectories (which you can get from calling `TrajsBuild`), and a function which calculates the statistics of interest for a single trajectory.

```r
# Define a function which calculates some statistics
# of interest for a single trajectory

characterizeTrajectory <- function(trj) {
  # Measures of speed
  derivs <- TrajDerivatives(trj)
  mean_speed <- mean(derivs$speed)
  sd_speed <- sd(derivs$speed)

  # Measures of straightness
  straightness<- TrajStraightness(trj)
  Emax <- TrajEmax(trj)

  # Periodicity
  corr <- TrajDirectionAutocorrelations(trj)
  first_min <- TrajDAFindFirstMinimum(corr)

  # Return a list with all of the statistics for this trajectory
  list(mean_speed = mean_speed,
       sd_speed = sd_speed,
       Emax = Emax,
       min_deltaS = first_min[1],
       min_C = first_min[2]
  )
}
```

Now that you've defined this function with the statistics of interest, you can pass it onto `TrajMergeStats` to cycle through all the trajectories stored in `trjs`.

```r
# Calculate all stats for trajectories in the list
# which was built in the previous example
stats <- TrajsMergeStats(trjs, characterizeTrajectory)
print(stats)
```

***Exercise:*** Play around with some variants on the `characterizeTrajectory` function!

## Follow-up activities

We've loaded a set of trajectory data into the `data` subfolder; these correspond to 6 trajectories from 6 different species of single-celled fungal zoospores.

Using both visualization and some of the quantitative metrics we discussed today, what can be said about the movement of these species? Are there some patterns that arise? What similarities or differences between the movement patterns do you observe? How might you quantify these – what metrics are informative and which ones are not in this particular example?

We encourage you to do this exploration in R, Python, or another programming language you feel comfortable in! Use a combination of your own analyses and built-in functions to explore.

## Acknowledgments

This workshop contains online materials written by Jim McLean.

## References

Batschelet, E. (1981). Circular statistics in biology. ACADEMIC PRESS, 111 FIFTH AVE., NEW YORK, NY 10003, 1981, 388.

Cheung, A., Zhang, S., Stricker, C., & Srinivasan, M. V. (2007). Animal navigation: the difficulty of moving in a straight line. Biological Cybernetics, 97(1), 47-61.

Galindo, L. J., Richards, T. A., & Nirody, J. A. (2023). Fungal zoospores show contrasting swimming patterns specific to phylum and cytology. bioRxiv, 2023-01.

Shamble, P. S., Hoy, R. R., Cohen, I., & Beatus, T. (2017). Walking like an ant: a quantitative and experimental approach to understanding locomotor mimicry in the jumping spider Myrmarachne formicaria. Proceedings of the Royal Society B: Biological Sciences, 284(1858).